CryptoExperts

WE INNOVATE TO SECURE YOUR BUSINESS

Audit of the Garaga Library

Date	June 27, 2025
Version	2.0
Page count	96
Authors	Thibauld Feneuil Matthieu Rivain

Contents

1 Introduction **4**
- 1.1 Context . . . 4
- 1.2 Audited code and documentation . . . 4
- 1.3 Methodology and summary of findings . . . 4
- 1.4 Structure of the report . . . 8

2 Documentation **9**
- 2.1 Background on elliptic curves . . . 9
  - 2.1.1 Group structure . . . 9
  - 2.1.2 Basic operations . . . 10
  - 2.1.3 Supported elliptic curves . . . 11
- 2.2 Elliptic curve inner product (ECIP) . . . 13
  - 2.2.1 Background on divisors and Weil reciprocity . . . 14
  - 2.2.2 Proving a sum of points . . . 15
  - 2.2.3 Using logarithmic derivatives . . . 16
  - 2.2.4 Proving an ECIP statement . . . 17
  - 2.2.5 Splitting scalars . . . 20
  - 2.2.6 Algorithmic description . . . 22
  - 2.2.7 Soundness of the ECIP check . . . 25
- 2.3 Pairings . . . 27
  - 2.3.1 Pairing definitions . . . 27
  - 2.3.2 Multi-pairing check . . . 29
  - 2.3.3 Multi Miller loop . . . 30
  - 2.3.4 Randomized field arithmetic . . . 35
  - 2.3.5 Detailed Miller loop iterations . . . 37
  - 2.3.6 Detailed multi-pairing check . . . 39
- 2.4 Auxiliary functionalities . . . 48
  - 2.4.1 Digital signature schemes . . . 48
  - 2.4.2 Groth16 . . . 51
- 2.5 Code architecture . . . 52

3 Code review **55**
- 3.1 Definitions . . . 55
  - 3.1.1 File `structs/fields.cairo` . . . 55
  - 3.1.2 File `structs/points.cairo` . . . 58
  - 3.1.3 File `curves.cairo` . . . 59
  - 3.1.4 File `structs.cairo` . . . 64
  - 3.1.5 File `definitions.cairo` . . . 64
- 3.2 Arithmetic operations . . . 65
- 3.3 Elliptic curves . . . 66
  - 3.3.1 File `ec_ops.cairo` . . . 66
  - 3.3.2 File `ec_ops_g2.cairo` . . . 70

3.3.3 File `pairing/pairing_check.cairo` 72
3.3.4 File `pairing/groth16.cairo` 77
3.3.5 File `pairing/single_pairing_tower.cairo` 80
3.4 Signatures 84
3.4.1 File `schnorr.cairo` 84
3.4.2 File `ecdsa.cairo` 86
3.5 Utilities 88
3.5.1 File `utils.cairo` 88
3.5.2 File `hashes.cairo` 88
3.5.3 File `utils/hashing.cairo` 88
3.5.4 File `utils/neg_3.cairo` 90
3.5.5 File `utils/calldata.cairo` 92

A Attack test vector **95**

1 Introduction

1.1 Context

StarkNet is a decentralized, permissionless Layer 2 scaling solution for Ethereum, designed to improve scalability without compromising security or decentralization. It is built using STARKs (Scalable Transparent ARguments of Knowledge), a cryptographic technology that allows for verifying the correctness of computations without re-executing them. StarkNet uses its own programming language, Cairo, to write smart contracts. Cairo is designed for writing provable programs that can be efficiently executed and verified using STARKs.

Garaga is an advanced cryptographic library designed for StarkNet, focusing on efficient elliptic curve and pairing operations, enabling the verification of some SNARKs (Succinct Non-interactive Arguments of Knowledge) such as Groth16. The StarkNet Foundation commissioned CryptoExperts to perform an audit of the Garaga library. This report presents the findings and results of this audit.

1.2 Audited code and documentation

The audited code is composed of the Cairo source files from the following GitHub repository:

`https://github.com/keep-starknet-strange/garaga/tree/v0.15.4/src/src`,

excluding files which are auto-generated and those in the folder `utils` that are related to a specific use case. The audited version of the code corresponds to the tag "v0.15.4", from February 13th, 2025.

While the auto-generated files are out of scope, the audited code further includes the Hydra backend (Python) generating these files, which is available at:

`https://github.com/keep-starknet-strange/garaga/tree/v0.15.4/hydra/garaga/precompiled_circuits`.

The provided documentary material is composed of scientific articles on the ECIP check [Eag22b] and the pairing check [NE24] as well as the following blog post on extension field arithmetic: `https://hackmd.io/@feltroidprime/B1eyHHXNT`. The README of the Garaga repository further provides a general overview of the library.

1.3 Methodology and summary of findings

The main goal of this audit is to validate the soundness of the reviewed Cairo implementation of Garaga library. More precisely, this audit aims

- to confirm that the implemented elliptic curve operations are compliant with their intended purpose,

- to check the absence of soundness flaws in the implementation which would allow an adversary to convince a verifier that an invalid statement is valid.

The audit methodology consists in a code review by cryptography experts, confronting our understanding of the code and keeping track of our observations. The audit is composed of two phases:

- **First phase: design review.** We perform a first review of the code and the provided documentary material. Whenever necessary, we also have Q&A sessions with the developper. The purpose is gaining a global understanding of the library design choices, algorithms and code architecture. The output of this phase is a documentation of the Garaga library (core principles and algorithms) which is provided in Section 2 of the present report.
- **Second phase: in-depth code review.** Given the global understanding gained in Phase 1, we perform an in-depth code review, documenting each file and stating our observations. The output of this phase will be given in Section 3 in the next version of the present report.

Our observations are categorized as follows:

- Observations that may impact the soundness of the Garaga library, rated as
  - high risk (flagged ●),
  - medium risk (flagged ●),
  - low risk (flagged ●).
- Observations related to coding practices and implementation choices (flagged ●).
  - These observations do not translate into a direct risk on the soundness of the verifier but addressing them would make the code clearer, more efficient, and/or less prone to errors.
- Observations related to documentation, comments, variable naming (flagged ■).
  - These observations do not translate into a direct risk on the soundness of the Garaga library but addressing them would facilitate the understanding of the code by third parties (users, developers, auditors).

Each observation comes with an associated recommendation to fix or improve the underlying issue.

Summary of findings on the Garaga library. Our observations are listed below with references to the corresponding report pages. The following table summarizes the number of observations.

Category	Number of findings
High risk	6
Medium risk	5
Low risk	5
Coding practices	30
Documentation	18
Total	**64**
Questions	4

- **Question 1: Scalar decomposition** . . . 19
- **Observation 1: Soundness of ECIP batching with small co-factors** . . . 21
- **Observation 2: Soundness of the ECIP check** . . . 25
- **Observation 3: Batching coefficients in randomized field arithmetic** . . . 36
- **Question 2: Boolean in ECDSA signature** . . . 50
- **Question 3: Choice of `u288` over `u256`** . . . 55
- **Observation 4: Generic structure versus concrete structure** . . . 56
- **Observation 5: Definition location** . . . 57
- **Observation 6: Structure definition** . . . 57
- **Observation 7: Curve identification in `G1Point`** . . . 58
- **Observation 8: Curve identification in `G2Point`** . . . 59
- **Observation 9: Integer type in point structure definition** . . . 59
- **Observation 10: Incorrect comment** . . . 60
- **Observation 11: Incomplete comment** . . . 60
- **Observation 12: Function name** . . . 61
- **Observation 13: Hardcoded curve constants** . . . 62
- **Observation 14: Inconsistent error handling** . . . 62
- **Observation 15: Definition location** . . . 62
- **Observation 16: Redundant definition of the $\mathbb{G}_1$ generator** . . . 63
- **Observation 17: Hardcoded decomposition of the Miller-loop exponent** . . . 63
- **Observation 18: Unused casting** . . . 64
- **Observation 19: Redundant code** . . . 64
- **Observation 20: Code organisation** . . . 64
- **Observation 21: Application-specific arithmetic functions** . . . 66
- **Observation 22: Old commented code** . . . 66
- **Observation 23: Curve membership test** . . . 67
- **Observation 24: Duplicate code** . . . 67
- **Observation 25: ECIP usage in subgroup membership test** . . . 68

- Observation 26: Optimization in the random point derivation 69
- Observation 27: Safe EC addition 70
- Observation 28: Completeness of the $\mathbb{G}_2$ EC multiplication 71
- Observation 29: Hardcoded curve 72
- Observation 30: Non-standard definition of sponge-based XOF 73
- Observation 31: Invalid domain separation in XOFs 73
- Observation 32: Ambiguous function name 74
- Observation 33: Confusing variable usage 74
- Observation 34: Large number of function arguments 75
- Observation 35: First iterations of the Miller loop 75
- Observation 36: Code redundancy for batched iterations 76
- Observation 37: Heavy handling of the batching data 76
- Observation 38: Confusing variable names 77
- Observation 39: Non-standard definition of sponge-based XOF 78
- Observation 40: Non-standard domain separation in XOFs 78
- Observation 41: Double instances of the randomized field arithmetic . 78
- Observation 42: Redundant code 79
- Observation 43: Application-dependent structure 79
- Observation 44: ECIP usage in Groth16 proof verification 80
- Observation 45: Subgroup membership 80
- Observation 46: Function location 80
- Question 4: Unclear conjugaison operation 81
- Observation 47: Representation of $G_{\Phi_{12}(q)}$ elements 83
- Observation 48: Optimized squaring in the cyclotomic subgroup 83
- Observation 49: Undocumented assumption 84
- Observation 50: Incorrect structure comment 84
- Observation 51: Same structure for signature and public key 85
- Observation 52: ECIP usage in Schnorr verification 85
- Observation 53: Incorrect check for the point at infinity 85
- Observation 54: Useless instruction 85
- Observation 55: Same structure for signature and public key 86
- Observation 56: ECIP usage in ECDSA verification 86
- Observation 57: Incorrect check for the point at infinity 87
- Observation 58: Inconsistent comment 87
- Observation 59: Modular reduction function 87
- Observation 60: Function API 89
- Observation 61: Data structure for hash states 89
- Observation 62: Optimized hashing with `u288` 90
- Observation 63: Sanity check on the input basis-(-3) decompositions . 91
- Observation 64: Separation between utility and application-dependent functions 92

1.4 Structure of the report

This report is organized in two parts. First, it includes a documentation of the library along with our observations on its core design principles. A second part focuses on low-level code analysis.

2 Documentation

Garaga is a Cairo library for verifying elliptic curve and pairing operations. This section provides documentation for the library. Section 2.1 offers background on elliptic curves and lists all the curves supported by the audited version of Garaga. The library's two main components are described in Sections 2.2 and 2.3. Specifically, Section 2.2 presents Garaga's method for verifying elliptic curve inner products (ECIP), also known as multi-scalar multiplication. Section 2.3 reviews Garaga's pairing-related functions, particularly the multi-pairing check. Sections 2.4.1 and 2.4.2 cover additional components, namely signature schemes and the Groth'16 proof system. Finally, Section 2.5 gives an overview of the code architecture.

2.1 Background on elliptic curves

An *elliptic curve* E over a finite field $\mathbb{F}_q$ of characteristic $p > 3$ is defined by the following *short Weiertrass equation*:

$$E : y^2 = x^3 + ax + b \tag{1}$$

where $a, b \in \mathbb{F}_q$ and $\Delta = -16(4a^3 + 27b^2) \neq 0$ (Δ is called *discriminant* of E). For every extension K of $\mathbb{F}_q$, the K-rational point of E is the set $E(K)$ of points $(x, y) \in K \times K$ satisfying (1) together with the *point at infinity* denoted $\mathcal{O}$. The points in $E(\mathbb{F}_q)$ are simply called the *rational points of* E.

In the following, we will denote by $p_E \in \mathbb{F}_q[x, y]$ the polynomial in the equation of E:

$$p_E(x, y) = x^3 + ax + b - y^2 \ ,$$

which is such that $P \in E(K)\backslash\{\mathcal{O}\}$ if and only if $p_E(P) = 0$.

2.1.1 Group structure

Let E be an elliptic curve defined over $\mathbb{F}_q$. For every extension K of $\mathbb{F}_q$, the set of K-rational points can be equipped with an addition law:

$$\begin{aligned} E(K) \times E(K) &\to E(K) \\ (P, Q) &\mapsto P + Q \end{aligned}$$

which is associative and commutative, has identity element $\mathcal{O}$ and, for which every $P \in E(K)$, has an inverse $-P \in E(K)$. This implies that $(E(K), +, \mathcal{O})$ has a finite Abelian group structure. The addition law is given by the so-called *chord-and-tangent rule.* Geometrically, this rule says that drawing a line which intersets the curve at several points implies that the sum of those points equals the identity $\mathcal{O}$.

We denote by $[k]P$ the *scalar multiplication* of a point P by an integer k which is defined as:

$$[k]P = \underbrace{P + P + \cdots + P}_{k \text{ times}}.$$

Let us recall that the cardinal of a finite Abelian group $(\mathbb{G}, +, \mathcal{O})$ is also called its *order*, denote $\#\mathbb{G}$, and that the *order* of an element $P \in \mathbb{G}$ is the minimum positive integer n such that $[n]P = \mathcal{O}$.

We denote n the order of the elliptic curve. In cryptography, we work in a subgroup of primer order r, where r divides n. We call *cofactor* the quotient $h = n/r$. When the co-factor is 1, the subgroup is the whole curve, so any non-zero rational point is a generator. When the co-factor is not 1, the subgroup is a strict subset of the curve. In what follows, we denote G a generator of this subgroup.

2.1.2 Basic operations

We recall hereafter the two basic operations underlying the group structure, namely point addition and point doubling.

Point addition. Let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ with $P \neq Q$ and $P, Q \neq \mathcal{O}$. Let

$$\ell : y = \lambda x - \mu \tag{2}$$

be the line through P and Q. We have

$$\lambda = \frac{y_1 - y_2}{x_1 - x_2} \quad \text{and} \quad \mu = \lambda x_1 - y_1 \tag{3}$$

The intersection of ℓ and E is the set of points $(x, y) \in K \times K$ satisfying both (2) and (1). In particular, the x-coordinates of those points satisfies $(\lambda x - \mu)^2 = x^3 + ax + b$ which gives:

$$p(x) := x^3 - \lambda^2 x^2 + 2\mu\lambda x + ax + b - \mu^2 = 0 \tag{4}$$

Since $P, Q \in \ell \cap E$, x_1 and x_2 are both solution of the previous equation (i.e. they are roots of p) hence we deduce:

$$p(x) := (x - x_1)(x - x_2)(x - x_3) \tag{5}$$

where x_3 us the third root of p. Defining $y_3 = -(\lambda x_3 - \mu)$, we get $E \cap \ell = \{P, Q, (x_3, -y_3)\}$ and $P + Q = (x_3, y_3)$. Finally, we can efficiently compute $P + Q$, since from (4) and (5) we have $x_1 + x_2 + x_3 = \lambda^2$ which gives:

$$x_3 = \lambda^2 - x_1 - x_2 = \left(\frac{y_1 - y_2}{x_1 - x_2}\right)^2 - x_1 - x_2 \ ,$$

and:

$$y_3 = -\lambda x_3 + \mu = \lambda(x_1 - x_3) - y_1 = \left(\frac{y_1 - y_2}{x_1 - x_2}\right)(x_1 - x_3) - y_1 \ .$$

Point doubling. When $P = Q \neq \mathcal{O}$ the sum $P + Q = [2]P$ is computed similarly than when $P \neq Q$ except that we take the tangent of E at P instead of the line through P and Q. The slope of this tangent satsifies:

$$\lambda = \frac{\frac{\partial p_E}{\partial x}(x_1, y_1)}{\frac{\partial p_E}{\partial y}(x_1, y_1)} = \frac{3x_1^2 + a}{2y_1}$$

which gives

$$x_3 = \lambda^2 - 2x_1 = \left(\frac{3x_1^2 + a}{2y_1}\right)^2 - 2x_1$$

and:

$$y_3 = \lambda(x_1 - x_3) - y_1 = \left(\frac{3x_1^2 + a}{2y_1}\right)(x_1 - x_3) - y_1$$

2.1.3 Supported elliptic curves

We describe all the elliptic curves currently implemented in Garaga library. In the Garaga library, all the elliptic curves are defined over prime fields.

BN254 Curve. The Barreto-Naehrig [BN06] family of curves is a class of pairing-friendly curves (with embedding degree $k = 12$) of the form:

$$E : y^2 = x^3 + b \ .$$

The field and the order of a BN curve are respectively $q(\mathtt{z})$ and $r(\mathtt{z})$ for some integer $\mathtt{z}$, where $q(\cdot)$ and $r(\cdot)$ are defined as:

$$q(\mathtt{z}) = 36\mathtt{z}^4 + 36\mathtt{z}^3 + 24\mathtt{z}^2 + 6\mathtt{z} + 1$$
$$r(\mathtt{z}) = 36\mathtt{z}^4 + 36\mathtt{z}^3 + 18\mathtt{z}^2 + 6\mathtt{z} + 1.$$

The BN254 curve is a BN curve with $\mathtt{z} = 4965661367192848881$ and has equation $y^2 = x^3 + 3$. Its cofactor is 1. The following point $G = (x_G, y_G)$ is a valid generator of the curve:

$$x_G = \mathtt{0x2523648240000001BA344D8000000008612100000000013A700000000000012}$$
$$y_G = \mathtt{0x0000000000000000000000000000000000000000000000000000000000000001}.$$

BLS12-381 Curve. The Barreto-Lynn-Scott [BLS03] family of curves is a class of pairing-friendly curves (with embedding degree $k \in \{12, 24\}$) of the form:

$$E : y^2 = x^3 + b \ .$$

The field and the order of a BLS curve are respectively $q(\mathtt{z})$ and $r(\mathtt{z})$ for some integer $\mathtt{z}$, where $q(\cdot)$ and $r(\cdot)$ are defined as:

$$q(\mathtt{z}) = \frac{1}{3}(\mathtt{z} - 1)^2(\mathtt{z}^4 - \mathtt{z}^2 + 1) + \mathtt{z}$$
$$r(\mathtt{z}) = \mathtt{z}^4 - \mathtt{z}^2 + 1.$$

The BLS12-381 curve is a BLS curve with `z = -0xd201000000010000` and has equation $y^2 = x^3 + 4$. Its cofactor is

$$h = \texttt{0x396C8C005555E1568C00AAAB0000AAAB} = 3 \cdot 11^2 \cdot 10177^2 \cdot 859267^2 \cdot 52437899^2 \ .$$

The following point $G = (x_G, y_G)$ is a valid generator of the curve:

$$\begin{aligned} x_G &= \texttt{0x17F1D3A73197D7942695638C4FA9AC0FC3688C4F9774B90...} \\ &\qquad \texttt{...5A14E3A3F171BAC586C55E83FF97A1AEFFB3AF00ADB22C6BB} \\ y_G &= \texttt{0x08B3F481E3AAA0F1A09E30ED741D8AE4FCF5E095D5D00AF...} \\ &\qquad \texttt{...600DB18CB2C04B3EDD03CC744A2888AE40CAA232946C5E7E1.} \end{aligned}$$

Secp256k1 Curve. The Secp256k1 curve is a Koblitz curve with equation $y^2 = x^3 + 7$, for which the field order q, the prime subgroup size r and the cofactor h are

$$\begin{aligned} q &= 2^{256} - 2^{32} - 32 \cdot 2^6 - 2^4 - 1 \\ r &= 2^{256} - \texttt{0x14551231950B75FC4402DA1732FC9BEBF} \\ h &= 1. \end{aligned}$$

The following point $G = (x_G, y_G)$ is a valid generator of the curve:

$$\begin{aligned} x_G &= \texttt{0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798} \\ y_G &= \texttt{0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8.} \end{aligned}$$

Secp256r1 Curve. The Secp256r1 curve is an elliptic curve with equation $y^2 = x^3 + ax + b$, where

$$\begin{aligned} a &= 2^{256} - 2^{224} + 2^{192} + 2^{96} - 4 \\ b &= \texttt{0x5AC635D8AA3A93E7B3EBBD55769886BC651D06B0CC53B0F63BCE3C3E27D2604B,} \end{aligned}$$

for which the field order q, the prime subgroup size r and the cofactor h are

$$\begin{aligned} q &= 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1 \\ r &= 2^{256} - 2^{224} + 2^{192} - \texttt{0x4319055258E8617B0C46353D039CDAAF} \\ h &= 1. \end{aligned}$$

The following point $G = (x_G, y_G)$ is a valid generator of the curve:

$$\begin{aligned} x_G &= \texttt{0x6B17D1F2E12C4247F8BCE6E563A440F277037D812DEB33A0F4A13945D898C296} \\ y_G &= \texttt{0x4FE342E2FE1A7F9B8EE7EB4A7C0F9E162BCE33576B315ECECBB6406837BF51F5.} \end{aligned}$$

Ed25519 Curve. The Ed25519 curve is a TwistedEdwards curve with equation $a'x^2 + y^2 = 1 + d'x^2y^2$, where

$$a' = 2^{255} - 20$$
$$d' = \texttt{0x52036CEE2B6FFE738CC740797779E89800700A4D4141D8AB75EB4DCA135978A3},$$

for which the field order q, the prime subgroup size r and the cofactor h are

$$q = 2^{255} - 19$$
$$r = 2^{252} + \texttt{0x14DEF9DEA2F79CD65812631A5CF5D3ED}$$
$$h = 8.$$

This curve is birationnally equivalent to a elliptic curve defined by a short Weierstrass equation $y^2 = x^3 + ax + b$, where

$$a = \texttt{0x5D4EACD3A5B9BEE63197E10D617B3DD66BB8B65D0CA52AF7AC71E18EF8BC172D}$$
$$b = \texttt{0x1D11B29BCFD0B3E0550DDB06105780D5F54831976B9FBC329004EBC1F364B2A4},$$

The following point $G = (x_G, y_G)$ is a valid generator of the equivalent curve:

$$x_G = \texttt{0x2A78DD0FD02C0339F00B8F02F1C20618A9C13FDF0D617C9ACA55C89B025AEF35}$$
$$y_G = \texttt{0x29C644A5C71DA22EBE483BA563798323CF6FD061807131659B7830F3F62C1D14}.$$

Grumpkin Curve. The Grumpkin curve is an elliptic curve with equation $y^2 = x^3 - 17$, for which the field order q, the prime subgroup size r and the cofactor h are

$$q = \texttt{0x30644E72E131A029B85045B68181585D2833E84879B9709143E1F593F0000001}$$
$$r = \texttt{0x30644E72E131A029B85045B68181585D97816A916871CA8D3C208C16D87CFD47}$$
$$h = 1.$$

The following point $G = (x_G, y_G)$ is a valid generator of the curve:

$$x_G = 1$$
$$y_G = \texttt{0x2CF135E7506A45D632D270D45F1181294833FC48D823F272C}.$$

Let us stress that the prime field of the Grumpkin curve corresponds to the scalar field of the BN254 curve, namely q_{Grumpkin} is equal to r_{BN254}.

2.2 Elliptic curve inner product (ECIP)

An elliptic curve inner product (ECIP), a.k.a. multi-scalar multiplication, is the operation which from input points $P_1, \ldots, P_n \in E(\mathbb{F}_q)$ and input scalars $e_1, \ldots, e_n \in \mathbb{Z}$, outputs the point Q defined as:

$$Q = \sum_{i=1}^{n} e_i P_i \ .$$

Verifying the result of such an operation can be more efficient than computing it in a context where the verifier can make use of hints provided by the prover. The Garaga library relies on this principle. It involves an optimized version of the method proposed by Eagen in [Eag22b]. We provide a didactic description of this method hereafter and discuss its soundness.

2.2.1 Background on divisors and Weil reciprocity

The method from [Eag22b] is based on the concepts of divisors, function field of an elliptic curve and the Weil reciprocity law. We provide some background on these concepts hereafter.

Let E be an elliptic curve defined over $\mathbb{F}_q$ with defining polynomial p_E (i.e. $p_E(x, y) = x^3 + ax + b - y^2$). Let K be an extension of $\mathbb{F}_q$. A *divisor* D on $E(K)$ is a formal sum:

$$D = \sum_{P \in E(K)} n_P(P)$$

for $(n_P)_{P \in E(K)}$ some integers. The set of divisors on $E(K)$ has a natural group structure, with addition law $\sum_P n_P(P) + \sum_P n'_P(P) = \sum_P (n_P + n'_P)(P)$. The *degree* of a divisor $D = \sum_P n_P(P)$ is defined as $\deg(D) = \sum_P n_P$.

The *function field* $K(E)$ is the field of *rational functions* $E(K) \to K$. Namely, every $f \in K(E)$ can be expressed as $f = f_1/f_2$ where f_1 and f_2 are bivariate polynomials with coefficients in K (i.e. $f_1, f_2 \in K[x, y]$). Furthermore, for every $f = f_1/f_2$ and $g = g_1/g_2$ in $K(E)$ we have $f = g$ if and only if $f_1 g_2 - g_1 f_2 = h p_E$ for some $h \in K[x, y]$. This equivalence relation is consistent since for every $P \in E(K)$ we have $p_E(P) = 0$, and consequently, if $f = g$, then for every $P \in E(K)$, we have $f_1(P)g_2(P) - g_1(P)f_2(P) = 0$ that is $f(P) = g(P)$.

Let $\overline{\mathbb{F}_q}$ denotes the algebraic closure of $\mathbb{F}_q$. A *uniformizer* of E at $P \in E(\overline{\mathbb{F}_q})$ is a generator of the ideal $\{h \in K(E);\ h(P) = 0\}$. For every $P \in E(K)$, E has a unique uniformizer at P up to constants in K. Let f be a non-zero function of $K(E)$. Then the *multiplicity* of f at P, denoted $\mathrm{ord}_P(f)$, is the unique integer n such that $f = gu^n$ where $g(P) \in K^*$ and u is a uniformizer of E at P. We have $f(P) \in K^*$ if and only if $\mathrm{ord}_P(f) = 0$. If $\mathrm{ord}_P(f) > 0$ (i.e. $f(P) = 0$) then f is said to have a *zero* at P and if $\mathrm{ord}_P(f) < 0$ (i.e. $f(P)$ is undefined/zero divides $f(P)$) then f is said to have a *pole* at P.

The divisor of the function $f \in K(E)^*$, denoted $\mathrm{div}(f)$ is defined as:

$$\mathrm{div}(f) = \sum_{P \in E(\overline{\mathbb{F}_q})} \mathrm{ord}_P(f)(P).$$

Two observations follow from this definition:

1. For every $f, g \in K(E)^*$, $\mathrm{div}(fg) = \mathrm{div}(f) + \mathrm{div}(g)$ and $\mathrm{div}(f/g) = \mathrm{div}(f) - \mathrm{div}(g)$.
2. If $\mathrm{div}(f) = \mathrm{div}(g)$ then $\mathrm{div}(f/g) = 0$ that is f/g is constant. We deduce that $\mathrm{div}(f)$ determines f up to constants in K^*.

Moreover, we have the two following known facts (see, e.g., [Sil09]):

1. For every $f \in K(E)^*$, $\deg(\mathrm{div}(f)) = 0$.
2. Let $D = \sum_P n_P(P)$ be a degree 0 divisor on E. Then, there exists $f \in K(E)^*$ such that $D = \mathrm{div}(f)$ if and only if $\sum_P [n_P]P = \mathcal{O}$.

Let f be a function and let $D = \sum_P n_P(P)$ of degree 0 such that the supports of D and of $\mathrm{div}(f)$ are disjoint. The action of f on D is defined as:

$$f(D) = \prod_P f(P)^{n_P}.$$

Then, the *Weil reciprocity law* states that for all f and g, non-zero functions of $K(E)$ such that $\mathrm{div}(f)$ and $\mathrm{div}(g)$ have disjoint supports, we have:

$$f\big(\mathrm{div}(g)\big) = g\big(\mathrm{div}(f)\big) \ .$$

2.2.2 Proving a sum of points

As suggested in [Eag22b], one can use the properties of divisors to prove a sum of points:

$$\sum_{i=1}^{n} P_i = \mathcal{O} \ .$$

Indeed, as recalled previously, the above equation holds if and only if there exists a rational function $D \in K(E)$ such that

$$\mathrm{div}(D) = \sum_{i=1}^{n} (P_i) - n(\mathcal{O}) \ .$$

Remark 1. *From now on, we shall use D to denote a rational function from $K(E)$ instead of a divisor. This is for the sake of consistency with the notations of [Eag22b] and the Garaga library. Looking ahead, the function D shall be identified and built from its divisor which explains this choice.*

Then by the Weil reciprocity law, we have that for any function $L \in K(E)$:

$$D(\mathrm{div}(L)) = L(\mathrm{div}(D)) \ .$$

Let L be a line of equation $L(x, y) = \mu + \lambda \cdot x - y$ passing through the points $A_0, A_1, A_2 \in E(K)$ so that $A_0 + A_1 + A_2 = \mathcal{O}$. The divisor of such a function is $\mathrm{div}(L) = (A_1) + (A_2) + (A_3) - 3(\mathcal{O})$. Assuming that the leading coefficient of D is 1, the Weil reciprocity equation rewrites:[1]

$$D(A_0) \cdot D(A_1) \cdot D(A_2) = \prod_{i=1}^{n} L(P_i) = \prod_{i=1}^{n} (\mu + \lambda \cdot x(P_i) - y(P_i)) \ . \tag{6}$$

[1] The leading coefficient of a rational function $D(x, y) \in K(E)$ is the coefficient of the leading term of $D(1/t^2, 1/t^3)$ as t tends to 0. For $L(x, y) = \mu + \lambda \cdot x - y$, one can check that the leading coefficient is the coefficient of y, which is -1.

The above equality is obtained from $D(\mathrm{div}(L)) = L(\mathrm{div}(D))$ by removing the factors $D(\mathcal{O})^3$ and $L(\mathcal{O})^n$ from both sides of the equation. This is sound because, by definition, we have that $(-1)^{3n}D^3L^n$ tends to 1 at $\mathcal{O}$ (because the leading coefficients of D and L are 1 and -1 respectively).

To verify a statement $\sum_{i=1}^n P_i = \mathcal{O}$, the proposed protocol runs as follows:

1. The prover computes D such that $\mathrm{div}(D) = \sum_{i=1}^n (P_i) - n(\mathcal{O})$. The prover sends D to the verifier.
2. The verifier picks a random point A_0. Let $L(x, y) = \mu + \lambda \cdot x - y$ the line through A_0, $A_1 := A_0$, $A_2 := -2A_0$. The verifier checks (6) with D, $\{A_i\}$, L, $\{P_i\}$.

2.2.3 Using logarithmic derivatives

To replace the (possibly large) product in the right-hand side of (6) by a sum, an idea proposed in [Eag22b] is to rely on the logarithmic derivative. Specifically, the checked equality becomes:

$$\frac{\mathrm{dlog}}{\mathrm{d}\mu}\big(D(A_0) \cdot D(A_1) \cdot D(A_2)\big) = \frac{\mathrm{dlog}}{\mathrm{d}\mu}\Big(\prod_{i=1}^n L(P_i)\Big) ,$$

for μ the constant term in L, which rewrites as

$$\sum_{i=0}^2 \frac{\mathrm{dlog}}{\mathrm{d}\mu}\big(D(A_i)\big) = \sum_{i=1}^n \frac{\mathrm{dlog}}{\mathrm{d}\mu}\big(L(P_i)\big) .$$

In the above derivatives, the coefficients μ and λ of the line L are treated as formal variables. We recall that the logarithmic derivative is defined as:

$$\frac{\mathrm{dlog}}{\mathrm{d}\mu}(f(\mu)) = \frac{f'(\mu)}{f(\mu)} \quad \text{with} \quad f'(\mu) = \frac{\mathrm{d}(f(\mu))}{\mathrm{d}\mu} .$$

In the case of $A_0 = A_1$ and $A_2 = -2A_0$ (referred to as "higher multiplicity challenge" in [Eag22b]), L is the tangent through A_0. We recall that both μ and λ are rational functions in the coordinates of A_0 (given by the point doubling formula). It is shown in [Eag22b] that for this particular case, the above equality rewrites as:

$$c_0 \cdot \frac{D'(A_0)}{D(A_0)} - c_2 \cdot \frac{D'(A_2)}{D(A_2)} = \sum_{i=1}^n \frac{x(A_0) - x(P_i)}{y(P_i) - \lambda \cdot x(P_i) - \mu} , \tag{7}$$

where

$$c_0 = c_2 + 2\lambda \quad \text{and} \quad c_2 = \frac{2y(A_2) \cdot \big(x(A_0) - x(A_2)\big)}{3x(A_2)^2 + a - 2\lambda \cdot y(A_2)} , \tag{8}$$

and where D' is the x-derivative of D which, for the form $D(x, y) = A(x) - y \cdot B(x)$ with $A(x), B(x) \in K[x]$, satisfies:

$$D'(x, y) = \frac{d}{dx}D(x, y) = A'(x) - \frac{3x^2 + a}{2y} \cdot B(x) - y \cdot B'(x) . \tag{9}$$

Remark 2. *Compared to the formula given in [Eag22b], both sides of Equation (7) are multiplied by a factor of -1 to maintain consistency with the script [Eag22a] and Garaga's implementation.*

2.2.4 Proving an ECIP statement

We now consider an ECIP statement of the form:

$$Q = \sum_{i=1}^{n} e_i P_i$$

given a list of points $\{P_i\}$ a list of scalars $\{e_i\}$ and a result point Q. The method of [Eag22b] considers a decomposition of the scalars in base -3, that is:

$$e_i = \sum_{j=0}^{\ell-1} d_{i,j}(-3)^j \quad \text{with} \quad d_{i,j} \in \{-1, 0, 1\} \ .$$

Computing Q can be done by computing the sequence Q_ℓ, $Q_{\ell-1}$, $\ldots$, Q_0 from $Q_\ell = \mathcal{O}$ to $Q_0 = Q$ by iterating

$$Q_j = -3Q_{j+1} + \sum_{i=1}^{n} d_{i,j} P_i$$

for $j = \ell - 1$ down to $j = 0$. For each of those ℓ row equalities, there exists a function D_j satisfying

$$\operatorname{div}(D_j) = (-Q_j) + 3(-Q_{j+1}) + \sum_{\substack{i=1 \\ d_{i,j} \neq 0}}^{n} (d_{i,j} P_i) - n_j(\mathcal{O}) \ .$$

where $n_j = |\{d_{i,j} \neq 0 \,;\, 1 \leq i \leq n\}| + 4$. The above divisor has between 4 and $4 + n$ points (counting multiplicities). The function D_j can be constructed from the points as described in [Eag22b]. See also `construct_function` in the associated SageMath script [Eag22a].

Let us denote

$$D = \prod_{j=0}^{\ell-1} D_j^{(-3)^j} \ .$$

We have

$$\operatorname{div}(D) = \sum_{j=0}^{\ell-1} \operatorname{div}(D_j)(-3)^j = (-Q) + \sum_{j=0}^{\ell-1} \left[\left(\sum_{i=1}^{n} (-3)^j (d_{i,j} P_i) \right) - n_j(\mathcal{O}) \right] \ .$$

The existence of the above rational function D attests the correctness of the ECIP computation $Q = \sum_{i=1}^{n} e_i P_i$. Unfortunately, one cannot compute the above function D for standard ECIP instances since it is of exponential degree. This is where logarithmic derivatives come into play. Following the approach of Section 2.2.3, one can evaluate the Weil

reciprocity check using the logarithmic derivatives of both sides of the equation, which means checking:

$$\underbrace{\sum_{i=0}^{2} \frac{\mathrm{dlog}}{\mathrm{d}\mu}(D(A_i))}_{\mathsf{LHS}} = \underbrace{\frac{\mathrm{dlog}}{\mathrm{d}\mu} L\Big((-Q) + \sum_{j=0}^{\ell-1}\sum_{i=1}^{n}(-3)^j(d_{i,j}P_i)\Big)}_{\mathsf{RHS}} \ . \tag{10}$$

Left-hand side. We have:

$$\mathsf{LHS} = \sum_{i=0}^{2}\sum_{j=0}^{\ell-1}(-3)^j \frac{\mathrm{dlog}}{\mathrm{d}\mu}(D_j(A_i)) = \sum_{j=0}^{\ell-1}(-3)^j \sum_{i=0}^{2} \frac{\mathrm{dlog}}{\mathrm{d}\mu}(D_j(A_i)) \ .$$

From Section 2.2.3, Equation (7), we deduce:

$$\mathsf{LHS} = \sum_{j=0}^{\ell-1}(-3)^j \Big(c_0 \cdot \frac{D'_j(A_0)}{D_j(A_0)} + c_2 \cdot \frac{D'_j(A_2)}{D_j(A_2)}\Big) \ .$$

where c_0 and c_2 are the coefficients defined in (8) and D'_j is the x-derivative of D_j given by (9). Defining $\mathsf{SumDLog}$ as the following rational function:

$$\mathsf{SumDLog}(x, y) := \sum_{j=0}^{\ell-1}(-3)^j \frac{D'_j(x, y)}{D_j(x, y)} \ , \tag{11}$$

we finally get:

$$\mathsf{LHS} = c_0 \cdot \mathsf{SumDLog}(A_0) - c_2 \cdot \mathsf{SumDLog}(A_2) \ . \tag{12}$$

Remark 3. *The* $\mathsf{SumDLog}$ *function solely depends on the* D_j *functions and not on the line* L*. It can thus be precomputed and committed as a hint to the verifier before receiving the line challenge. This optimization over the original method of [Eag22b] was introduced in the Garaga library.*

Right-hand side. Remember the scalar decomposition in base -3: $e_i = \sum_{j=0}^{\ell-1} d_{i,j}(-3)^j$ for some digits $d_{i,j} \in \{-1, 0, 1\}$. For all $i \in [1, n]$, we further denote:

$$e_i^+ = \sum_{\substack{j=0 \\ d_{i,j}=1}}^{\ell-1} (-3)^j \quad \text{and} \quad e_i^- = \sum_{\substack{j=0 \\ d_{i,j}=-1}}^{\ell-1} (-3)^j \quad \text{so that} \quad e_i = e_i^+ - e_i^- \ .$$

The right-hand side can then be developed as follows:

$$\begin{aligned}
\mathsf{RHS} &= \frac{\mathrm{dlog}}{\mathrm{d}\mu} L\Big((-Q) + \sum_{j=0}^{\ell-1}\sum_{i=1}^{n}(-3)^j(d_{i,j}P_i)\Big) \\
&= \frac{\mathrm{dlog}}{\mathrm{d}\mu}\left(L(-Q)\cdot\Big(\prod_{i=1}^{n} L(P_i)^{e_i^+}\Big)\cdot\Big(\prod_{i=1}^{n} L(-P_i)^{e_i^-}\Big)\right) \\
&= \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(-Q) + \sum_{i=1}^{n} e_i^+ \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(P_i) + \sum_{i=1}^{n} e_i^- \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(-P_i) \qquad (13)
\end{aligned}$$

where according to Section 2.2.3, Equation (7), we have

$$\frac{\mathrm{dlog}}{\mathrm{d}\mu} L : P \mapsto \frac{x(A_0) - x(P)}{y(P) - \lambda\cdot x(P) - \mu}\ .$$

ECIP check. Before running the protocol, the prover and the verifier both have the points $\{P_i\}$, the scalars $\{e_i\}$ and the (alleged) result Q. The prover aims at convincing the verifier that $Q = \sum_{i=1}^{n} e_i P_i$. The ECIP check protocol runs as follows:

1. The prover computes D_j such that $\mathrm{div}(D_j) = (-Q_j) + 3(-Q_{j+1}) + \sum_{i=1}^{n}(d_{i,j}P_i)$ for all $j \in [0, \ell-1]$. The prover computes the $\mathsf{SumDLog}$ function using Equation (11). The prover sends $\mathsf{SumDLog}$ to the verifier.

2. The verifier picks a random point A_0. Let $L(x,y) = \mu + \lambda\cdot x - y$ the line through A_0, $A_1 := A_0$, $A_2 := -2A_0$. The verifier evaluates the Weil reciprocity check of Equation (10) (using Equation (12) for the LHS and Equation (13) for the RHS) with $\mathsf{SumDLog}$ (from the prover), and $\{A_i\}$, L (derived from the random point A_0).

✎ **Question 1: Scalar decomposition**

An alternative simpler way to proceed is the following one. For all $j \in \{0,\ldots,\ell-1\}$, since $Q_j = -3Q_{j+1} + \sum_{i=1}^{n} d_{i,j}P_i$, there exists a function D_j satisfying

$$\mathrm{div}(D_j) = (-Q_j) + 3(-Q_{j+1}) + \sum_{i=1}^{n} d_{i,j}(P_i) - n_j(\mathcal{O})$$

where $n_j = \sum_i d_{i,j} - 4$. Let us denote those functions $\{D_j\}_j$ differ from those used above, because the factors $\{d_{i,j}\}_{i,j}$ are outside of the divisors $\{(P_i)\}_i$. This alternative definition of D_j implies that they might have poles in the P_i's (while they only have zeros in the P_i's in the original definition) and hence must be represented as fraction of polynomials. Besides this difference, the computation of the LHS remains the same

as in the original method. On the other hand, the RHS becomes:

$$\begin{aligned}
\mathsf{RHS} &= \frac{\mathrm{dlog}}{\mathrm{d}\mu} L\Big((-Q) + \sum_{j=0}^{\ell-1}\sum_{i=1}^{n}(-3)^j d_{i,j}(P_i)\Big) \\
&= \frac{\mathrm{dlog}}{\mathrm{d}\mu} L\Big((-Q) + \sum_{i=1}^{n} e_i(P_i)\Big) \\
&= \frac{\mathrm{dlog}}{\mathrm{d}\mu}\Big(L(-Q)\cdot\prod_{i=1}^{n} L(P_i)^{e_i}\Big) \\
&= \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(-Q) + \sum_{i=1}^{n} e_i \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(P_i).
\end{aligned}$$

This more direct approach does not require decomposing each scalar e_i as $e_i^+ - e_i^-$. Let us stress that avoiding the decomposition roughly halve the computational cost of the RHS (which is expected to be the heavier part of the ECIP check).

Is there any good reason of using the $e_i^+ - e_i^-$ decomposition instead of the above approach?

2.2.5 Splitting scalars

As stated in [Eag22b], "this technique can be unsound if it is possible for either e_i^+ or e_i^- to be divisible by the field characteristic but not be zero". More generally, assume two different scalars $e_i \neq \hat{e}_i$ are such that $e_i^+ = \hat{e}_i^+ \bmod q$ and $e_i^- = \hat{e}_i^- \bmod q$, we then have

$$e_i^+ \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(P_i) + e_i^- \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(-P_i) = \hat{e}_i^+ \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(P_i) + \hat{e}_i^- \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(-P_i)$$

holding on $\mathbb{F}_q$. This makes it possible to substitute e_i by $\hat{e}_i$ and thus to prove a false ECIP statement. To avoid such a caveat, [Eag22b] suggests splitting the scalars, which is the solution implemented in Garaga. Specifically, each scalar e_i is split into two halves e_i^H, e_i^L such that $e = e_i^H \cdot \delta + e_i^L$ for some constant $\delta \approx \sqrt{q}$. In the current implementation, δ is fixed to 2^{128}. Then, the former ECIP statement is replaced by the three following ECIP statements:

$$Q^L = \sum_{i=1}^{n} e_i^L P_i \ , \quad Q^H = \sum_{i=1}^{n} e_i^H P_i \ , \quad Q^{H,\mathrm{shift}} = \delta Q^H \ ,$$

while the ECIP result is computed as $Q := Q^L + Q^{H,\mathrm{shifted}}$. Each ECIP statement gives rise to its own version of the Weil reciprocity check (Equation (10)), with the three checks using the same random point A_0. The three checks are batched using a random linear combination. Specifically, in addition to the random point A_0, the verifier picks a random coefficient γ and checks the following equation:

$$\gamma \cdot \mathsf{LHS}^L + \gamma^2 \cdot \mathsf{LHS}^H + \gamma^3 \cdot \mathsf{LHS}^{\mathrm{shift}} = \gamma \cdot \mathsf{RHS}^L + \gamma^2 \cdot \mathsf{RHS}^H + \gamma^3 \cdot \mathsf{RHS}^{\mathrm{shift}} \tag{14}$$

where $\mathsf{LHS}^L, \mathsf{LHS}^H, \mathsf{LHS}^{\text{shift}}$ (resp. $\mathsf{RHS}^L, \mathsf{RHS}^H, \mathsf{RHS}^{\text{shift}}$) are the left-hand sides (resp. right-hand sides) in the Weil reciprocity checks of the three above ECIP statements.

We can observe that the left term of Equation (14) can be written as

$$\gamma \cdot \mathsf{LHS}^L + \gamma^2 \cdot \mathsf{LHS}^H + \gamma^3 \cdot \mathsf{LHS}^{\text{shift}} = c_0 \cdot \mathsf{SumDLog}^{\text{RLC}}(A_0) - c_2 \cdot \mathsf{SumDLog}^{\text{RLC}}(A_2)$$

where

$$\mathsf{SumDLog}^{\text{RLC}} := \gamma \cdot \mathsf{SumDLog}^L + \gamma^2 \cdot \mathsf{SumDLog}^H + \gamma^3 \cdot \mathsf{SumDLog}^{\text{shift}}.$$

Instead of providing the three functions $\mathsf{SumDLog}^L$, $\mathsf{SumDLog}^H$ and $\mathsf{SumDLog}^{\text{shift}}$ as hints, the prover only needs to provides $\mathsf{SumDLog}^{\text{RLC}}$. More precisely, the prover:

1. commits all the points $\{P_1, \ldots, P_n, Q^L, Q^H, Q^{H,\text{shift}}\}$, together with the scalars $\{e_1, \ldots, e_n\}$,
2. derives the batching challenge γ,
3. commits the witness rationale function $\mathsf{SumDLog}^{\text{RLC}}$, then
4. derives the evaluation challenge A_0.

Observation 1: Soundness of ECIP batching with small co-factors

By deriving the batching challenge γ before committing the witness rationale function $\mathsf{SumDLog}^{\text{RLC}}$, the implemented protocol does not batch the three relations

$$\mathsf{LHS}^L = \mathsf{RHS}^L \ , \quad \mathsf{LHS}^H = \mathsf{RHS}^H \ , \quad \mathsf{LHS}^{\text{shift}} = \mathsf{RHS}^{\text{shift}} \ ,$$

but rather batches the three relations

$$Q^L - \sum_{i=1}^{n} e_i^L P_i = \mathcal{O} \ , \quad Q^H - \sum_{i=1}^{n} e_i^H P_i = \mathcal{O} \ , \quad Q^{H,\text{shift}} - \delta Q^H = \mathcal{O} \ .$$

over the elliptic curve. More precisely, it checks that the following batched relation

$$\gamma \cdot \left(Q^L - \sum_{i=1}^{n} e_i^L P_i \right) + \gamma^2 \cdot \left(Q^H - \sum_{i=1}^{n} e_i^H P_i \right) + \gamma^3 \cdot \left(Q - (\delta Q^H + Q^L) \right) = \mathcal{O}$$

holds for a random value γ, which ensures that the three relations over the EC points hold with overwhelming probability. Let us stress that γ is not a uniformly random value over the scalars since it is uniformly sampled from $\{0, \ldots, q-1\}$, and not from $\{0, \ldots, r-1\}$, although the bias is negligible. However, the batching is not sound if the elliptic curve has a small co-factor $h > 1$. For example, let us assume that $\Delta := Q^L - \sum_{i=1}^{n} e_i^L P_i$ is not zero, but a point of small order: $\Delta \neq \mathcal{O}$ and $h \cdot \Delta = \mathcal{O}$. In that case, the prover will succeed to convince the verifier about the validity of this invalid statement as soon as $\gamma \cdot \Delta = \mathcal{O}$, which occurs with probability at most $1/h$.

Recommendation

To avoid the above issue for elliptic curves with small co-factor (e.g., the BLS12-381 curve or the ED25519 curve), we recommend to effectively batch the ECIP checks and not the relations between the underlying EC points. Concretely, $\mathsf{SumDLog}^L$, $\mathsf{SumDLog}^H$ and $\mathsf{SumDLog}^{\mathrm{RLC}}$ should be committed *before* sampling γ. Alternatively, the ECIP check should reject any input point of small order by implementing an additional check.

2.2.6 Algorithmic description

We provide hereafter a detailed algorithmic description of the method as implemented in Garaga:

- Algorithm 1 describes the function ECIP.DeriveChallengeRelatedValues that computes all the objects related to the evaluation challenge A_0, namely the line L tangent through A_0, the point $A_2 := -2A_0$, and the two coefficients (c_0, c_2) of Equation (8).
- Algorithm 2 describes the function ECIP.ComputeLHS that computes the LHS term of the ECIP check using the witness rational function SumDLog.
- Algorithm 3 describes the function ECIP.ComputeRHS that computes the RHS term of the ECIP check. Before adding the term related to $e_i \cdot P_i$ in RHS, ECIP.ComputeRHS checks whatever $P_i \neq \mathcal{O}$ and $e_i \neq 0$ (or equivalently $|e_i^+| \neq |e_i^-|$).
- Algorithm 4 describes the general-purpose function MultiScalarMultiplication that output the result of the ECIP (a.k.a. multi-scalar multiplication) while checking all the logarithmic derivative relations. Following the Fiat-Shamir heuristic, the challenges are derived as the outputs of hash functions that take the committed values as inputs.

Algorithm 1 ECIP.DeriveChallengeRelatedValues

Input: $A_0 \in E(K)$
Output: $L(x, y) : y - (\mu + \lambda x)$, $A_2 \in E(K)$, $(c_0, c_2) \in \mathbb{F}_q^2$

▷ Compute the line L tangent through A_0.

$\lambda \leftarrow \dfrac{3\mathtt{x}(A_0)^2 + a}{2\mathtt{y}(A_0)}$

$\mu \leftarrow \mathtt{y}(A_0) - \lambda \cdot \mathtt{x}(A_0)$

▷ Compute the point $A_2 := -2A_0$.

$\mathtt{x}(A_2) \leftarrow \lambda^2 - 2\mathtt{x}(A_0)$

$\mathtt{y}(A_2) \leftarrow \mu + \lambda \cdot \mathtt{x}(A_2)$

▷ Compute the two coefficients (c_0, c_1) used for the logarithmic derivative check.

$c_2 \leftarrow \dfrac{2\mathtt{y}(A_2) \cdot (\mathtt{x}(A_0) - \mathtt{x}(A_2))}{3\mathtt{x}(A_2)^2 + a - 2\lambda \cdot \mathtt{y}(A_2)}$

$c_0 \leftarrow c_2 + 2\lambda$

return $(L, A_2, (c_0, c_2))$

Algorithm 2 ECIP.ComputeLHS

Input: $\mathsf{SumDLog} \in K(E)$, $(A_0, A_2) \in E(K)^2$, $(c_0, c_2) \in \mathbb{F}_q^2$
Output: $\mathsf{LHS} \in \mathbb{F}_q$

Parse $\mathsf{SumDLog}(x, y)$ as $\dfrac{a_1(x)}{a_2(x)} + y \cdot \dfrac{b_1(x)}{b_2(x)}$

$\mathsf{eval}_{A_0} \leftarrow \dfrac{a_1(\mathtt{x}(A_0))}{a_2(\mathtt{x}(A_0))} + \mathtt{y}(A_0) \cdot \dfrac{b_1(\mathtt{x}(A_0))}{b_2(\mathtt{x}(A_0))}$

$\mathsf{eval}_{A_2} \leftarrow \dfrac{a_1(\mathtt{x}(A_2))}{a_2(\mathtt{x}(A_2))} + \mathtt{y}(A_2) \cdot \dfrac{b_1(\mathtt{x}(A_2))}{b_2(\mathtt{x}(A_2))}$

$\mathsf{LHS} \leftarrow c_0 \cdot \mathsf{eval}_{A_0} - c_2 \cdot \mathsf{eval}_{A_2}$

return LHS

Algorithm 3 ECIP.ComputeRHS

Input: $\boldsymbol{P} = (P_1, \ldots, P_n)$, $L(x, y) : y - (\mu + \lambda x)$, $\mathtt{x}(A_0)$, $\{(e_i^+, e_i^-)\}_{1 \le i \le n}$, Q
Output: $\mathsf{RHS} \in \mathbb{F}_q$

$\mathsf{RHS} = 0$

for $i = 1$ **to** n **do**

  if $|e_i^+| \neq |e_i^-|$ and $P \neq \mathcal{O}$ **then**

    $\mathsf{RHS} \leftarrow \mathsf{RHS} + e_i^+ \cdot \dfrac{\mathtt{x}(A_0) - \mathtt{x}(P_i)}{\mathtt{y}(P_i) - (\mu + \lambda \cdot \mathtt{x}(P_i))} + e_i^- \cdot \dfrac{\mathtt{x}(A_0) - \mathtt{x}(P_i)}{-\mathtt{y}(P_i) - (\mu + \lambda \cdot \mathtt{x}(P_i))}$

if $Q \neq \mathcal{O}$ **then**

  $\mathsf{RHS} \leftarrow \mathsf{RHS} + \dfrac{\mathtt{x}(A_0) - \mathtt{x}(Q)}{-\mathtt{y}(Q) - (\mu + \lambda \cdot \mathtt{x}(Q))}$

return RHS

Algorithm 4 MultiScalarMultiplication

Input: $\boldsymbol{P} = (P_1, \ldots, P_n)$, $(e_1, \ldots, e_n)$, ScalarDigitDecomp
Hint: $Q^L, Q^H, Q^{H,\text{shift}} \in E(K)$, $\mathsf{SumDLog}^{\text{RLC}} \in K(E)$
Output: $Q := \sum_{i=1}^{n} e_i P_i$

▷ Run some sanity checks.

For all $Q \in \{Q^L, Q^H, Q^{H,\text{shift}}\}$, check that $Q = \mathcal{O}$ or Q is on the curve.
Check that $\mathsf{SumDLog}^{\text{RLC}} := \frac{a_1(x)}{a_2(x)} + y \cdot \frac{b_1(x)}{b_2(x)}$ satisfies $\deg a_1(x) \leq n+3$, $\deg a_2(x) \leq n+4$, $\deg b_1(x) \leq n+4$, and $\deg b_2(x) \leq n+7$.

▷ Derive the challenges.

$\gamma = \text{Hash}(n, \boldsymbol{P}, Q^L, Q^H, Q^{H,\text{shift}}, (e_1, \ldots, e_n))$ ▷ $\gamma \in \mathbb{F}_q$
$A_0 = \text{Hash}(\gamma, \mathsf{SumDLog}^{\text{RLC}})$ ▷ $A_0 \in E(K)$

▷ Precompute some values.

$(L, A_2, (c_0, c_2)) \leftarrow \mathsf{ECIP.DeriveChallengeRelatedValues}(A_0)$
Decompose all the scalars $\{e_i\}_{1 \leq i \leq n}$ as $\{(e_i^{L,+}, e_i^{L,-}, e_i^{H,+}, e_i^{H,-})\}_i$

▷ Compute and compare $\mathsf{RHS}^{\text{RLC}}$ and $\mathsf{LHS}^{\text{RLC}}$.

$\mathsf{RHS}^L \leftarrow \mathsf{ECIP.ComputeRHS}(\boldsymbol{P}, L, \mathtt{x}(A_0), \{(e_i^{L,+}, e_i^{L,-})\}_i, Q^L)$
$\mathsf{RHS}^H \leftarrow \mathsf{ECIP.ComputeRHS}(\boldsymbol{P}, L, \mathtt{x}(A_0), \{(e_i^{H,+}, e_i^{H,-})\}_i, Q^H)$
$\mathsf{RHS}^{\text{shift}} \leftarrow \mathsf{ECIP.ComputeRHS}(\{Q^H\}, L, \mathtt{x}(A_0), \{(\delta^+, \delta^-)\}, Q^{H,\text{shift}})$
$\mathsf{RHS}^{\text{RLC}} \leftarrow \gamma \cdot \mathsf{RHS}^L + \gamma^2 \cdot \mathsf{RHS}^H + \gamma^3 \cdot \mathsf{RHS}^{\text{shift}}$
$\mathsf{LHS}^{\text{RLC}} \leftarrow \mathsf{ECIP.ComputeLHS}(\mathsf{SumDLog}^{\text{RLC}}, (A_0, A_2), (c_0, c_2))$
Check that $\mathsf{LHS}^{\text{RLC}} = \mathsf{RHS}^{\text{RLC}}$

$Q \leftarrow Q^L + Q^{H,\text{shift}}$
return Q

2.2.7 Soundness of the ECIP check

Observation 2: Soundness of the ECIP check

We present below a method to break the soundness of the ECIP check. This attack targets the variant of the original method that relies on the sum of logarithmic derivatives.

In a nutshell, the attack exploits two different values of $\hat{e}_i^+$ (or $\hat{e}_i^-$) that are congruent modulo q, the prime of the base field. Under the scalar split constraint, only one of these values is admissible in the ECIP statement. However, the other value can still be used to construct a SumDLog function and prove a statement with an incorrect result Q.

In its basic form, as introduced hereafter, the attack requires constructing the prime q of the base field in a specific way. However, we believe it can be adapted to remove this requirement.

One way to mitigate the attack is to revert to the original method. However, this approach comes with increased complexity and, more importantly, lacks a formal soundness proof. Although [Eag22b] provides some informal arguments supporting its soundness, the ECIP check protocol itself does not include a rigorous proof. Our preliminary investigations leave it unclear whether such a proof is feasible or whether the original method may also suffer soundness issues.

Recommendation

We recommend switching to the original method from [Eag22b] *only if* a formal soundness proof can be established. Otherwise, we suggest adopting a more standard approach in which each iteration of the ECIP loop is verified independently, using dedicated hints.

We describe an approach to break the soundness of the ECIP check. For the sake of simplicity, let us consider a simple ECIP statement $Q = eP$ with a single point P and a single scalar e. Even though e may be small enough to avoid the issue described above (see "Splitting scalars"), we consider $\hat{e} = \hat{e}^+ - \hat{e}^- \neq e$ such that

$$\hat{e}^+ = e^+ \bmod q \quad \text{and} \quad \hat{e}^- = e^- \bmod q \tag{15}$$

where $\hat{e}^+$ is made of the 1-digits of $\hat{e}$ and $\hat{e}^-$ is made of the (-1)-digits of $\hat{e}$ in the same way as e^+ and e^- for e.

It is easy to construct such a pair $(e, \hat{e})$ if the base-(-3) expansion of q has only $\{0, 1\}$ digits: simply define $\hat{e}^+ = e^+ + q$ and $\hat{e}^- = e^-$, with the constraint that the base-(-3) expansion of $\hat{e}^+$ should have only $\{0, 1\}$ digits (e.g. taking $e^+ = 0$).

Let $\hat{Q} = \hat{e}P \neq Q = eP$. Then, as for the method described in Section 2.2.4, there

exists a rational function SumDLog satisfying

$$c_0 \cdot \mathsf{SumDLog}(A_0) - c_2 \cdot \mathsf{SumDLog}(A_2) = -\frac{\mathrm{dlog}}{\mathrm{d}\mu} L(\hat{Q}) + \hat{e}^+ \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(P) + \hat{e}^- \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(-P)$$

for all points A_0 (and associated A_2 and L). Because of the modulo-q equivalence between the scalars (see Equation (15)), the above equality implies:

$$c_0 \cdot \mathsf{SumDLog}(A_0) - c_2 \cdot \mathsf{SumDLog}(A_2) = -\frac{\mathrm{dlog}}{\mathrm{d}\mu} L(\hat{Q}) + e^+ \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(P) + e^- \frac{\mathrm{dlog}}{\mathrm{d}\mu} L(-P) \tag{16}$$

(with $\hat{Q} = \hat{e}P$ and e^+, e^- in place of $\hat{e}^+, \hat{e}^-$). We deduce that, thanks to the existence of the rational function SumDLog satisfying the above equality for all points A_0 (and associated A_2 and L), one can prove the false statement that $\hat{Q}$ equals eP.

We have implemented this attack for validation. A test vector is provided in Appendix A.

Extending to any prime q. One limitation of the above attack is to constrain the prime q to have a base-(-3) expansion with only $\{0, 1\}$ digits. We can address the case of a prime q of base-(-3) expansion with only $\{0, -1\}$ digits (by using $\hat{e}^-$ instead of $\hat{e}^+$). To support any prime q, we can adapt the way the SumDLog function is constructed.

We still consider $\hat{e} = \hat{e}^+ - \hat{e}^- \neq e$ satisfying Equation (15) but do not impose any constraint on the base-(-3) expansion of $\hat{e}^+$ and $\hat{e}^-$. Instead, we consider their base-2 expansions:

$$\hat{e}^+ = \sum_{j=0}^{\ell} d_j^+ \, 2^j \quad \text{and} \quad \hat{e}^- = \sum_{j=0}^{\ell} d_j^- \, 2^j$$

for some digits $d_j^+, d_j^- \in \{0, 1\}$. We can then compute $\hat{Q} = \hat{e}P \neq Q$ with the sequence $\hat{Q}_\ell$, $\hat{Q}_{\ell-1}$, …, $\hat{Q}_0$ from $\hat{Q}_\ell = \mathcal{O}$ to $\hat{Q}_0 = \hat{Q}$ by iterating

$$\hat{Q}_j = 2\hat{Q}_{j+1} + d_j^+ P + d_j^- (-P)$$

for $j = \ell - 1$ down to $j = 0$. For each of those ℓ row equalities, there exists a function $\hat{D}_j$ satisfying

$$\mathrm{div}(\hat{D}_j) = -(Q_j) + 2(Q_{j+1}) + (d_j^+ P) + (-d_j^- P) - n_j(\mathcal{O})$$

where $n_j = 1 + d_j^+ + d_j^-$. Now constructing the SumDLog function from those $\{\hat{D}_j\}$, as in the original method, namely:

$$\mathsf{SumDLog}(x, y) := \sum_{j=0}^{\ell-1} 2^j \frac{\hat{D}'_j(x, y)}{\hat{D}_j(x, y)} \ ,$$

we get a rational function satisfying the broken ECIP statement of Equation (16).

Remark 4. *Let us note that, in the original method, the divisors of the functions D_j consist solely of zeros at non-zero points and a single pole of high multiplicity at $\mathcal{O}$. In contrast, due to the construction used here –and the choice of a positive basis 2 instead of the negative basis -3– each $\hat{D}_j$ has a pole at Q_j. Despite this structural difference, we believe the attack remains applicable.*

2.3 Pairings

The Garaga library provides pairing-related functionality for the elliptic curves BN254 and BLS12-381. Detailed definitions and algorithmic descriptions of these functions are presented in the following.

2.3.1 Pairing definitions

Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be two additive Abelian groups with identity element denoted $\mathcal{O}$ and let $\mathbb{G}_T$ be a multiplicative Abelian group with identity element denote 1. In practice $\mathbb{G}_1$ and $\mathbb{G}_2$ will be some subgroups of points of an elliptic curve and $\mathbb{G}_T$ will be a subgroup of the multiplicative group of a finite field $\mathbb{F}_{q^k}$.

A *pairing* is a function:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$$

satisfying the following properties:

- *bilinearity*: for every $P \in \mathbb{G}_1$ and $Q_1, Q_2 \in \mathbb{G}_2$:

  $$e(P, Q_1 + Q_2) = e(P, Q_1)e(P, Q_2) \ ,$$

  and for every $P_1, P_2 \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$:

  $$e(P_1 + P_2, Q) = e(P_1, Q)e(P_2, Q) \ ,$$

- *non-degeneracy*: if $e(P, Q) = 1$ for every $Q \in \mathbb{G}_2$, then $P = \mathcal{O}$ and if $e(P, Q) = 1$ for every $P \in \mathbb{G}_1$, then $Q = \mathcal{O}$,

- *efficiency*: given any $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$, $e(P, Q)$ can be efficiently computed.

For cryptographic applications, one usually choose $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ to be cyclic groups of prime order r. For technical reasons, $\mathbb{G}_1$ and $\mathbb{G}_2$ need to be distinct.

Defining the subgroups. Both curves BN254 and BLS12-381 have *embedding degree* $k = 12$, implying that they work over the field extension $\mathbb{F}_{q^{12}}$. The concrete field extensions are given in Table 1.

The two groups $\mathbb{G}_1$ and $\mathbb{G}_2$ are defined as follows:

- For BN254:
  - $\mathbb{G}_1 \subset E(\mathbb{F}_q)$ where $E : y^2 = x^3 + 3$

Field	Field extension	
	BN254	BLS12-381
$\mathbb{F}_{q^{12}}$	$\mathbb{F}_q[w]/(w^{12} - 18w^6 + 82)$	$\mathbb{F}_q[w]/(w^{12} - 2w^6 + 2)$
$\mathbb{F}_{q^2}$	$\mathbb{F}_q[u]$ with $u := w^6 - 9$	$\mathbb{F}_q[u]$ with $u := w^6 - 1$

Table 1: Definition of the field extensions for the curves BLS12-381 and BN254.

- $\mathbb{G}_2 \subset E'(\mathbb{F}_{q^2})$ where $E' : y'^2 = x'^3 + 3/(9+u)$,

where E' is obtained from E using the twisting transformation:

$$\Psi : \begin{cases} E'(\mathbb{F}_{q^2}) & \to E(\mathbb{F}_{q^{12}}) \\ (x', y') & \mapsto (w^2 \cdot x', w^3 \cdot y'). \end{cases}$$

- For BLS12-381:
  - $\mathbb{G}_1 \subset E(\mathbb{F}_q)$ where $E : y^2 = x^3 + 4$
  - $\mathbb{G}_2 \subset E'(\mathbb{F}_{q^2})$ where $E' : y^2 = x^3 + 4(1+u)$,

  where E' is obtained from E using the twisting transformation:

$$\Psi : \begin{cases} E'(\mathbb{F}_{q^2}) & \to E(\mathbb{F}_{q^{12}}) \\ (x', y') & \mapsto (x'/w^2, y'/w^3). \end{cases}$$

Defining the pairing. For every $P \in E(\mathbb{F}_{q^k})$, let us introduce the Miller function $f_{m,P}$ defined as a rational function with divisor:

$$(f_{m,P}) = m(P) - ([m]P) - (m+1)(\mathcal{O}) \ .$$

This function satisfies the two identities:

$$f_{a+b,P} = f_{a,P} \cdot f_{b,P} \cdot \frac{\ell_{[a]P,[b]P}}{v_{[a+b]P}} \tag{17}$$

$$f_{ab,P} = f_{a,P}^b \cdot f_{b,[a]P}$$

with

$$(\ell_{P,Q}) = (P) + (Q) + (-P-Q) - 3(\mathcal{O})$$
$$(v_P) = (P) + (-P) - 2(\mathcal{O}).$$

The Garaga library uses the "optimal pairings" as defined in the following theorem.

Theorem 1 ([Ver10]). *Let $\lambda = mr$ with $r \nmid m$ and write $\lambda = \sum_{i=0}^{l} c_i q^i$ then:*

$$\begin{aligned} a_{(c_i)_i} : \mathbb{G}_1 \times \mathbb{G}_2 &\to \mu_r \subset \mathbb{F}_{q^k}^* \\ (P, Q) &\mapsto \left(\prod_{i=0}^{l} f_{c_i,Q}^{q^i}(P) \cdot \prod_{i=1}^{l} \frac{\ell_{[m_i]Q,[c_i q^i]Q}(P)}{v_{[m_{i+1}]Q}}(P) \right)^{(q^k-1)/r} \end{aligned}$$

where $m_i = \sum_{j=0}^{i-1} c_j q^j$, defines a pairing which is non-degenerate if and only if

$$mkq^{k-1} \not\equiv ((q^k - 1)/r) \sum_{i=0}^{l} i c_i q^{i-1} \bmod r.$$

When the embedding degree is even –which is the case with BN254 and BLS12-381–, [BKLS02] observes that vertical lines can be omitted since their evaluations vanish after the final exponentiation. Then the pairing is defined as:

$$\begin{aligned} a_{(c_i)_i} : \mathbb{G}_1 \times \mathbb{G}_2 &\to \mu_r \subset \mathbb{F}_{q^k}^* \\ (P, Q) &\mapsto \left(\prod_{i=0}^{l} f_{c_i,Q}^{q^i}(P) \cdot \prod_{i=1}^{l} \ell_{[m_i]Q,[c_i q^i]Q}(P) \right)^{(q^k-1)/r} . \end{aligned}$$

ForBN254 and BLS12-381, the following values for λ are used:

- For BN254 ($l = 3$):
$$\lambda = 6\,\mathbf{z}^2 + 2 + q - q^2 + q^3 \ ,$$
- For BLS12-381 ($l = 1$):
$$\lambda = -\mathbf{z} + q \ .$$

2.3.2 Multi-pairing check

The main pairing functionality implemented in the Garaga library is the multi-pairing check. This functionality verifies that some pairs of points $(P_1, Q_1) \in \mathbb{G}_1 \times \mathbb{G}_2$, ..., $(P_{n_p}, Q_{n_p}) \in \mathbb{G}_1 \times \mathbb{G}_2$ satisfy:

$$\prod_{j=1}^{n_p} e(P_j, Q_j) = 1 \ .$$

This multi-paring check is implemented following the ideas from [NE24]. First, since the implementation relies on optimal pairings as defined in Theorem 1, the above check is equivalent to:

$$\left(\prod_{i=0}^{l} \prod_{j=1}^{n_p} f_{c_i,Q_j}^{q^i}(P_j) \cdot \prod_{i=1}^{l} \prod_{j=1}^{n_p} \ell_{\Psi([m_i]Q_j),\Psi([c_i q^i]Q_j)}(P_j) \right)^{(q^k-1)/r} = 1 \ .$$

Eliminating the final exponentiation. Let $h := q^k - 1/r$. To efficiently check whether a field element $x \in \mathbb{F}_{q^k}$ satisfies $x^h = 1$, [NE24] proposes the following approach. First, instead of checking $x^h = 1$ directly, one equivalently checks that there exists a $c \in \mathbb{F}_{q^k}$ (provided as a hint by the prover) such that $x = c^r$. The advantage of this method is that r is much smaller than h, making the exponentiation significantly cheaper. However, [NE24] observes that one can improve upon this by instead checking if there exists a c such that $x = c^{mr}$, where the factor m is chosen such that exponentiation by mr is efficient. In practice, one takes $mr := \lambda$, the same value λ used in the optimal pairing, which has been specifically chosen for its sparse q-basis decomposition, allowing for efficient exponentiation via the Frobenius application.

While all $x \in \mathbb{F}_{q^k}$ satisfying $x^h = 1$ are r-residues (i.e., there exists c such that $x = c^r$), they are not all λ-residues because $d := \gcd(m, h) \neq 1$. So while the existence of c such that $x = c^\lambda$ is sufficient to prove that $x^h = 1$, this won't work for all x satisfying $x^h = 1$. We can decompose $q^k - 1$ as the product $r \cdot d' \cdot h'$, where d' is the smallest multiple of d such that d' and h' are pairwise coprime. This gives the following decomposition of the multiplicative group:

$$\mathbb{F}_{q^k}^* \cong (\mathbb{Z}/r\mathbb{Z}) \times \left(\mathbb{Z}/d'\mathbb{Z}\right) \times \left(\mathbb{Z}/h'\mathbb{Z}\right).$$

The second term in the decomposition, $(\mathbb{Z}/d'\mathbb{Z})$, prevents us from computing a λ-th root of x, i.e., from computing a d-th root of x. Fortunately, for the curves BN254 and BLS12-381, the d'-th roots of unity lie in a strict subfield $\mathbb{F}_{q^{k'}}$ of $\mathbb{F}_{q^k}$, where $k' < k$ is the smallest integer such that $d' \mid q^{k'} - 1$. Specifically, the d'-th roots of unity lie $\mathbb{F}_{q^3}$ for these curves. Thus, we can check that an element x satisfies $x^h = 1$ by verifying that there exists $c \in \mathbb{F}_{q^k}$ and $u \in \mathbb{F}_{q^{k'}}$ such that $x \cdot u = c^\lambda$, which works for both BN254 and BLS12-381.[2] The prover can construct u and c as follows:

$$u := (x^{h'})^{(-h'^{-1}) \bmod d'} \quad \text{and} \quad c := (x \cdot u)^{(\lambda^{-1}) \bmod h'}.$$

To sum up, for BN254 and BLS12-381, checking that $\prod_{j=1}^{n_p} e(P_j, Q_j) = 1$ is equivalent to check that there exists $c \in \mathbb{F}_{q^k}$ and $u \in \mathbb{F}_{q^{k'}}$ (where $k = 12$ and $k' = 3$) such that:

$$\prod_{i=0}^{l} \prod_{j=1}^{n_p} f_{c_i, Q_j}^{q^i}(P_j) \cdot \prod_{i=1}^{l} \prod_{j=1}^{n_p} \ell_{\Psi([m_i]Q_j), \Psi([c_i q^i]Q_j)}(P_j) \cdot c^{-\lambda} \cdot u = 1 \ ,$$

where $m_i = \sum_{j=0}^{i-1} c_j q^j$.

2.3.3 Multi Miller loop

To compute the term $f_{c_i, Q_j}(P_j)$, the usual method consists of using Miller's algorithm, which implements a double-and-add strategy based on the recursive relation (17). However, instead of running the Miller loop for each pair of points and then accumulating

[2] In cases where the d'-th roots of unity do not lie in a strict subfield, we would need to check that u is indeed a d'-root of unity. To verify $x^h = 1$, we check that there exist $c, u \in \mathbb{F}_{q^k}$ such that $x \cdot u = c^\lambda$ and $u^{d'} = 1$.

results, we can run all Miller loops in parallel and use the same accumulator. This optimization is called a *multi Miller loop* [Sco19]. Once we computed the product of the associated Miller loops, we need to multiply the result by the product of line evaluations $\prod_{i=1}^{l}\prod_{j=1}^{n_p}\ell_{\Psi([m_i]Q),\Psi([c_iq^i]Q)}(P_j)$, by the $c^{-\lambda}$ and by u. We can embed the computation of $c^{-\lambda}$ in the Miller loop. We describe in Algorithm 5 the multi Miller loop which computes the exponentiation of c at the same time. The output of the loop is $c^{-s}\cdot\prod_{j=1}^{n_p}f_{s,Q_j}(P_j)$ for some parameter s, together with $[s]Q_j$ for all $j\in\{1,\dots,n_p\}$.

Algorithm 5 Multi Miller loop with embedded c exponentiation

Parameter: n_p (number of pairs of points), $s := 2^{L-1}+\sum_{i=0}^{L-2}b_i\cdot 2^i$ with $b_i\in\{0,1\}$
Input: $c\in\mathbb{F}_{q^k}$, $(P_1,Q_1),\dots,(P_{n_p},Q_{n_p})\in\mathbb{G}_1\times\mathbb{G}_2$
Output: $c^{-s}\cdot\prod_{j=1}^{n_p}f_{s,Q_j}(P_j)$ and $\{[s]Q_j\}_j$

$f\leftarrow c^{-1}$
Set $T_j=Q_j$ for all $j\in\{1,\dots,n_p\}$
▷ At the end of the iteration i, we have $T_j := [s_i]Q_j$, $f := c^{-s_i}\cdot\prod_{j=1}^{n_p}f_{s_i,Q_j}(P_j)$, where $s_i := 2^{L-2-i}+\sum_{j=0}^{L-2-i}b_{i+j}2^i$.
for $i=L-2$ **to** 0 **do**
  $f\leftarrow f^2$
  if $b_i=1$ **then**
    $f\leftarrow f\cdot c^{-1}$
  for $j=1$ **to** n_p **do**
    if $b_i=0$ **then**
      $f\leftarrow f\cdot\ell_{\Psi(T_j),\Psi(T_j)}(P_j)$
      $T_j\leftarrow[2]T_j$
    if $b_i=1$ **then**
      $f\leftarrow f\cdot\ell_{\Psi(T_j),\Psi(Q_j)}(P_j)\cdot\ell_{\Psi(T_j+Q_j),\Psi(T_j)}(P_j)$
      $T_j\leftarrow[2]T_j+Q_j$
return $f,\{T_j\}_j$

Let us assume that λ can be written as $s+\sum_{i=1}^{l}\epsilon_iq^i$ with $\epsilon_1,\dots,\epsilon_l\in\{-1,0,1\}$. Then one needs to check the following equality:

$$\underbrace{\left(c^{-s}\cdot\prod_{j=1}^{n_p}f_{s,Q_j}(P_j)\right)}_{\text{Multi Miller Loop}}\cdot\prod_{i=1}^{l}\prod_{j=1}^{n_p}\ell_{\Psi([s+\sum_{h=1}^{i-1}\epsilon_h\cdot q^h]Q_j),\Psi([\epsilon_iq^i]Q_j)}(P_j)\cdot\prod_{i=1}^{l}c^{\epsilon_i\cdot q^i}\cdot u=1.$$

We describe in Algorithm 6 this check. Let us note that it relies on the fact that $\mathbb{G}_2$ is the so-called "trace-zero subgroup" so that the Frobenius endomorphishm satisfies $\pi_q(Q)=[q]Q$ for any $Q\in\mathbb{G}_2$.

Algorithm 6 Multi-pairing check for optimal pairings, assuming $\lambda = s + \sum_{i=1}^{l} \epsilon_i \cdot q^i$ with $\epsilon_1, \ldots, \epsilon_l \in \{-1, 0, 1\}$.

Input: $(P_1, Q_1), \ldots, (P_{n_p}, Q_{n_p}) \in \mathbb{G}_1 \times \mathbb{G}_2$
Hint: $c \in \mathbb{F}_{q^k}$ and $u \in \mathbb{F}_{q^{k'}}$ (with $k' < k$) such that $\prod_{i=1}^{n_p} f_{\lambda,Q_j}(P_j) \cdot u = c^\lambda$
Output: `True` if and only if $\prod_{j=0}^{n_p} e(P_j, Q_j) = 1$

```
f, {T_1, ..., T_n} ← MultiMillerLoop(c, (P_1, Q_1), ..., (P_{n_p}, Q_{n_p}))   ▷ T_j := [s]Q_j
for i = 1 to l do
    for j = 1 to n_p do
        Q_j ← π_q(Q_j)
        f ← f · (c^{ε_i})^{q^i} · ℓ_{Ψ(T_j),Ψ([ε_i]Q_j)}(P_j)
        T_j ← T_j + [ε_i]Q_j
f ← f · u
return f = 1
```

Line computation. Algorithms 5 and 6 require to evaluate some lines $\ell_{\Psi(Q_1),\Psi(Q_2)}(P)$ for some points $P \in \mathbb{G}_1 \subset E(\mathbb{F}_q)$ and $Q_1, Q_2 \in \mathbb{G}_2 \subset E'(\mathbb{F}_{q^2})$. A naive implementation would consist in computing the $\mathbb{F}_{q^k}$-rational points $\Psi(Q_1)$ and $\Psi(Q_2)$, building the line $\ell_{\Psi(Q_1),\Psi(Q_2)}$ and evaluating the line at P. Such an implementation would involve multiplications over $\mathbb{F}_{q^k}$, which can actually be avoided. One can instead compute the line ℓ_{Q_1,Q_2} using $\mathbb{F}_{q^2}$ operations from which one can deduce the line $\ell_{\Psi(Q_1),\Psi(Q_2)}$. Let us first describe how we build the line ℓ_{Q_1,Q_2}.

In practice, in the Miller loop, we need to compute $[2]T$ when the current bit is 0 and $[2]T + Q$ when the current bit is 1, for some accumulator $T \in E'(\mathbb{F}_{q^{k/d}})$ and $Q \in \mathbb{G}_2$, the second input point. Then after the Miller loop, one needs to compute $T + Q$. We describe three algorithms that deals with each case:

- AddStep (Algorithm 7) takes T and Q as inputs and outputs both $T + Q$ and $\ell_{T,Q}$. It is used for the accumulation of the last line evaluation after the Miller loop;
- DoubleStep (Algorithm 8) takes T as input and outputs both $[2]T$ and $\ell_{T,T}$. It is used in the Miller loop when the current bit is 0;
- DoubleAndAddStep (Algorithm 9) takes T and Q as inputs and outputs $[2]T + Q$, $\ell_{T,Q}$ and $\ell_{T+Q,T}$. It is used in the Miller loop when the current bit is 1.

Line evaluation. The previous algorithms compute the line ℓ_{Q_1,Q_2} from Q_1 and Q_2 in the specific contexts where we need it. We now describe how to compute $\ell_{\Psi(Q_1),\Psi(Q_2)}(P)$ for a point $P \in \mathbb{G}_1 \subset E(\mathbb{F}_q)$ using ℓ_{Q_1,Q_2}.

To optimize the line evaluation, one can precompute $P' = (x', y')$ such that $x' := \frac{-x}{y}$ and $y' := \frac{1}{y}$, where $P := (x, y)$ and evaluate the normalized line $\ell'(x', y') := \ell(x, y)/y$. Let $\ell(x, y) = y - (\lambda x - \mu)$, we get $\ell'(x', y') = 1 + \lambda x' + \mu y'$. Evaluating ℓ' over P' instead of evaluating ℓ over P does not impact the pairing result since the factor $1/y$

Algorithm 7 AddStep

Input: $T := (x_T, y_T), Q := (x_Q, y_Q) \in E'(\mathbb{F}_{q^{k/d}})$
Output: $\mathtt{ln} := \ell_{T,Q}$, $T' := T + Q$

$\lambda \leftarrow \frac{y_T - y_Q}{x_T - x_Q}$
$\mu \leftarrow \lambda \cdot x_T - y_T$
$x' \leftarrow \lambda^2 - x_T - x_Q$
$y' \leftarrow \lambda \cdot (x_T - x') - y_T$
$\mathtt{ln} \leftarrow \mathsf{Line}(\lambda, \mu)$
$T' \leftarrow (x', y')$
return $\mathtt{ln}, T'$

Algorithm 8 DoubleStep

Input: $T := (x, y) \in E'(\mathbb{F}_{q^{k/d}})$
Output: $\mathtt{ln} := \ell_{T,T}$, $T' := [2]T$

$\lambda \leftarrow \frac{3x^2}{2y}$
$\mu \leftarrow \lambda \cdot x - y$
$x' \leftarrow \lambda^2 - 2x$
$y' \leftarrow \lambda \cdot (x - x') - y$
$\mathtt{ln} \leftarrow \mathsf{Line}(\lambda, \mu)$
$T' \leftarrow (x', y')$
return $\mathtt{ln}, T'$

Algorithm 9 DoubleAndAddStep

Input: $T := (x_T, y_T), Q := (x_Q, y_Q) \in E'(\mathbb{F}_{q^{k/d}})$
Output: $\mathtt{ln}_1 := \ell_{T,Q}$, $\mathtt{ln}_2 := \ell_{T+Q,T}$, $T' := [2]T + Q$

$\lambda_1 \leftarrow \frac{y_T - y_Q}{x_T - x_Q}$
$\mu_1 \leftarrow \lambda_1 \cdot x_T - y_T$
$x' \leftarrow \lambda_1^2 - x_T - x_Q$ ▷ $x' := \mathtt{x}(T + Q)$
$\lambda_2 \leftarrow -\lambda_1 - \frac{2y_T}{x' - x_T}$
$\mu_2 \leftarrow \lambda_2 \cdot x_T - y_T$
$x'' \leftarrow \lambda_2^2 - x_T - x'$
$y'' \leftarrow \lambda_2 \cdot (x_T - x'') - y_T$
$\mathtt{ln}_1 \leftarrow \mathsf{Line}(\lambda_1, \mu_1)$
$\mathtt{ln}_2 \leftarrow \mathsf{Line}(\lambda_2, \mu_2)$
$T' \leftarrow (x'', y'')$
return $\mathtt{ln}_1, \mathtt{ln}_2, T'$

lies in the strict subfield of $\mathbb{F}_{q^k}$, namely $\mathbb{F}_q$. It remains to determine how to compute $\ell'_{\psi(Q_1),\psi(Q_2)}(x',y') = 1 + \lambda^\Psi x' + \mu^\Psi y'$ from the line $\ell'_{Q_1,Q_2}(x',y') = 1 + \lambda x' + \mu y'$. This computation depends to the considered curve:

- **For BN254:** Let u be the field element $w^6 - 9$. Then u spans the subfield $\mathbb{F}_{q^2}$, i.e. one can decompose each element z of $\mathbb{F}_q$ as $z := z_0 + u \cdot z_1$, with $z_0, z_1 \in \mathbb{F}_q$. The twisting mapping for the BN254 curve is $\Psi(x,y) \mapsto (w^2 \cdot x, w^3 \cdot y)$, so that we have:

$$\begin{aligned}
\lambda^\Psi &= \frac{\Psi_y(\mathtt{y}(Q_1)) - \Psi_y(\mathtt{y}(Q_1))}{\Psi_x(\mathtt{x}(Q_1)) - \Psi_x(\mathtt{x}(Q_1))} = \frac{w^3 \cdot (\mathtt{y}(Q_1) - \mathtt{y}(Q_1))}{w^2 \cdot (\mathtt{x}(Q_1) - \mathtt{x}(Q_1))} = w \cdot \lambda \\
\mu^\Psi &= \lambda^\Psi \Psi_x(\mathtt{x}(Q_1)) - \Psi_y(\mathtt{y}(Q_1)) = w \cdot w^2 \cdot \mathtt{x}(Q_1) - w^3 \cdot \mathtt{y}(Q_1) = w^3 \cdot \mu.
\end{aligned}$$

  Let us decompose λ and μ as $\lambda := \lambda_0 + u \cdot \lambda_1$ and $\mu := \mu_0 + u \cdot \mu_1$, with $(\lambda_0, \lambda_1, \mu_0, \mu_1) \in \mathbb{F}_q^4$. We then have:

$$\begin{aligned}
\ell'_{\Psi(Q_1),\Psi(Q_2)}(P') &= 1 + \lambda^\Psi x' + \mu^\Psi y' = 1 + w \cdot \lambda x' + w^3 \cdot \mu y' \\
&= 1 + w \cdot (\lambda_0 + u \cdot \lambda_1) x' + w^3 \cdot (\mu_0 + u \cdot \mu_1) y' \\
&= 1 + w \cdot (\lambda_0 + (w^6 - 9) \cdot \lambda_1) x' + w^3 \cdot (\mu_0 + (w^6 - 9) \cdot \mu_1) y' \\
&= 1 + w \cdot (\lambda_0 - 9 \cdot \lambda_1) \cdot x' + w^3 \cdot (\mu_0 - 9 \cdot \mu_1) \cdot y' \\
&\qquad + w^7 \cdot \lambda_1 \cdot x' + w^9 \cdot \mu_1 \cdot y'.
\end{aligned}$$

- **For BLS12-381:** Let u be the field element $w^6 - 1$. Then u spans the subfield $\mathbb{F}_{q^2}$, i.e. we can decompose each element z of $\mathbb{F}_q$ as $z := z_0 + u \cdot z_1$, with $z_0, z_1 \in \mathbb{F}_q$. The twisting mapping for the BN254 curve is $\Psi(x,y) \mapsto (x/w^2, y/w^3)$, so that we have:

$$\begin{aligned}
\lambda^\Psi &= \frac{\Psi_y(\mathtt{y}(Q_1)) - \Psi_y(\mathtt{y}(Q_1))}{\Psi_x(\mathtt{x}(Q_1)) - \Psi_x(\mathtt{x}(Q_1))} = \frac{(\mathtt{y}(Q_1) - \mathtt{y}(Q_1))/w^3}{(\mathtt{x}(Q_1) - \mathtt{x}(Q_1))/w^2} = \frac{1}{w} \cdot \lambda \\
\mu^\Psi &= \lambda^\Psi \Psi_x(\mathtt{x}(Q_1)) - \Psi_y(\mathtt{y}(Q_1)) = \frac{1}{w} \cdot \frac{1}{w^2} \cdot \mathtt{x}(Q_1) - \frac{1}{w^3} \cdot \mathtt{y}(Q_1) = \frac{1}{w^3} \cdot \mu.
\end{aligned}$$

  Let us decompose λ and μ as $\lambda := \lambda_0 + u \cdot \lambda_1$ and $\mu := \mu_0 + u \cdot \mu_1$, with $(\lambda_0, \lambda_1, \mu_0, \mu_1) \in \mathbb{F}_q^4$. We then have:

$$\begin{aligned}
w^3 \cdot \ell'_{\Psi(Q_1),\Psi(Q_2)}(P') &= w^3 \cdot \left(1 + \lambda^\Psi x' + \mu^\Psi y'\right) = w^3 \cdot \left(1 + \frac{1}{w} \cdot \lambda x' + \frac{1}{w^3} \cdot \mu y'\right) \\
&= w^3 + w^2 \cdot (\lambda_0 + u \cdot \lambda_1) x' + (\mu_0 + u \cdot \mu_1) y' \\
&= w^3 + w^2 \cdot (\lambda_0 + (w^6 - 1) \cdot \lambda_1) x' + (\mu_0 + (w^6 - 1) \cdot \mu_1) y' \\
&= (\mu_0 - \mu_1) \cdot y' + w^2 \cdot (\lambda_0 - \lambda_1) \cdot x' + w^3 \\
&\qquad + w^6 \cdot \mu_1 \cdot y' + w^8 \cdot \lambda_1 \cdot x'.
\end{aligned}$$

  Instead of computing $\ell'_{\Psi(Q_1),\Psi(Q_2)}(P')$, one computes $w^3 \cdot \ell'_{\Psi(Q_1),\Psi(Q_2)}(P')$, which do not impact the pairing result since w^3 lives in a strict subfield of $\mathbb{F}_{q^k}$, namely $\mathbb{F}_{q^4}$.

We describe in Algorithm 10 how to compute $\ell'_{\Psi(Q_1),\Psi(Q_2)}(P)$ from ℓ_{Q_1,Q_2} and $P'(x',y')$.

Algorithm 10 Evaluate$_\Psi$

Input: $\ell_{Q_1,Q_2}(x,y) = y - (\lambda x - \mu)$ for some $Q_1, Q_2 \in \mathbb{G}_2 \subset E'(\mathbb{F}_{q^{k/d}})$, $P' = (x', y')$ for some $P := (x, y) \in \mathbb{G}_1 \subset E(\mathbb{F}_q)$

Output: $\ell'_{\Psi(Q_1),\Psi(Q_2)}(P) \in \mathbb{F}_{q^k}$ (for BN254), $w^3 \cdot \ell'_{\Psi(Q_1),\Psi(Q_2)}(P) \in \mathbb{F}_{q^k}$ (for BLS12-381)

1: **if** the curve is BN254 **then**
2: Parse λ as $\lambda_0 + \lambda_1 \cdot u$, for some $\lambda_0, \lambda_1 \in \mathbb{F}_q$
3: Parse μ as $\mu_0 + \mu_1 \cdot u$, for some $\mu_0, \mu_1 \in \mathbb{F}_q$
4: **return** $1 + w \cdot (\lambda_0 - 9\lambda_1)x' + w^3 \cdot (\mu_0 - 9\mu_1) \cdot y' + w^7 \cdot \lambda_1 \cdot x' + w^9 \cdot \mu_1 \cdot y'$

5: **if** the curve is BLS12-381 **then**
6: Parse λ as $\lambda_0 + \lambda_1 \cdot u$, for some $\lambda_0, \lambda_1 \in \mathbb{F}_q$
7: Parse μ as $\mu_0 + \mu_1 \cdot u$, for some $\mu_0, \mu_1 \in \mathbb{F}_q$
8: **return** $(\mu_0 - \mu_1)y' + w^2 \cdot (\lambda_0 - \lambda_1)x' + w^3 + w^6 \cdot \mu_1 \cdot y' + w^8 \cdot \lambda_1 \cdot x'$

2.3.4 Randomized field arithmetic

While one can avoid $\mathbb{F}_{q^k}$-operations in the line evaluation as described above, one still needs $\mathbb{F}_{q^k}$-operations to accumulate the line evaluations in teh variable f returned by the multi Miller loop. To avoid this costly arithmetic in a verification context, one can rely on a *randomized field arithmetic* (RFA). Assume the implementation aims to verify that a polynomial relation $\mathcal{R}$ (with $\mathbb{F}_q$ coefficients) vanishes for some $a_1, \ldots, a_n \in \mathbb{F}_{q^k}$, i.e., $\mathcal{R}(a_1, \ldots, a_n) = 0$. The verification of any (sequence of) $\mathbb{F}_{q^k}$ operations can be expressed as such a statement. The approach implemented in Garaga consists in checking that there exists a polynomial $Q(X) \in \mathbb{F}_q[X]$ such that

$$\mathcal{R}(A_1(X), \ldots, A_n(X)) = P_k(X) \cdot Q(X)$$

where $A_1, \ldots, A_n \in \mathbb{F}_q[X]$ are the degree-$(k-1)$ polynomials representing the $\mathbb{F}_{q^k}$ elements $a_1, \ldots, a_n$ and $P_k(X)$ is the irreducible degree-k polynomial used to build $\mathbb{F}_{q^k}$ as $\mathbb{F}_q[X]/P_k(X)$. To proceed, the prover provides this polynomial $Q(X)$ as hint and the verification checks that the above relation holds for a random point $z \in \mathbb{F}_q$ named the *evaluation challenge*, namely it checks the equality $\mathcal{R}(A_1(z), \ldots, A_n(z)) = P_k(z) \cdot Q(z)$. According to the Schwartz-Zippel lemma, if the polynomial relation was not satisfied, then the evaluated relation does not hold except with probability:

$$\varepsilon_{\text{eval}} := \frac{(k-1) \cdot d}{q} ,$$

where d is the degree of the polynomial relation $\mathcal{R}$.

Batching. Since the implementation involves many $\mathbb{F}_{q^k}$-operations, it has to check several relations $\mathcal{R}_1, \ldots, \mathcal{R}_m$. To check that $\mathcal{R}_j(a_1, \ldots, a_n) = 0$ for some $a_1, \ldots, a_n \in \mathbb{F}_{q^k}$ for all $j \in \{1, \ldots, m\}$, a standard strategy is to linearly batch the relations using coefficients 1, β, β^2, $\ldots$, β^{m-1} for β for a random field element $\beta \in \mathbb{F}_q$ named the *batch challenge*.

Specifically, the implementation then checks that there exists a polynomial $Q(X) \in \mathbb{F}_q[X]$ (still provided as hint) staisfying

$$\sum_{j=1}^{m} \beta^{j-1} \cdot \mathcal{R}_j(A_1(X), \ldots, A_n(X)) = P_k(X) \cdot Q(X) \ . \tag{18}$$

By the Schwartz-Zippel lemma, if $\mathcal{R}_j(a_1, \ldots, a_n) \neq 0$ for at least on j, then the above relation does not hold except with probability:

$$\varepsilon_{\text{batch}} := \frac{m-1}{q} \ .$$

Instead of using the above standard way to batch relations, Garaga defines the batched relation as:

$$\sum_{j=1}^{m} \beta^{2^{j-1}} \cdot \mathcal{R}_j(A_1(X), \ldots, A_n(X)) = P_k(X) \cdot Q(X) \ , \tag{19}$$

namely it uses coefficients $\beta^{2^0}, \ldots, \beta^{2^{m-1}}$ instead of $\beta^0, \ldots, \beta^{m-1}$. In that case, the soundness error obtained from the Schwartz-Zippel lemma is the following:

$$\varepsilon_{\text{batch}} := \frac{2^{m-1}}{q} \ .$$

Observation 3: Batching coefficients in randomized field arithmetic

To batch m relations, Garaga utilizes the coefficients $\beta, \beta^2, \beta^{2^2}, \ldots, \beta^{2^{m-1}}$ for a pseudo-random field element β (output of a hash function). As explained above, it leads to a soundness error of $\frac{2^{m-1}}{q}$, and so prevents to batch many coefficients ($\frac{2^{m-1}}{q}$ should be negligible, i.e. smaller than $2^{-\lambda}$ to achieve a λ-bit security). The current implementation is secure because m is small enough, but this choice may lead to an insecure implementation in the future if larger parameters are used.

Recommendation

We recommend to use lower-degree coefficients, namely to batch the relations using the coefficients $1, \beta, \ldots, \beta^{m-1}$. It leads to a soundness error of $\frac{m-1}{q}$ and so we can take $m < q \cdot 2^{-\lambda}$, while the upper bound is currently $\log_2(q) - \lambda$ (typically ≈ 128).

Challenge Sampling. The verification soundness relies on the fact that the prover can not guess the evaluation and batch challenges before deriving them. In an interactive setting, those challenges would be sampled by the verifier, but in a non-interactive setting, one relies on the Fiat-Shamir transformation: the challenges are the outputs of hash

functions that takes as input the values committed by the prover. The security of the Fiat-Shamir transformation relies on the hardness of predicting a hash output. To be secure, the hash input for the batch challenge should include the field elements $a_1 \ldots, a_n \in \mathbb{F}_{q^k}$, or at least values that fully determine those field elements. Then, the hash input for the batch challenge should include the hash digest of the previous hash, together with the hint polynomial $Q(X)$. We refer the reader to Algorithm 21 for the concrete instantiation of those hashes in the case of the multi-pairing check.

Algorithms. We describe hereafter the basic algorithms implementing the randomized field arithmetic in Garaga. Let v be a field element in $\mathbb{F}_{q^k}$. We define the *reduced representation* $v_{[z]}$ of v in $z \in \mathbb{F}_q$ as $v_{[z]} := V(z)$ where $V(X) \in F_q[X]$ is the degree-$(k-1)$ polynomial representing v, and we denote $\mathsf{Eval}_z(v)$ the evaluation routine of the reduced representation $v_{[z]}$ of v.

We describe the three algorithms used in Garaga to aggregate and check the relations:

- RFA.FirstRelation (Algorithm 11) gets the batch challenge β, together with the first relation in the reduced form $\texttt{rel} := \mathcal{R}_1(a_{1[z]}, \ldots, a_{n[z]})$, and returns a data structure $\texttt{rels}_0$ that represents the aggregated relations.
- RFA.AddRelation (Algorithm 12) gets the data structure $\texttt{rels}_i$ that represents the previous aggregated relations, together with a new relation in the reduced form $\texttt{new_rel} := \mathcal{R}_{i+1}(a_{1[z]}, \ldots, a_{n[z]})$, and returns an updated $\texttt{rels}_{i+1}$.
- RFA.CheckAll (Algorithm 13) gets the data structure $\texttt{rels}_i$ that represents the aggregated relations, together with the hint polynomial $Q(X)$ and returns a field element that is equal to *zero* if the checked relations are valid.

Algorithm 11 RFA.FirstRelation

Input: $\beta \in \mathbb{F}_q, \texttt{rel} \in \mathbb{F}_q$

1: $\beta_0 \leftarrow \beta$
2: $\texttt{lhs}_0 \leftarrow \beta_0 \cdot \texttt{rel}$
3: **return** $\texttt{rels}_0 := (\texttt{lhs}_0, \beta_0)$

Algorithm 12 RFA.AddRelation

Input: $\texttt{rels}_i := (\texttt{lhs}_i, \beta_i), \texttt{new_rel} \in \mathbb{F}_q$

1: $\beta_{i+1} \leftarrow \beta_i^2$
2: $\texttt{lhs}_{i+1} \leftarrow \texttt{lhs}_i + \beta_{i+1} \cdot \texttt{new_rel}$
3: **return** $\texttt{rels}_{i+1} := (\texttt{lhs}_{i+1}, \beta_{i+1})$

2.3.5 Detailed Miller loop iterations

In some contexts, the points Q_j are known in advance, e.g. they are fixed by the application. This is the case, for example, when verifying a Groth16 proof. When a point Q_j is known,

Algorithm 13 RFA.CheckAll

Input: $\texttt{rels}_i := (\texttt{lhs}_i, \beta_i), Q(X) \in \mathbb{F}_q[X]$

1: $\texttt{final_check} \leftarrow \texttt{lhs}_i - Q(z) \cdot P_k(z)$

2: **return** `final_check`

all the lines involved in the Miller loop can be precomputed ahead of time. Consequently, the verifier can compute these values *offline* and reuse them during the execution of the Miller loop.

Assume that the first n_f points Q_j are known in advance. Algorithm 14 describes a single iteration of the multi Miller loop. The algorithm distinguishes between two cases based on the current bit in the scalar. It uses precomputed line values for each pair (P_j, Q_j) where $j < n_f$. The algorithm takes the following inputs:

- $f_{[z]}$: the reduced representation of the accumulator at the beginning of the iteration;
- for all $j < n_f$, the precomputed line evaluations corresponding to the known Q_j;
- for all $j \geq n_f$, the point T_j needed to compute line functions when Q_j is not known in advance;
- `mml_data`: iteration-independent data, including the points $\{\hat{P}_j\}_j$ and $\{Q_j\}_j$, the reduced representation ${c^{(\text{inv})}}_{[z]}$ of c^{-1}, and the challenge z used for randomized field arithmetic.

Since the accumulator operates directly in the reduced representation under randomized field arithmetic over $\mathbb{F}_{q^k}$, each line evaluation is first reduced before being multiplied into the accumulator. The algorithm returns the updated reduced representation of the accumulator at the end of the iteration, along with the updated points $\{T_j\}_j$ required for subsequent iterations.

It is worth noting that [NE24] also suggests using precomputed line evaluations even when the points Q_j are not known in advance. In this case, the prover supplies the precomputed lines, and the verifier must ensure that they have been correctly constructed. Although verifying the correctness of these lines is computationally cheaper than computing them from scratch, it introduces additional communication between the prover and the verifier. Furthermore, in a non-interactive setting, these precomputed lines must be included in the hash used to derive the challenges for the randomized field arithmetic. As a result, the overall efficiency gain from this optimization is uncertain. For these reasons, the Garaga library does not implement this variant, i.e., it does not use prover-supplied line evaluations.

The accumulator in the Miller loop is computed using randomized field arithmetic, which is designed to verify certain relations over elements of $\mathbb{F}_{q^k}$. The prover provides the value $r_{i+1[z]}$ –the accumulator after several Miller loop iterations– in reduced form. The verifier then checks that the computed reduced representation $f_{[z]}$ is consistent with

Algorithm 14 MML-Iteration$_b$

Input: $\mathtt{in}_{[z]}$, $\{T_j\}_{j \geq n_f}$, $\{\mathtt{lines}_j\}_{j < n_f}$, $\mathtt{mml_data}$

Parse $\mathtt{mml_data}$ as $\left(\{P'_j\}_j, \{Q_j\}_{j \geq n_f}, z, {c^{(\mathrm{inv})}}_{[z]}\right)$
$f_{[z]} \leftarrow \mathtt{in}^2_{[z]}$
if b is $\mathtt{0}$ **then**
  for $j \in \{0, \ldots, n_p - 1\}$ **do**
    if $j \geq n_f$ **then**
      $(\mathtt{ln}_j, T'_j) \leftarrow \mathsf{DoubleStep}(T_j)$ ▷ $\mathtt{ln}_j := \ell_{T_j,T_j}, T'_j := [2]T_j$
    else
      Parse $\mathtt{lines}_j$ as $\mathtt{ln}_j$
    $g \leftarrow \mathsf{Evaluate}_\Psi(\mathtt{ln}_j, P'_j)$ ▷ $g := \ell'_{\Psi(T_j),\Psi(T_j)}(P'_j)$
    $f_{[z]} \leftarrow f_{[z]} \cdot \mathsf{Eval}_z(g)$
if b is $\mathtt{1}$ **then**
  $f_{[z]} \leftarrow f_{[z]} \cdot {c^{(\mathrm{inv})}}_{[z]}$
  for $j \in \{0, \ldots, n_p - 1\}$ **do**
    if $j \geq n_f$ **then**
      $(\mathtt{ln}_{j,1}, \mathtt{ln}_{j,2}, T'_j) \leftarrow \mathsf{DoubleAndAddStep}(T_j, Q_j)$
      ▷ $\mathtt{ln}_{j,1} := \ell_{T_j,Q_j}, \mathtt{ln}_{j,2} := \ell_{T_j+Q_j,T_j}, T'_j := [2]T_j + Q_j$
    else
      Parse $\mathtt{lines}_j$ as $(\mathtt{ln}_{j,1}, \mathtt{ln}_{j,2})$
    $g_1 \leftarrow \mathsf{Evaluate}_\Psi(\mathtt{ln}_{j,1}, P'_j)$ ▷ $g_1 := \ell'_{\Psi(T_j),\Psi(Q_j)}(P'_j)$
    $g_2 \leftarrow \mathsf{Evaluate}_\Psi(\mathtt{ln}_{j,2}, P'_j)$ ▷ $g_2 := \ell'_{\Psi(T_j+Q_j),\Psi(T_j)}(P'_j)$
    $f_{[z]} \leftarrow f_{[z]} \cdot \mathsf{Eval}_z(g_1) \cdot \mathsf{Eval}_z(g_2)$
return $f_{[z]}, \{T'_j\}_{j \geq n_f}$

this value, by verifying the relation $f - r_{i+1} = 0$ using the verification procedure of the randomized field arithmetic.

Algorithm 15 describes the execution of multiple Miller loop iterations with batched random field arithmetic. It takes as input both the initial accumulator value $f_{[z]}$ and the claimed final value $r_{i+1[z]}$, each in reduced form. The algorithm recomputes the final value by repeatedly invoking Algorithm 14, and then verifies consistency by comparing the recomputed and provided accumulator values. This final verification step is performed using the batched randomized field arithmetic check.

2.3.6 Detailed multi-pairing check

Let us now provide a detailed algorithmic description of Garaga's multi-pairing check.

Algorithm 15 MML-BatchedIterations$_{\vec{b}}$

Input: $(\texttt{in}_{[z]}, \texttt{out}_{[z]}), \{T_j\}_{j \geq n_f}, \{\texttt{lines}_{h,j}\}_{h,j<n_f}, \texttt{mml_data}, \texttt{rels}_i$

$f_{[z]} \leftarrow \texttt{in}_{[z]}$
for $h \in \{1, \ldots, |\vec{b}|\}$ **do**
  $f_{[z]}, \{T_j\}_{j \geq n_f} \leftarrow \text{MML-Iteration}_{b_h}(f_{[z]}, \{T_j\}_{j \geq n_f}, \{\texttt{lines}_{h,j}\}_{j<n_f}, \texttt{mml_data})$
$\texttt{rels}_{i+1} \leftarrow \text{RFA.AddRelation}(\texttt{rels}_i, f_{[z]} - \texttt{out}_{[z]})$
return $\{T_j\}_{j \geq n_f}, \texttt{rels}_{i+1}$

Multi-pairing check initialization. We describe in Algorithm 16 a preprocessing step that derives all the useful quantities relative to the λ-th root c. It takes as input $c, c^{(\text{inv})} \in \mathbb{F}_{q^k}$ and $u \in \mathbb{F}_{q^{k'}}$, together with the two challenges of the randomized field arithmetic (the evaluation point z and the batching challenge β). It outputs:

- the reduced representations $c_{[z]}, c^{(\text{inv})}{}_{[z]}, u_{[z]}$ respectively of c, c^{-1} and u;
- for all $1 \leq i \leq l$, the reduced representation $c^{(\text{frob},i)}{}_{[z]}$ of $(c^{\epsilon_i})^{-q^i}$ which will be used after the Miller loop.

It also checks that $c^{(\text{inv})}$ is the inverse of c using RFA.FirstRelation.

Algorithm 16 InitializeMultiPairingCheck

Input: $c, c^{(\text{inv})} \in \mathbb{F}_{q^k}$, $u \in \mathbb{F}_{k'}$, $z, \beta \in \mathbb{F}_q$
Output: $c_{[z]}, c^{(\text{inv})}{}_{[z]}, u_{[z]}, \mathbf{c}^{(\text{frob})}{}_{[z]}, \texttt{lhs}$

$c_{[z]} \leftarrow \text{Eval}_z(c)$
$c^{(\text{inv})}{}_{[z]} \leftarrow \text{Eval}_z(c^{(\text{inv})})$
$\texttt{rels}_0 \leftarrow \text{RFA.FirstRelation}(\beta, c_{[z]} \cdot c^{(\text{inv})}{}_{[z]} - 1)$ ▷ Check that $c \cdot c^{(\text{inv})} = 1$
$u_{[z]} \leftarrow \text{Eval}_z(u)$
for $i = 1$ **to** l **do**
  if $\epsilon_i = 1$ **then**
    $c^{(\text{frob},i)}{}_{[z]} \leftarrow \text{Eval}_z\left((c^{(\text{inv})})^{q^i}\right)$
  else if $\epsilon_i = -1$ **then**
    $c^{(\text{frob},i)}{}_{[z]} \leftarrow \text{Eval}_z\left(c^{q^i}\right)$
$\mathbf{c}^{(\text{frob})}{}_{[z]} \leftarrow \{c^{(\text{frob},i)}{}_{[z]}\}_i$
return $c_{[z]}, c^{(\text{inv})}{}_{[z]}, u_{[z]}, \mathbf{c}^{(\text{frob})}{}_{[z]}, \texttt{rels}_0$

Initializing accumulators & first iteration. Before running the Miller loop, one needs to initialize f and $\{T_j\}_j$. As described in Algorithm 5, f is set as c^{-1} and $\{T_j\}_j$ as $\{Q_j\}_j$. The Garaga library has a specific function that initializes those variables (in the reduced

form for f) and directly runs the first Miller loop iterations. This initialization step is described in Algorithm 17.

Algorithm 17 MML-FirstIterations

Input: $\texttt{out}_{[z]}, \{\texttt{lines}_{h,j}\}_{h,j<n_f}, \texttt{mml_data}, \texttt{rels}_i$

1: Parse $\texttt{mml_data}$ as $\left(\{P'_j\}_j, \{Q_j\}_{j\geq n_f}, z, c^{(\text{inv})}{}_{[z]}\right)$
2: $f_{[z]} \leftarrow c^{(\text{inv})}{}_{[z]}$
3: $\{T_j\}_{j\geq n_f} \leftarrow \{Q_j\}_{j\geq n_f}$
4: **if** the curve is BN254 **then**
5: $\quad f_{[z]}, \{T_j\}_{j\geq n_f} \leftarrow \text{MML-Iteration}_0(f_{[z]}, \{T_j\}_{j\geq n_f}, \{\texttt{lines}_{h,j}\}_{j<n_f}, \texttt{mml_data})$
6: **if** the curve is BLS12-381 **then**
7: $\quad f_{[z]}, \{T_j\}_{j\geq n_f} \leftarrow \text{MML-Iteration}_1(f_{[z]}, \{T_j\}_{j\geq n_f}, \{\texttt{lines}_{h,j}\}_{j<n_f}, \texttt{mml_data})$
8: $\texttt{rels}_{i+1} \leftarrow \text{RFA.AddRelation}(\texttt{rels}_i, f_{[z]} - \texttt{out}_{[z]})$
9: **return** $\{T_j\}_{j\geq n_f}, \texttt{rels}_{i+1}$

Finalizing multi-pairing check. After running the Miller loop, it remains to compute the term

$$v := \prod_{i=1}^{l}\prod_{j=1}^{n_p} \ell_{\Psi([s+\sum_{h=1}^{i-1}\epsilon_h\cdot q^h]Q_j),\Psi([\epsilon_i q^i]Q_j)}(P_j) \cdot \prod_{i=1}^{l} c^{\epsilon_i\cdot q^i} \cdot u$$

to check $f \cdot v = 1$, where $f := c^{-s} \cdot \prod_{j=1}^{n_p} f_{s,Q_j}(P_j)$ is the output of the Miller loop. Let us observe that the last computed line (for each pair j) is $\ell_{\Psi(T_j),\Psi(T'_j)}$ with

$$T_j := [s + \sum_{h=1}^{\ell-1}\epsilon_h \cdot q^h]Q_j$$
$$T'_j := [\epsilon_i q^i]Q_j \ .$$

We have that $T_j + T'_j = [s + \sum_{h=1}^{\ell}\epsilon_h]Q_j = [\lambda]Q_j = [m]\left([r]Q_j\right) = \mathcal{O}$, since $Q_j \in \mathbb{G}_2$ and $\mathbb{G}_2$ is of order r. Therefore, $\ell_{\Psi(T_j),\Psi(T'_j)}$ is the vertical line $v_{\Psi(T_j)}$, and so $\ell_{\Psi(T_j),\Psi([\epsilon_i]Q_j)}(P_j)$ lives in a strict subfield of $\mathbb{F}_{q^k}$ and does not impact the pairing results. One can thus ignore the last line evaluations and compute v as

$$v = \prod_{i=1}^{l-1}\prod_{j=1}^{n_p} \ell_{\Psi([s+\sum_{h=1}^{i-1}\epsilon_h\cdot q^h]Q_j),\Psi([c_i q^i]Q_j)}(P_j) \cdot \prod_{i=1}^{l} c^{\epsilon_i\cdot q^i} \cdot u \ .$$

We describe in Algorithm 18 the verification of the above computation. More precisely, this algorithm takes reduced representations $\texttt{in}_{[z]}$ and $\texttt{out}_{[z]}$ of $\texttt{in} \in \mathbb{F}_{q^k}$ and $\texttt{out} \in \mathbb{F}_{q^k}$, and checks:

$$\texttt{out} = \texttt{in} \cdot \prod_{i=1}^{l-1}\prod_{j=1}^{n_p} \ell_{\Psi([s+\sum_{h=1}^{i-1}\epsilon_h\cdot q^h]Q_j),\Psi([\epsilon_i q^i]Q_j)}(P_j) \ .$$

To compute $[q]Q_j$, it relies on the fact that the group $\mathbb{G}_2$ is built as the "trace-zero subgroup" which is such that $[q]Q = \pi_q(Q)$ for all $Q \in \mathbb{G}_2$, where $\pi_q : (x, y) \mapsto (x^q, y^q)$ is the Frobenius endomorphism.

Algorithm 18 EvaluateLastLines

Input: $(\texttt{in}_{[z]}, \texttt{out}_{[z]})$, $\{T_j\}_{j \geq n_f}$, $\{\texttt{lines}_{h,j}\}_{h,j<n_f}$, $\texttt{mml_data}$, $\texttt{rels}_i$
Output: $\texttt{rels}_{i+1}$

1: Parse $\texttt{mml_data}$ as $\left(\{P'_j\}_j, \{Q_j\}_{j \geq n_f}, z, c^{(\text{inv})}{}_{[z]}\right)$
2: $f_{[z]} \leftarrow \texttt{in}_{[z]}$
3: **for** $i = 1$ **to** $l - 1$ **do**
4: **for** $j = 1$ **to** n_p **do**
5: $Q_j \leftarrow \pi_q(Q_j)$
6: **if** $j \geq n_f$ **then**
7: $(\texttt{ln}_j, T'_j) \leftarrow \mathsf{AddStep}(T_j, [\epsilon_i]Q_j)$ ▷ $\texttt{ln}_j := \ell_{T_j,[\epsilon_i]Q_j}, T'_j := T_j + [\epsilon_i]Q_j$
8: **else**
9: Parse $\texttt{lines}_{i,j}$ as $\texttt{ln}_j$
10: $g \leftarrow \mathsf{Evaluate}_\Psi(\texttt{ln}_j, P'_j)$ ▷ $g := \ell'_{\Psi(T_j),\Psi([\epsilon_i]Q_j)}(P'_j)$
11: $f_{[z]} \leftarrow f_{[z]} \cdot \mathsf{Eval}_z(g)$
12: $\texttt{rels}_{i+1} \leftarrow \mathsf{RFA.AddRelation}(\texttt{rels}_i, f_{[z]} - \texttt{out}_{[z]})$
13: **return** $\texttt{rels}_{i+1}$

We then describe in Algorithm 19 the scaling of the result of the multi Miller loop to avoid the final exponentiation. Following the approach described in Section 2.3.2, this algorithm takes reduced representations $\texttt{in}_{[z]}$ and $\texttt{out}_{[z]}$ of $\texttt{in} \in \mathbb{F}_{q^k}$ and $\texttt{out} \in \mathbb{F}_{q^k}$, and checks:

$$\texttt{out} = \texttt{in} \cdot \prod_{i=1}^{l} c^{\epsilon_i \cdot q^i} \cdot u \ .$$

Algorithm 19 ScaleMillerLoopResult

Input: $(\texttt{in}_{[z]}, \texttt{out}_{[z]})$, $\mathbf{c}^{(\text{frob})}{}_{[z]}$, $u_{[z]}$, $\texttt{rels}_i$
Output: $\texttt{rels}_{i+1}$

1: $f_{[z]} \leftarrow \texttt{in}_{[z]} \cdot u_{[z]}$
2: Parse $\mathbf{c}^{(\text{frob})}{}_{[z]}$ as $\{c^{(\text{frob},i)}{}_{[z]}\}_i$
3: **for** $i = 1$ **to** l **do**
4: $f_{[z]} \leftarrow f_{[z]} \cdot c^{(\text{frob,i})}{}_{[z]}$ ▷ $c^{(\text{frob,i})}{}_{[z]} := \mathsf{Eval}_z\left((c^{\epsilon_i})^{-q^i}\right)$
5: $\texttt{rels}_{i+1} \leftarrow \mathsf{RFA.AddRelation}(\texttt{rels}_i, f_{[z]} - \texttt{out}_{[z]})$
6: **return** $\texttt{rels}_{i+1}$

Finally, Algorithm 20 describes how to perform the final verification step of the multi-pairing check using the output of the Miller loop. It first verifies that the final line evalu-

ations and the result scaling have been correctly computed by invoking EvaluateLastLines and ScaleMillerLoopResult, respectively. It then completes the verification of all batched relations from the randomized field arithmetic by calling RFA.CheckAll with the hint polynomial $Q(X)$. If all checks pass, the algorithm returns zero.

Algorithm 20 CheckFinalize

Input: $(r_{n-3[z]}, r_{n-2}, r_{n-1}), \{T_j\}_{j \geq n_f}, \{\texttt{lines}_{h,j}\}_{h,j<n_f}, \texttt{mml_data}, \mathbf{c}^{(\text{frob})}{}_{[z]}, u_{[z]}, Q(X), \texttt{rels}_i$

1: $r_{n-2[z]} = \mathsf{Eval}_z(r_{n-2})$
2: $r_{n-1[z]} = \mathsf{Eval}_z(r_{n-1})$
3: $\texttt{inputs} \leftarrow \left(r_{n-3[z]}, r_{n-2[z]}, \{T_j\}_{j \geq n_f}, \{\texttt{lines}_{h,j}\}_{h,j<n_f}, \texttt{mml_data}, \texttt{rels}_i\right)$
4: $\texttt{rels}_{i+1} \leftarrow \mathsf{EvaluateLastLines}(\texttt{inputs})$
5: $\texttt{rels}_{i+2} \leftarrow \mathsf{ScaleMillerLoopResult}(r_{n-2[z]}, r_{n-1[z]}, \mathbf{c}^{(\text{frob})}{}_{[z]}, u_{[z]}, \texttt{rels}_{i+1})$
6: $\texttt{final_check} \leftarrow \mathsf{RFA.CheckAll}(\texttt{rels}_{i+2}, Q(X))$
7: **return** `final_check`

Complete multi-pairing check. We give in Algorithm 21 the complete procedure to check that the input pairs of points $(P_1, Q_1), \ldots, (P_{n_p}, Q_{n_p})$ satisfy

$$\prod_{j=1}^{n} e(P_j, Q_j) = 1 \ .$$

Together with the pairs of points, it takes as input a queue structure `all_lines` that contains all the precomputed lines for the points Q_j that are known in advance. We denote by `all_lines.pop()` the procedure that extracts the first element of the queue (that corresponds to a set of lines). It also takes as input few hints: the λ-th root c of the Miller loop output, its inverse $c^{(\text{inv})} := c^{-1}$, the scaling factor $u \in \mathbb{F}_{q^{k'}} \subset \mathbb{F}_{q^k}$, some intermediate values $r_0, \ldots, r_{L+2} \in \mathbb{F}_{q^k}$ of the Miller loop accumulator, and the hint polynomial $Q(X)$ to check the relations over the randomized field arithmetic. The multi-pairing check proceeds as follows:

1. It computes the RFA challenges, namely the batching challenge β and the evaluation challenge z. First, β is computed by hashing the input pairs of points $\{(P_j, Q_j)\}_j$, the hint values $(c, c^{(\text{inv})}, u)$ for the check of the Miller loop result, and the intermediate values $\{r_i\}_i$ of the Miller loop accumulator. We note that these values fully determine all values involved in the RFA relations, so no additional values need to be included in the hash input. Then, z is computed as the hash digest of β and the hint polynomial $Q(X)$.

2. It prepares all the variables that will be useful in the Miller loop. More precisely, it computes the reduced representation of all the data relative the decomposition $c^{\lambda} \cdot u^{-1}$ of the Miller loop output using InitializeMultiPairingCheck and it precomputes

$P'_j := (x'_j, y'_j)$ as $x'_j = \frac{1}{\mathtt{y}(P_j)}$ and $y'_j = -\frac{\mathtt{x}(P_j)}{\mathtt{y}(P_j)}$. Then, it gathers in all the constant values useful for computing the Miller loop in the data structure `mml_data`.

3. It checks the accumulator value after the first Miller loop iteration is equal to r_0 using MML-FirstIterations. Then, it checks that the accumulator value after the i-th batch of iterations is equal to r_{i+1} using MML-BatchedIterations.

4. Finally, using CheckFinalize, it checks that the last line evaluations are well-computed and that the multi-pairing result belongs to $\mathbb{F}_{q^k}^*/(\mathbb{F}_q^k)^r$ thanks to the decomposition $c^\lambda \cdot u^{-1}$. At the same time, it checks that all the RFA relations.

Signed loop iterations. Algorithm 21 further uses a signed representation of the Miller loop "exponent". In the standard Miller algorithm (see Algorithm 5), an iteration is parametrized by a bit $b \in \{0,1\}$ and based on the recursive relation

$$\mathtt{f}_{2a+b} = (c^{-1})^b \cdot \mathtt{f}_a^2 \cdot \frac{\ell_{[a]Q,[a]Q}(P) \cdot \ell_{[2a]Q,[b]Q}(P)}{v_{[2a+b]Q}(P)}$$

where $\mathtt{f}_x := (c^{-x} \cdot f_{x,Q}(P))$. The iterations of the Miller loop in Algorithm 21 additionally rely on a sign parameter $\mathtt{sign} \in \{-1,+1\}$ and compute $\mathtt{f}_{2a+\mathtt{sign}\cdot b} := c^{2a+\mathtt{sign}\cdot b} \cdot f_{2a+\mathtt{sign}\cdot b,Q}(P)$ from $\mathtt{f}_{2a+b}$. While $\mathtt{sign} = +1$ leads to the same formula than before, $\mathtt{sign} = -1$ has a similar recursive relative except that c^{-1} and Q are respectively replaced by c and $-Q$. The advantage of introducing this parameter is to give more flexibility in the bit decomposition. As described throught the routine MML-BatchedIterations, the verification of several consecutive iterations is batched. In this decomposition of the Miller loop "exponent", the same $\mathtt{sign}_i \in \{0,1\}$ is used for all the bits in $\mathtt{bits}_i$, enabling a simple replacement of the inputs $c^{(\mathrm{inv})}$ and Q of MML-BatchedIterations by c and $-Q$ whenever $\mathtt{sign}_i = 1$. This is the role of `mml_data_neg`, that contains the same data than `mml_data` except for $(c^{(\mathrm{inv})}{}_{[z]}, \{Q_j\})_j$ which is replaced by $(c_{[z]}, \{-Q_j\})_j$.

We give in Table 2 the bit decompositions $(\mathtt{sign}_i, \mathtt{bits}_i)_{i\in\{0,\ldots,L-1\}}$ used for BN254 and BLS12-381. We can verify that those two decompositions satisfy $\lambda = m_L$ with the recursive relation:

$$\forall i \in \{0,\ldots,L-1\},\ m_{i+1} = m_i \cdot 2^{|\mathtt{bits}_i|} + \mathtt{sign}_i \cdot \sum_{h=1}^{|\mathtt{bits}_i|} (\mathtt{bits}_i)_h \cdot 2^{h-1}$$

with $m_0 := 2$ for BN254 and $m_0 := 3$ for BLS12-381.

Degree of the RFA hint polynomial. The degree of the hint $Q(X)$ depends on the maximum degree of checked relations: by denoting $\deg_{\mathrm{line}}$ the degree of the field element (when interpreted as a polynomial of w) outputed by $\mathsf{Evaluate}_\Psi$, we have that

- the verification that $c^{(\mathtt{inv})}$ is indeed the inverse of c in InitializeMultiPairingCheck leads to a relation degree

$$\deg_{\mathrm{inv}} := 2 \cdot (k-1) - k\ ;$$

i	$\texttt{sign}_i$	$\texttt{bits}_i$
0	-1	10
1	$+1$	10
2	$+1$	00
3	-1	10
4	-1	10
5	$+1$	00
6	-1	10
7	$+1$	01
8	$+1$	10
9	-1	01
10	$+1$	00
11	$+1$	00
12	$+1$	01
13	$+1$	00
14	-1	10
15	$+1$	10
16	-1	01
17	$+1$	00
18	$+1$	00
19	-1	10
20	$+1$	10
21	$+1$	00
22	-1	10
23	-1	10
24	$+1$	01
25	$+1$	00
26	-1	01
27	$+1$	00
28	-1	01
29	-1	01
30	-1	10
31	$+1$	00

(a) BN254

i	$\texttt{sign}_i$	$\texttt{bits}_i$
0	$+1$	0
1	$+1$	1
2	$+1$	00
3	$+1$	1
4	$+1$	00
5	$+1$	00
6	$+1$	00
7	$+1$	00
8	$+1$	1
9	$+1$	00
10	$+1$	00
11	$+1$	00
12	$+1$	00
13	$+1$	00
14	$+1$	00
15	$+1$	00
16	$+1$	00
17	$+1$	00
18	$+1$	00
19	$+1$	00
20	$+1$	00
21	$+1$	00
22	$+1$	00
23	$+1$	00
24	$+1$	0
25	$+1$	1
26	$+1$	00
27	$+1$	00
28	$+1$	00
29	$+1$	00
30	$+1$	00
31	$+1$	00
32	$+1$	00
33	$+1$	00

(b) BLS12-381

Table 2: Bit decomposition of λ

Algorithm 21 MultiPairingCheck

Input: $(P_1, Q_1), \ldots, (P_{n_p}, Q_{n_p}) \in \mathbb{G}_1 \times \mathbb{G}_2$
Precomputation: a queue `ln_queue` that contains all the precomputed lines over $\mathbb{G}_2$
Hint: $c, c^{(\text{inv})} \in \mathbb{F}_{q^k}$, $u \in \mathbb{F}_{q^{k'}}$, $(r_0, \ldots, r_{L+2}) \in \mathbb{F}_{q^k}^{L+3}$ and $Q(X) \in \mathbb{F}_q[X]$.

▷ Derive the batching and evaluation challenges.
$\beta = \text{Hash}(\{(P_j, Q_j)\}_j, c, c^{(\text{inv})}, u, \{r_i\}_i)$ ▷ $\beta \in \mathbb{F}_q$
$z = \text{Hash}(\beta, Q(X))$ ▷ $z \in \mathbb{F}_q$

▷ Miller Loop – Initialization.
$c_{[z]}, c^{(\text{inv})}{}_{[z]}, u_{[z]}, \mathbf{c}^{(\text{frob})}{}_{[z]}, \texttt{rels} \leftarrow \mathsf{InitializeMultiPairingCheck}(c, c^{(\text{inv})}, u, z, \beta)$
for $j = 1$ **to** n_p **do**
  $P'_j \leftarrow \left(\frac{1}{\mathtt{y}(P_j)}, -\frac{\mathtt{x}(P_j)}{\mathtt{y}(P_j)}\right)$
$\texttt{mml_data} = \left(\{P'_j\}_j, \{Q_j\}_{j \geq n_f}, z, c^{(\text{inv})}{}_{[z]}\right)$
$\texttt{mml_data_neg} = \left(\{P'_j\}_j, \{-Q_j\}_{j \geq n_f}, z, c_{[z]}\right)$

▷ Miller Loop
$r_{0[z]} \leftarrow \mathsf{Eval}_z(r_0)$
$\{T_j\}_{j \geq n_f}, \texttt{rels} \leftarrow \mathsf{MML\text{-}FirstIterations}(r_{0[z]}, \texttt{ln_queue.pop()}, \texttt{mml_data}, \texttt{rels})$
for $i = 0$ **to** $L - 1$ **do**
  $r_{i+1[z]} \leftarrow \mathsf{Eval}_z(r_{i+1})$
  if $\texttt{sign}_i = 1$ **then**
    $\texttt{inputs} \leftarrow \left((r_{i[z]}, r_{i+1[z]}), \{T_j\}_{j \geq n_f}, \texttt{ln_queue.pop()}, \texttt{mml_data}, \texttt{rels}\right)$
    $(\{T_j\}_{j \geq n_f}, (\texttt{lhs}_{i+1}, \beta_{i+1})) \leftarrow \mathsf{MML\text{-}BatchedIterations}_{\texttt{bits}_i}(\texttt{inputs})$
  else if $\texttt{sign}_i = -1$ **then**
    $\texttt{inputs} \leftarrow \left((r_{i[z]}, r_{i+1[z]}), \{T_j\}_{j \geq n_f}, \texttt{ln_queue.pop()}, \texttt{mml_data_neg}, \texttt{rels}\right)$
    $(\{T_j\}_{j \geq n_f}, (\texttt{lhs}_{i+1}, \beta_{i+1})) \leftarrow \mathsf{MML\text{-}BatchedIterations}_{\texttt{bits}_i}(\texttt{inputs})$

▷ Final Checks
$\texttt{st} \leftarrow (r_{L[z]}, r_{L+1}, r_{L+2})$
$\texttt{inputs} \leftarrow \left(\texttt{st}, \{T_j\}_{j \geq n_f}, \texttt{ln_queue.pop()}, \texttt{mml_data}, \mathbf{c}^{(\text{frob})}{}_{[z]}, u_{[z]}, Q(X), \texttt{rels}\right)$
$\texttt{check} \leftarrow \mathsf{CheckFinalize}(\texttt{inputs})$
Check that $\texttt{check} = 0$
Check that $r_{L+2} = 1$

- the verification that the accumulator value is well-computed in a batch of Miller loop iterations for some bits bits is of relation degree $\deg_{\mathsf{bits}}$, where $\deg_{\mathsf{bits}}$ is defined by induction as

$$\deg_{\mathsf{bits}} := -k + \begin{cases} k - 1 & \text{if } |\mathsf{bits}| = 0 \\ 2 \cdot \deg_{\mathsf{bits}'} + (b \cdot (k-1)) + (1+b) \cdot \deg_{\text{line}} \cdot n_p & \text{otherwise} \end{cases}$$

  where $\mathsf{bits} := \mathsf{bits}' \parallel b$ with $b \in \{0, 1\}$ (when $|\mathsf{bits}| > 1$);

- the verification that the last line evaluations are well-computed leads to a relation degree
$$\deg_{\text{last-evals}} = (k-1) + (l-1) \cdot n_p \cdot \deg_{\text{line}} - k \quad ;$$
- the verification that the Miller loop output has been honestly scaled is of relation degree
$$\deg_{\text{scale}} = (k-1) + (k-2) + l \cdot (k-1) - k \ .$$

We then have

$$\deg Q(X) = \max\left(\{\deg_{\mathsf{bits}_i}\}_i, \deg_{\text{inv}}, \deg_{\text{last-evals}}, \deg_{\text{scale}}\right) \ .$$

We give in Tables 3 and 4 the degree of all the RFA relations for some values n_p, together with the degree of $Q(X)$.

	BN254	BLS12-381
L	32	34
$\#\{r_i\}_i$	35	37
$\deg_{\text{line}}$	9	8
$\deg_{\text{inv}}$	10	10
$\deg_{\text{scale}}$	42	20

Table 3: Degrees of n_p-independent RFA relations.

	$n_p = 1$		$n_p = 2$		$n_p = 3$	
	BN254	BLS12-381	BN254	BLS12-381	BN254	BLS12-381
$\deg_{\text{last-evals}}$	17	-	35	-	53	-
$\deg_{\mathsf{0}}$	19	18	28	26	37	34
$\deg_{\mathsf{1}}$	39	37	57	53	75	69
$\deg_{\mathsf{00}}$	59	56	86	80	113	104
$\deg_{\mathsf{01}}$	79	75	115	107	151	139
$\deg_{\mathsf{10}}$	99	94	144	134	189	174
$\deg_{\mathsf{11}}$	119	113	173	161	227	209
$\deg_{\mathsf{000}}$	139	132	202	188	265	244
$\deg Q(X)$	99	56	144	80	189	104
Cost Hash β	453	477	456	480	459	483
Cost Hash z	101	58	146	82	191	106

Table 4: Degrees of n_p-dependent RFA relations and hashing cost.

Hashing cost. Finally, let us exhibit the hashing cost (in terms of hash input size) of the two hash operations involved in the multi-pairing check, one to derive the batching

challenge β, the other to derive the evaluation challenge z. The first hash operation consists of hashing

$$\underbrace{n_p \cdot 3}_{\{(P_j,Q_j)\}_j} + \underbrace{2k + k'}_{c,c^{(\mathrm{inv})},u} + \underbrace{(L+3) \cdot k}_{\{r_i\}_i}$$

$\mathbb{F}_q$-elements, while the second hash operations consists of hashing

$$\underbrace{1}_{\beta_0} + \underbrace{\deg Q(X) + 1}_{Q(X)}$$

$\mathbb{F}_q$-elements. Table 4 further illustrates these quantities for the BN254 and BLS12-381 curves and different n_p values.

2.4 Auxiliary functionalities

2.4.1 Digital signature schemes

Garaga libary implements two EC-based signature schemes, namely EC-Schnorr and ECDSA. The key pair generation algorithm is the same for both schemes and works as follows. Let G be a public generator of the prime subgroup of order n. The key generation proceeds as follows:

1. Sample a scalar d from the set $\{1, \ldots, n-1\}$.
2. Compute $P \leftarrow [d]G$.
3. If $\mathtt{y}(P)$ is odd, set P as $-P$.
4. Output d as the private key and P as the public key.

The third instruction aims to ensure that $\mathtt{y}(P)$ is even, which is a requirement to be compatible with BIP340 [WNR20]. Indeed, either $\mathtt{y}(P)$ or $\mathtt{y}(-P)$ is odd, since $\mathtt{y}(P) + \mathtt{y}(-P) = 0 \bmod q$.

Elliptic Curve Based Schnorr Signature Algorithm – EC-Schnorr. In what follows, we describe the EC-Schnorr signature scheme, based on BIP340 [WNR20]. Given a private key d and a message m, the signing algorithm proceeds as follows:

1. Sample a scalar k from the set $\{1, \ldots, n-1\}$.
2. Compute $R \leftarrow [k]G$ and set x_R as $x_R := \mathtt{x}(R)$. If $\mathtt{y}(R)$ is odd, replace k by $n-k$.
3. Compute a scalar e as the output of a hash function that takes as inputs m and x_R:

   $$e \leftarrow \mathrm{Hash}(m, x_R) \in \{0, \ldots, n-1\} \ ;$$

   If $e = 0$, go to Step 1.

4. Compute $s = k - e \cdot d \bmod n$. If $s = 0$, go to Step 1.
5. Output the signature $\sigma := (x_R, s)$.

The verification of a signature $\sigma = (x_R, s)$ for a message m using a public key P proceeds as follows:

1. Check that $x_R \in \{0, \ldots, q-1\}$ and $s \in \{1, \ldots, n-1\}$. If the checks fail, output `False` and terminate.
2. Compute e such that $e \leftarrow \mathrm{Hash}(m, x_R)$.
3. Compute $R \leftarrow [s]G + [e]P$. If $R = \mathcal{O}$ or $\mathtt{y}(R)$ is odd, output `False` and terminate.
4. Output `True` if $\mathtt{x}(R) = x_R$, and `False` otherwise.

The Garaga library relies on an *extended* signature format. A signature (x_R, s) is extended as a triple (x_R, e, s) where $e := \mathrm{Hash}(m, x_R)$. Verifying an extended signature (x_R, e, s) is thus similar to verifying a standard signature, except that one checks that e is well-derived from m and x_R, instead of recomputing it. This design choice aims to externalize the hash verification. We describe in Algorithm 22 the verification algorithm that checks that an extended EC-Schnorr signature is valid, assuming that e is equal to $\mathrm{Hash}(m, x_R)$. The algorithm does not implement this check, and so the user needs to check this assumption by themselves to achieve a sound verification. Externalizing this check enables a partial (hash-independent) verification algorithm.

Algorithm 22 EC-Schnorr.PartialVerify

Input: The extended signature $\sigma = (x_R, e, s)$, the public key P_{pk}
Hint: Hint for the multi-scalar multiplication
Output: Indicate whether the signature σ is valid, **assuming that e is the correct hash of x_R and the message**.

```
if e, s ∉ {1, ..., n − 1} then
    return False
if y(P_pk) is odd, P_pk is not on the curve or P_pk = O then
    return False
R = [s]G − [e]P_pk, using the hinted multi-scalar multiplication of Section 2.2
if R = O then
    return False
if y(R) is odd then
    return False
if x(R) ≠ x_R then
    return False
return True
```

Elliptic Curve Digital Signature Algorithm – ECDSA. In what follows, we describe the ECDSA signature scheme as implemented in the Garaga library. Given a private key d and a message m, the signing algorithm proceeds as follows:

1. Compute z as the output of a hash function that takes m as input:
$$z \leftarrow \text{Hash}(m) \in \{1, \ldots, n-1\} \ ;$$
2. Sample a scalar k from the set $\{1, \ldots, n-1\}$;
3. Compute $R \leftarrow [k]G$;
4. Compute $r \leftarrow \mathtt{x}(R) \bmod n$. If $r = 0$, goto Step 1.
5. Compute $k_{inv} \leftarrow k^{-1} \bmod n$.
6. Compute $s \leftarrow k_{inv} \cdot (r \cdot d + z)$. If $s = 0 \bmod n$, goto Step 1.
7. Output the signature $\sigma := (r, s)$.

Then to verify a signature $\sigma = (r, s)$ of a message m using a public key P, it proceeds as follows:

1. Compute z as the output of a hash function that takes m as input:
$$z \leftarrow \text{Hash}(m) \ ;$$
2. Check that $r, s \in \{1, \ldots, n-1\}$. If the check fails, output `False` and terminate.
3. Compute $s_{inv} \leftarrow s^{-1} \pmod{n}$.
4. Compute $u_1 = s_{inv} \cdot z \pmod{n}$ and $u_2 = s_{inv} \cdot r \pmod{n}$.
5. Compute $R \leftarrow [u_1]G + [u_2]P$. If $R = \mathcal{O}$, output `False` and terminate.
6. Compute $v_R \leftarrow \mathtt{x}(R) \bmod n$.
7. Output `True` if $v_R = r$, and `False` otherwise.

We describe in Algorithm 22 the verification algorithm that checks that an EC-Schnorr signature is valid, assuming that z has been correctly derived from the message m. The algorithm does not compute z, the user needs to give z as input. Externalizing this computation enables a hash-independent verification algorithm. The signature also includes a value v which is the parity bit of $\mathtt{y}(R)$.

✎ **Question 2: Boolean in ECDSA signature**

What is the motivation for introducing the parity bit of $\mathtt{y}(R)$? It has no impact on the forgery security, while degrading the signature size. What is the role of the value v in the signature in ECDSA.PartialVerify? Our guess would be that v aims to ensure the unicity of a signature (without it, $(r, -s)$ would be a valid signature for the same

Algorithm 23 ECDSA.PartialVerify

Input: The extended signature $\sigma = (r, s, z, v)$, the public key P_{pk}
Hint: Hint for the multi-scalar multiplication
Output: Indicate whatever the signature σ is valid, **assuming that z is correctly derived from the message**.

```
if r, s ∉ {1, ..., n − 1} then
    return False
if P_pk is not on the curve or P_pk = O then
    return False
s_inv = s^{-1} (mod n)
u_1 = s_inv · z (mod n) and u_2 = s_inv · r (mod n)
R' = [u_1]G + [u_2]P_pk, using the hinted multi-scalar multiplication of Section 2.2
if R' = O then
    return False
if y(R')%2 ≠ v then
    return False
if (x(R') mod n) ≠ r then
    return False
return True
```

message) as in BIP 146 [LW16]. However, implemented as described in Algorithm 23, the value v does not ensure such a unicity.

2.4.2 Groth16

Here after, we will describe the verification procedure for the Groth16's proof system. We refer the interested readers to `https://www.rareskills.io/post/groth16` for details about the complete proof system. Groth16 is a proof system with a *per-circuit trusted setup*, meaning that a trusted third party needs to generate a pair of proving and verification keys which are related to the circuit instance to be proven. The proving key is out of the scope of this document. The verification key vk is of the form $(f, \gamma, \delta, \{\Psi_i\}_{i=1}^{\ell})$, where $\gamma, \delta \in \mathbb{G}_2$, $\Psi_1, \ldots, \Psi_\ell \in \mathbb{G}_1$ and $f \in \mathbb{F}_{q^k}^*$. A proof transcript is of the form $((a_i)_{i=1}^{\ell}, A, B, C)$ where $a_1, \ldots, a_\ell \in \mathbb{F}_q$, $A, C \in \mathbb{G}_1$ and $B \in \mathbb{G}_2$. Then, the verification procedure simply consists of computing:

$$X = \sum_{i=1}^{\ell} a_i \Psi_i \in \mathbb{G}_1$$

and checking that:

$$e(X, \gamma) \cdot e(C, \delta) \cdot e(A, B) \cdot f = 1 \ .$$

We formally describe the verification procedure in Algorithm 24. Let us stress that it relies on a tweaked version of the multi-pairing check described in Section 2.3.2. While the

original check verifies that the pairing product is equal to one, the tweaked check verifies that the pairing product times a *public* factor $f \in \mathbb{F}_{q^k}^*$ is equal to one. This is done by using an additional RFA relation.

Algorithm 24 Groth16.Verify

Input: The verification key $vk := \left(f, \gamma, \delta, \{\Psi\}_{i=1}^{\ell}\right)$, the proof transcript $\sigma := ((a_i)_{i=1}^{\ell}, A, B, C)$
Hint: Hints for the multi-scalar multiplication and the multi-pairing check
Output: Indicate whether the proof transcript σ is valid

1: Compute $X = \sum_{i=1}^{\ell} a_i \Psi_i \in \mathbb{G}_1$, using the hinted multi-scalar multiplication of Section 2.2.
2: Check that

$$e(X, \gamma) \cdot e(C, \delta) \cdot e(A, B) \cdot f = 1 \ ,$$

using the (tweaked) multi-pairing check of Section 2.3.2.

2.5 Code architecture

In this section, we provide an overview of the code architecture of the Garaga library, which is available in the folder `src/src/` of the GitHub repository.

- The folder `core` contains an additional layer to the Cairo's Core Library. It implements additional utilities to handle arithmetic circuits.
- The folder `definitions` contains definitions of most of the constants and data structures. It also implements methods to handle them.
  - The file `structs/fields.cairo` contains two templates of data structures to represent an element of the degree-12 field extension: one (`E12D`) when the field extension is defined as a quotient ring of the polynomial ring for a degree-12 irreducible polynomial, and the other (`E12T`) when the field extension is defined with a tower construction of degrees 2,3 and 2. While the Core Library already has a 384-bit integer type (for the arithmetic circuits) which is convenient for some curves like BLS12-381, the file `structs/fields.cairo` provides a 288-bit integer type which is more convenient for other curves like BN254. The file further provides methods to handle those data structures and some constants like the zero and one values for all the data types.
  - The file `structs/points.cairo` contains the definition of data structures for the $\mathbb{F}_q$-rational and $\mathbb{F}_{q^2}$-rational points (i.e. respectively elements of $\mathbb{G}_1$ and of $\mathbb{G}_2$), for the pairs of points of $\mathbb{G}_1 \times \mathbb{G}_2$, for the lines of $\mathbb{G}_2$, together with some constants representing the point at infinity in $\mathbb{G}_1$ and $\mathbb{G}_2$.
  - The file `curves.cairo` implements the data structure that represents an elliptic curve, together with hardcoded curves: `BN254`, `BLS12_381`, `SECP256K1`,

SECP256R1, ED25519, GRUMPKIN. It also provides a $\mathbb{G}_1$ generator for BN254, a $\mathbb{G}_2$ generator for BLS12_381, and the bit decomposition of the Miller loop's exponent for BN254 and BLS12_381.

  - The file struct.cairo contains additional utilities to handle the above data structure.

- The folder ec contains all the source code manipulating the objects relative to elliptic curves.
  - The file ec_ops.cairo implements the operations over the rational points in $\mathbb{G}_1$. It implements the curve membership test, the infinity test, the negation, the addition, the hashing, the ECIP check (a.k.a. multi-scalar multiplication),
  - The file ec_ops_g2.cairo implements the operations over the points in $\mathbb{G}_2$: the curve membership test, the negation, the addition, the scalar multiplication, and the equality test.
  - The file pairing/pairing_check.cairo implements the multi-pairing check over BN254 and BLS12_381 for two pairs of points, where the $\mathbb{G}_2$ points are known in advance by the verifier.
  - The file pairing/groth16.cairo implements the verification of a Groth16 proof for BN254 and BLS12_381. It involves a multi-pairing check for three pairs of points, where the two first $\mathbb{G}_2$ points are known in advance by the verifier.
  - The file pairing/single_pairing_tower.cairo implements the pairing computation over BN254 and BLS12_381, assuming that the extension field is defined using a tower construction.

- The folder signatures implements EC-based digital signature schemes.
  - The file ecdsa.cairo implements the ECDSA digital signature scheme, using the ECIP check (a.k.a. multi-scalar multiplication).
  - The file schnorr.cairo implements the Schnorr digital signature scheme, using the ECIP check (a.k.a. multi-scalar multiplication).

- The folder utils implements various utilities.
  - The file hashing.cairo contains some hash operations based the Hades permutation.
  - The file neg_3.cairo contains methods to handle the base-(-3) integer decomposition and the positive-negative decomposition.
  - The file calldata.cairo gathers most of the serialization-related methods of the library.
  - The files drand.cairo, noir.cairo, noir/honk_transcript.cairo, and risc0.cairo contain some application-dependent extension of the Garaga library. Those files are out-of-scope of the audit.

- The folder `circuits` provides hardcoded and auto-generated functions implementing arithmetic circuits. Those files are generated by the Hydra backend of the Garaga library.
  - The file `dummy.cairo` contains a dummy circuit, which is never used in the library.
  - The file `ec.cairo` contains several functions used in `ec_ops.cairo`. The Python implementation of those functions is available in the Hydra's file

    `compilable_circuits/common_cairo_fustat_circuits.py`.
  - The file `extf_mul.cairo` contains several functions used in `pairing/pairing_check.cairo` and `pairing/groth16.cairo`. The Python implementation of those functions is available in the Hydra's file

    `compilable_circuits/cairo1_mpcheck_circuits.py`.
  - The file `isogeny.cairo` contains a single function used in `utils/drand.cairo`. The Python implementation of this function is available in the Hydra's file

    `compilable_circuits/apply_isogeny.py`.
  - The file `multi_pairing_check.cairo` contains several functions used in `pairing/pairing_check.cairo`. The Python implementation of those functions is available in the Hydra's file

    `compilable_circuits/cairo1_mpcheck_circuits.py`.
  - The file `tower_circuits.cairo` contains several functions used in `pairing/single_pairing_tower.cairo`. The Python implementation of those functions is available in the Hydra's file

    `compilable_circuits/cairo1_tower_pairing.py`.
- The root folder contains additional source files.
  - The file `basic_field_ops.cairo` implements several functions dealing with the modular arithmetic.
  - The file `definitions.cairo` implements some serialization-related methods of the structures implemented in the folder `definitions`.
  - The file `hashes.cairo` contains a single function that implements a specific instance of the Poseidon hash function.

3 Code review

This section provides a detailed code review of the library. Section 3.1 focuses on definitions. Section 3.2 covers arithmetic operations. Section 3.3 delves into elliptic curve operations, including the multi-pairing check. Section 3.4 discusses signature schemes, while Section 3.5 covers additional utilities.

Due to the soundness issues with the ECIP method identified in Section 2.2, its implementation has become obsolete. As a result, we do not provide a low-level code review for this part of the code in the following.

3.1 Definitions

The folder `definitions` contains definitions of most of the constants and data structures of the Garaga library, together with few methods to handle them.

3.1.1 File `structs/fields.cairo`

The file `structs/fields.cairo` deals with the definitions of field-related structures.

Unsigned 288-bit integer. To handle the 256-bit fields, the file provides a structure `u288` that represents an unsigned 288-bit integer, defined as the concatenation of three unsigned 96-bit integers (`u96`) that are accessible through the attributes `limb0`, `limb1`, `limb2`. More precisely, the three integers `limb0`, `limb1`, `limb2` represent the integer

$$\texttt{limb0} + \texttt{limb1} \cdot 2^{96} + \texttt{limb2} \cdot 2^{192} \in \{0, \ldots, 2^{288} - 1\} \ .$$

Several functions enable the manipulation of this structure:

- Functions for serialization and deserialization of an unsigned 288-bit as a triple of `felt252` elements. Each unsigned 96-bit integer that composes the 288-bit integer is embedded in a `felt252` element.
- Functions implementing the trait `num::traits::Zero`, specifically, a getter function that outputs zero and two functions that test whether an integer is zero or non-zero, respectively.
- Functions implementing the trait `num::traits::One`, specifically, a getter function that outputs one and two functions that test whether an integer is one or different from one, respectively.

✎ **Question 3: Choice of `u288` over `u256`**

According to the Cairo documentation:

`https://docs.starknet.io/guides/cairo-101/integers/`,

there exists natively a type `u256` that represents an unsigned 256-bit integer. What is

the reason for defining a type `u288` instead of simply using `u256`?

Field element `E12D<T>`. The file provides a structure `E12D<T>` that represents a field element in a field extension $\mathbb{F}_{q^{12}}$ defined as a quotient ring of the polynomial ring for a degree-12 irreducible polynomial $p(w)$: $\mathbb{F}_{q^{12}} \cong \mathbb{F}_q[w]/p(w)$. The field element is naturally represented as

$$\begin{aligned}\mathtt{w0} + \mathtt{w1} \cdot w^1 + \mathtt{w2} \cdot w^2 + \mathtt{w3} \cdot w^3 + \mathtt{w4} \cdot w^4 + \mathtt{w5} \cdot w^5 \\ + \mathtt{w6} \cdot w^6 + \mathtt{w7} \cdot w^7 + \mathtt{w8} \cdot w^8 + \mathtt{w9} \cdot w^9 + \mathtt{w10} \cdot w^{10} + \mathtt{w11} \cdot w^{11}\end{aligned}$$

where the coefficients $\mathtt{w0}, \ldots, \mathtt{w11}$ are the 12 attributes of the structure. Each attribute is of type `T`.

Several functions enable the manipulation of this structure when the structure attributes are unsigned 288-bit integers (`u288`) or unsigned 384-bit integers (`u384`):

- Functions for the serialization and deserialization of a field element as a tuple of `felt252` elements of size 12×3 (for `u288`) or 12×4 (for `u384`). Each attribute is a unsigned 96-bit integer embedded in a `felt252` element.
- Functions implementing the trait `num::traits::One`, specifically, a getter function that outputs one and two functions that test whether an integer is one or different from one, respectively.

Field element `E12T`. The file provides a structure `E12T` that represents a field element in a degree-12 field extension defined with a tower construction of degrees 2, 3, and 2. Specifically, the extension field $\mathbb{F}_{q^{12}}$ is built as:

$$\mathbb{F}_{q^{12}} \cong \mathbb{F}_{q^6}[w]/p_3(w) \quad \text{with} \quad p_3(w) \in \mathbb{F}_{q^6}[w] \ ,$$

where $\mathbb{F}_{q^6} \cong \mathbb{F}_{q^2}[v]/p_2(v)$ with $p_2(v) \in \mathbb{F}_{q^2}[v]$ and $\mathbb{F}_{q^2} \cong \mathbb{F}_q[u]/p_1(u)$ with $p_1(u) \in \mathbb{F}_q[u]$. A field element is then represented as:

$$\begin{aligned}&\big[(\mathtt{c1b2a1} \cdot u + \mathtt{c1b2a0}) \cdot v^2 + (\mathtt{c1b1a1} \cdot u + \mathtt{c1b1a0}) \cdot v + (\mathtt{c1b0a1} \cdot u + \mathtt{c1b0a0})\big] \cdot w \\ &\quad + \big[(\mathtt{c0b2a1} \cdot u + \mathtt{c0b2a0}) \cdot v^2 + (\mathtt{c0b1a1} \cdot u + \mathtt{c0b1a0}) \cdot v + (\mathtt{c0b0a1} \cdot u + \mathtt{c0b0a0})\big]\end{aligned}$$

where the coefficients $\mathtt{c0b0a0}, \mathtt{c0b0a1}, \ldots$ are the 12 attributes of the structure. Each attribute is an unsigned 384-bit integer (`u384`).

Observation 4: Generic structure versus concrete structure

While `E12D<T>` is defined as a generic structure, `E12T` is defined as a concrete structure.

Recommendation

We recommend homogenizing the way the structures are defined. To maintain flexibility in the attribute type, we suggest using a generic structure for `E12T`.

Scaling factor for the Miller loop. As explained in Section 2.3.2, the multi-pairing check relies on a hint $u \in \mathbb{F}_{q^{k'}}$ that scales the results of the Miller loop. The structure `MillerLoopResultScalingFactor<T>` aims to represent such a scaling factor when the latter lives in the subfield $\mathbb{F}_{q^6}$ of $\mathbb{F}_{q^{12}}$, where `T` is the attribute type. It relies on the quotient ring representation (with a degree-12 irreducible polynomial) of the extension field. Using the same notation as for `E12D<T>`, an element of the subfield $\mathbb{F}_{q^6}$ can be represented using six attributes $\mathtt{w0}, \mathtt{w2}, \ldots, \mathtt{w10}$ that correspond to the element

$$\mathtt{w0} + \mathtt{w2} \cdot w^2 + \mathtt{w4} \cdot w^4 + \mathtt{w6} \cdot w^6 + \mathtt{w8} \cdot w^8 + \mathtt{w10} \cdot w^{10} \in \mathbb{F}_{q^6} \ .$$

Observation 5: Definition location

While it is defined in the `definitions` folder, the structure `MillerLoopResultScalingFactor<T>` is specific to the multi-pairing check. This structure is always used with the structures `MPCheckHintBN254` and `MPCheckHintBLS12_381` defined in the file `ec/pairing/pairing_check.cairo`.

Recommendation

We recommend moving this structure definition in the file `ec/pairing/pairing_check.cairo`.

The structure `E12DMulQuotient`. The file `fields.cairo` implements a generic structure `E12DMulQuotient<T>` that represents a degree-10 polynomial with 11 attributes for the underlying coefficients. It aims to be used as a hint polynomial for randomized field arithmetic while verifying degree-2 relations (e.g. multiplications) in a degree-12 field extension.

Observation 6: Structure definition

The advantage of defining a structure that represents the hint polynomial for the specific case of degree-2 relation is not clear, compared to directly implementing this polynomial as a coefficient array. For example, the RFA hint polynomial for checking the multi-pairing check is defined as `Array<u288>` or `Array<u384>` in the file `ec/pairing/pairing_check.cairo`. Moreover, this structure is used only in the Groth16 proof verification.

Recommendation

We recommend avoiding to define such structure since the advantage of defining it is not clear. Or, at least, we recommend to move this structure definition in the file `groth16.cairo`. Let us stress that following the recommendation of Observation 41 (●) would remove the only context where this structure was used.

3.1.2 File `structs/points.cairo`

The file `structs/fields.cairo` deals with the definitions of structures related to EC points.

$\mathbb{F}_q$-rational points. The file provides a structure `G1Point` that represents a $\mathbb{F}_q$-rational point, i.e. an element of the group $\mathbb{G}_1$. Such a point is represented with two coordinates `x` and `y`, stored in 384-bit variables (`u384`). Serializing a point consists in serializing the two `u384` attributes (using the functions `serialize_u384` and `deserialize_u384`). The trait `num::traits::Zero` is implemented for this structure: the point at infinity is represented by the couple of field element $(0, 0)$, which is a valid representation as soon as the parameter b in the Weierstrass equation of the curve is non-zero.

Let us mention that a specific instance of a point represented with `G1Point` is not associated to a curve. From a point instance, there is no direct way to identify the underlying curve. For this reason, each time an operation is performed on a point (e.g. point addition, scalar multiplication), the underlying function must further be provided with the curve parameters. This is done through an additional argument encoding the curve (or the identifier of a supported the curve).

● Observation 7: Curve identification in `G1Point`

As explained above, there is no way to identify the curve that corresponds to a given point.

Recommendation

We recommend adding a small attribute to the structure `G1Point` to identify the curve on which the point is defined. We think that such an attribute would simplify the code without computational overhead.

$\mathbb{F}_{q^2}$-rational points. The file provides a structure `G2Point` that represents a $\mathbb{F}_{q^2}$-rational point, i.e. an element of the group $\mathbb{G}_2$. Such a point can be represented with two elements of $\mathbb{F}_2$. In practice, the structure has four `u384` attributes $(\texttt{x0}, \texttt{x1}, \texttt{y0}, \texttt{y1})$ where $\texttt{x1} \cdot u + \texttt{x0}$ and $\texttt{y1} \cdot u + \texttt{y0}$ represent the x-coordinate and the y-coordinate of the point, respectively, where $\mathbb{F}_{q^2} \cong \mathbb{F}_q[u]/p(u)$. Serializing such a point consists in serializing the four

`u384` attributes (using the functions `serialize_u384` and `deserialize_u384`). The trait `num::traits::Zero` is implemented for this structure: the point at infinity is represented by the pair of field elements $(0, 0)$. Similarly than with `G1Point`, a specific instance of `G2Point` is not associated to a curve.

Observation 8: Curve identification in `G2Point`

As explained above, there is no way to identify the curve that corresponds to a given point.

Recommendation

We recommend adding a small attribute to the structure `G2Point` to identify the curve on which the point is defined. We think that such an attribute would simplify the code without computational overhead.

Pair of points in $\mathbb{G}_1 \times \mathbb{G}_2$. The file provides a structure `G1G2Pair` that represents a pair of points in $\mathbb{G}_1 \times \mathbb{G}_2$, with a `G1Point` attribute `p` and a `G2Point` attribute `q`.

Lines of $\mathbb{G}_2$. The file provides a structure `G2Line<T>` that represents a line of $\mathbb{G}_2$. Such a line can be represented with two elements of $\mathbb{F}_{q^2}$, the slope and the constant term. In practice, the structure has four attributes, `r0a0`, `r0a1`, `r1a0`, `r1a1`, where `r0a1` $\cdot\, u +$ `r0a0` and `r1a1` $\cdot\, u +$ `r1a0` are respectively the slope and the constant term on $\mathbb{F}_{q^2} \cong \mathbb{F}_q[u]/p(u)$.

Observation 9: Integer type in point structure definition

While the definitions of `G1Point`, `G2Point` and `G1G2Pair` use the integer type `u384` to represent field elements, the structure `G2Line<T>` does not specify the integer type (generic structure). Although the Garaga library currently only uses the former structures with `u384`, this may change in the future. For instance, one could adopt the `u256` or `u288` types to represent the BN254 field elements, potentially reducing the cost of hashing.

Recommendation

We recommend homogenizing these structure definitions, making them agnostic of the integer type everywhere.

3.1.3 File `curves.cairo`

The file `structs/fields.cairo` deals with the definitions of EC-curve-related structures.

Curve definition. The file provides a structure `Curve` that represents an elliptic curve using the short Weierstrass equation $E : y^2 = x^3 + ax + b$. The attribute `p` of type `u384` corresponds to the prime modulus (the order of the underlying prime field), while the attribute `n` of type `u256` corresponds to the order of the curve. Then the two attributes (`a`, `b`) of type `u384` are the two parameters that define the curve equation. The attribute `g` of type `u384` is a generator of the multiplicative group of the field, while the attribute `min_one` of type `u384` represents the field element -1. Finally, the attribute `G` of type `G1Point` is a (public) generator of the curve.

Six curves are hardcoded in the file: `BN254`, `BLS12_381`, `SECP256K1`, `SECP256R1`, `ED25519`, and `GRUMPKIN`. The parameters of those curves correspond to those in Section 2.1.

Observation 10: Incorrect comment

The comment that describes the attribute `g` of the structure `Curve` states that `g` is a `generator of Fp`. Instead of being a generator of the field, it is a generator of the multiplicative group of the field.

Recommendation

We recommend fixing this comment.

Curve getters. Together with the hardcoded curves, the file provides a list of functions enabling the recovery of curve parameters from a curve index. A curve index is a unique identifier that refers to one of the hardcoded curves. We summarize the hardcoded curves and their indexes in the following table:

Curve Index	Curve Name
0	BN254
1	BLS12_381
2	SECP256K1
3	SECP256R1
4	ED25519
5	GRUMPKIN

Observation 11: Incomplete comment

The list of the curve indexes given in a comment in the source code (line 35 of the file `curves.cairo`) is not up-to-date. The `GRUMPKIN` curve is missing.

Recommendation

We recommend updating this comment.

The implemented getter functions are the following:

- `get_p(curve_index)` returns the prime modulus of the selected curve, or zero if the curve index is invalid;
- `get_n(curve_index)` returns the order of the selected curve, or zero if the curve index is invalid;
- `get_a(curve_index)` returns the a parameter of the curve equation, or zero if the curve index is invalid;
- `get_b(curve_index)` returns the b parameter of the curve equation, or zero if the curve index is invalid;
- `get_b2(curve_index)` returns the b parameter of the twisted curve equation for `BN254` and `BLS12_381`, or raises an error for any other curves or when the curve index is invalid;
- `get_g(curve_index)` returns the generator of the multiplicative group of the base field of the selected curve, or zero if the curve index is invalid;
- `get_min_one(curve_index)` returns the field element -1 of the selected curve, or zero if the curve index is invalid;
- `get_G(curve_index)` returns the generator of the selected curve, or the point at infinity if the curve index is invalid;
- `get_modulus(curve_index)` returns the prime modulus of the selected curve as a `CircuitModulus` variable, or raises an error if the curve index is invalid;
- `get_curve_order_modulus(curve_index)` returns the order of the selected curve as a `CircuitModulus` variable, or raises an error if the curve index is invalid;

Observation 12: Function name

The function name `get_b2` is not explicit.

Recommendation

We recommend choosing a better name for the function, for example `get_b_twist`.

Observation 13: Hardcoded curve constants

Some curve parameters are hardcoded in dedicated structures, while others are hardcoded in the getter functions (b parameter of the twisted curve, prime modulus and curve order as `CircuitModulus` variables).

Recommendation

To simplify the maintenance of the library, we recommend gathering *all* the hardcoded curve constants in dedicated structures.

Observation 14: Inconsistent error handling

The functions `get_b2`, `get_modulus` and `get_curve_order_modulus` raise an error when the curve index is invalid, while the others return some default constants.

Recommendation

We recommend homogenizing the handling of errors across these functions.

Processed pairs. The file provides two structures `BLSProcessedPair` and `BNProcessedPair` that represent the processed pairs of points in the multi-pairing check.

In the context of a BLS curve, a processed pair of points $(P, Q) \in \mathbb{G}_1 \times \mathbb{G}_2$ is of the form:

$$\left(\frac{1}{\mathtt{y}(P)}, -\frac{\mathtt{x}(P)}{\mathtt{y}(P)}\right) \in \mathbb{F}_q^2 \ ,$$

which is represented by two attributes `yInv` and `xNegOverY`.

In the context of a BN curve, a processed pair of points $(P, Q) \in \mathbb{G}_1 \times \mathbb{G}_2$ is as a triple:

$$\left(\frac{1}{\mathtt{y}(P)}, -\frac{\mathtt{x}(P)}{\mathtt{y}(P)}, -\mathtt{y}(Q)\right) \in \mathbb{F}_q \times \mathbb{F}_q \times \mathbb{F}_{q^2} \ ,$$

which is represented by four attributes `yInv` and `xNegOverY`, `QyNeg0` and `QyNeg1` (where the two latter attributes represent the coordinate $-\mathtt{y}(Q)$).

Observation 15: Definition location

The two above structures are application-dependent, i.e. are only used for the multi-pairing check.

Recommendation

We recommend moving the definitions of those structures in the file `ec/pairing/pairing_check.cairo`, or at least to move them in the file `definitions/points.cairo`.

Additional constants. The file implements additional constants:

- The $\mathbb{G}_1$ generator of the BN254 curve (`BN254_G1_GENERATOR`);
- The $\mathbb{G}_2$ generator of the BLS12-381 curve (`BLS_G2_GENERATOR`);
- The square z^2 of the BLS12-381 seed `z` (`BLS_X_SEED_SQ`) and its decomposed form (`BLS_X_SEED_SQ_EPNS`), together with a third root of unity (`THIRD_ROOT_OF_UNITY_BLS12_381_G1`), used for the $\mathbb{G}_1$ subgroup membership.
- The decomposition of the Miller-loop exponent for the BN254 curve (`bn_bits`) and BLS12-381 curve (`bls_bits`).

Observation 16: Redundant definition of the $\mathbb{G}_1$ generator

The $\mathbb{G}_1$ generator for the BN254 curve is defined twice: in the structure `BN254` (attribute `G`) and as the constant `BN254_G1_GENERATOR`.

Recommendation

We recommend removing the constant `BN254_G1_GENERATOR`.

Observation 17: Hardcoded decomposition of the Miller-loop exponent

The file `curves.cairo` provides the decomposition of the Miller-loop exponent for two curves (`bn_bits` and `bls_bits`) in some specific bases. Similar decompositions are provided in the file `ec/pairing/single_pairing_tower.cairo` for other bases.

Recommendation

We recommend gathering all the useful decompositions in the file `curves.cairo`. We also recommend improving the constant names by using capital letters, properly indicating the underlying curve (`BN254` instead of just `bn`) and reflecting the underlying basis.

3.1.4 File `structs.cairo`

The file `structs.cairo` is the module file for the folder `definitions/structs`. Besides handling the import, this files declares how to cast a `felt252` variable into a `u96` variable.

Observation 18: Unused casting

The line that declares how to cast a `felt252` variable into an `u96` variable is not used.

Recommendation

We recommend removing this line.

3.1.5 File `definitions.cairo`

The file `definitions.cairo` is the module file for the folder `definitions`. It defines how `u384` variables should be serialized (and deserialized): the four 96-bit attributes of `u384` are embedded in four `felt252` elements. It also defines how arrays of `u384` variables should be serialized (by concatenation of the serialized elements).

Observation 19: Redundant code

The process to serialize and deserialize `u384` elements is implemented twice. The serialization is implemented in `serialize_u384` and in `u384Serde.serialize`, while the deserialization is implemented in `deserialize_u384` and in `u384Serde.deserialize`.

Recommendation

We recommend removing this redundancy.

Observation 20: Code organisation

The file `definitions.cairo` implements the (de)serialization for `u384` variables (and arrays of such variables). However, this does not seem to be the appropriate file for these utilities.

Recommendation

We recommend moving this code in the file `definitions/structs/fields.cairo`, since this file already implements the (de)serialization for `u288` variables. Otherwise,

it could be moved in a utility file (in `utils`).

3.2 Arithmetic operations

The source file `basic_field_ops.cairo` implements the basic arithmetic operations in a prime field. The field elements are represented as unsigned 384-bit integers (`u384`). this implies that the library supports field of orders at most 2^{384}. The file includes the following basic modular operations:

- the **modular negation** (`neg_mod_p`) that computes $-a \pmod p$ from two values a and p;
- the **modular addition** (`add_mod_p`) that computes $a+b \pmod p$ from three values a, b and p;
- the **modular subtraction** (`sub_mod_p`) that computes $a-b \pmod p$ from three values a, b and p;
- the **modular multiplication** (`mul_mod_p`) that computes $a \times b \pmod p$ from three values a, b and p;
- the **modular inverse** (`inv_mod_p`) that computes $1/a \pmod p$ from two values a and p;

Those functions take one or two operands as input, together with the modulus, and output the corresponding result. Those functions are implemented using Cairo's arithmetic circuits from the Core Library. If the modulus is stricly smaller than 2 or, in case of `inv_mod_p`, if the input value is non-invertible, the circuit evaluation returns an error.

The file `basic_field_ops.cairo` implements additional operations:

- The functions

  `compute_yInvXnegOverY_BN254` and `compute_yInvXnegOverY_BLS12_381`

  computes $(\frac{1}{y}, -\frac{x}{y})$ modulo p from a pair (x, y), where p is the field order of the underlying curve. Those functions are used in the multi-pairing check to preprocess the rational points in order to efficiently evaluate the lines in the Miller loop (c.f. Section 2.3.2);
- The function `is_even_u384` tests whether an unsigned 384-bit integer is even or not;
- The function `batch_3_mod_p` computes $x \cdot c_0 + y \cdot c_0^2 + z \cdot c_0^3 \pmod p$ from five values (x, y, z, c_0, p);
- The function (`u512_mod_bls12_381`) computes $(a \cdot 2^{256} + b) \pmod{p_{\text{BLS12-381}}}$ from two values (a, b), where $p_{\text{BLS12-381}}$ is the field order of `BLS12_381`. Both a and b are encoded by an `u32` array in big-endian representation.

Observation 21: Application-specific arithmetic functions

The functions `compute_yInvXnegOverY_BN254/BLS12_381` and `u512_mod_bls12_381` are dedicated to specific applications (and so have hardcoded modulus), while the other are not.

Recommendation

We recommend to move the functions `compute_yInvXnegOverY_BN254/BLS12_381` in the file `ec/pairing/pairing_check.cairo` and the function `u512_mod_bls12_381` in the file `utils/drand.cairo`.

Observation 22: Old commented code

The file contains a commented-out function, `run_BN254_EVAL_AND_HASH_E12D_circuit`.

Recommendation

We recommend removing this unnecessary commented code.

3.3 Elliptic curves

The folder `ec` contains the implementation of all the functions related to elliptic curves.

3.3.1 File `ec_ops.cairo`

The file `ec_ops.cairo` implements several functions to manipulate the $\mathbb{F}_q$-rational points of an elliptic curve (type `G1Point`):

- The function `is_infinity` tests whether a point $P \in \mathbb{F}_q^2$ is the point at infinity. It relies on the method `is_zero` of `G1Point`.
- The function `is_on_curve` tests whether a point P belongs to a given curve *and is different from the point at infinity*. The function relies on the Cairo function `run_IS_ON_CURVE_G1_circuit`, which is auto-generated using the Python method `BasicEC._is_on_curve_G1_weirstrass` of the Hydra file `ec.py`. This method simply checks the Weierstrass equation, namely it checks whether the coordinates satisfy $\mathtt{y}(P)^2 = \mathtt{x}(P)^3 + a \cdot \mathtt{x}(P) + b$. The function returns the corresponding Boolean.
- The function `assert_on_curve` performs the same check as the function `is_on_curve`, except that it raises an error when the input point does not belong to the provided curve or when it is the point at infinity.

- The function `negate` computes the negation $(x_P, -y_P)$ of a point $P = (x_P, y_P)$.
- The function `assert_in_subgroup` tests whether a point $P \in \mathbb{F}_q^2$ belongs to the prime subgroup of a given curve and raises an error when it does not. This function is only implemented for the BN254 curve and the BLS12-381 curve: calling this function with another curve raises an error. For the BN254 curve, it simply checks that the point belongs to the curve since the cofactor is equal to one. For the BLS12-381 curve, it uses endomorphims as described in [Bow19, Sco21], namely it checks that $\varphi(P) = -[\mathtt{z}^2]P$ where $\varphi : (x, y) \mapsto (w \cdot x, y)$ with w a cubic root of unity. The Garaga library implements the equivalent[3] check $[\mathtt{z}^2]\varphi'(P) = -P$ with $\varphi'(x, y) \mapsto (w^2 \cdot x, y)$, as implemented in the `gnark-crypto` library[4].
- The function `update_hash_state` updates a width-3 state of a sponge construction by hashing the two coordinates of the given point, assuming the last state coordinate corresponds to the sponge capacity. The underlying permutation is the width-3 `hades_permutation`.

Observation 23: Curve membership test

In the state of the art, testing that a point is on curve usually consists in testing that a point belongs to the given curve, including the point at infinity. While the Garaga library seems self-consistent with the choice of excluding the point at infinity, this is non-standard and prone to misinterpretation.

Recommendation

We recommend either modifying the function name to explicitly indicate that the point at infinity is excluded, or updating the functionality to include the point at infinity in the membership test, while using a separate test for the point at infinity where necessary.

Observation 24: Duplicate code

The function `assert_on_curve` include the code of the function `is_on_curve` which is duplicated.

Recommendation

[3] Since $\mathtt{z}^6 + 1$ is a multiple of the curve order, one has $[\mathtt{z}^6]P = -P$.

[4] https://github.com/Consensys/gnark-crypto/blob/ff4c0ddbe1ef37d1c1c6bec8c36fc43a84c86be5/ecc/bls12-381/g1.go#L492

We recommend calling the function `is_on_curve` inside the function `assert_on_curve` to avoid duplicating code.

● Observation 25: ECIP usage in subgroup membership test

The subgroup membership test (`assert_in_subgroup`) relies on the ECIP check described in Section 2.2 (`scalar_mul_g1_fixed_small_scalar`) to check $[\mathtt{z}^2]\varphi'(P) = -P$. However, as stated by Observation 2 (●), this method is not sound.

Recommendation

We recommend fixing Observation 2 (●), namely switching to a sound ECIP method.

EC addition. The function `ec_safe_add` computes the sum of two points P and Q, assuming that they belong to the curve. When $P = \mathcal{O}$ it returns Q and when $Q = \mathcal{O}$ it returns P. When $P = -Q$, it returns $\mathcal{O}$. When $P = Q$, the function relies on the arithmetic circuit implementing the doubling operation (as defined in Section 2.1.2). The latter is provided by the Cairo function `run_DOUBLE_EC_POINT_circuit` generated using the Python method `BasicEC.double_point` of the Hydra file `ec.py`. Otherwise, the function relies on the arithmetic circuit implementing the standard addition operation (as defined in Section 2.1.2). The latter is provided by the Cairo function `run_ADD_EC_POINT_circuit` generated using the Python method `BasicEC.add_points` of the Hydra file `ec.py`.

Random EC point sampling. The utility function `derive_ec_point_from_X` derives a random EC point P from a random input field element $x_0 \in \mathbb{F}_q$. It proceeds as follows:

1. Initialize a counter i to zero;
2. Compute a value y such that (x_i, y) is a valid EC point if such a point exists;
3. If there is no such point, hash the field element x_i and the counter i to deduce a new value x_{i+1}, increment the counter, and restart from Step 2;
4. Return the obtained point (x_i, y), along with a proof π that no points exist with an x-coordinate in $\{x_0, \ldots, x_{i-1}\}$.

The function `derive_ec_point_from_X` implements the verifier procedure, which receives the initial x_0, the final y coordinate, and the proof π. The function rederives the values $x_1, \ldots, x_i$ by hashing as in the procedure above. It efficiently checks that there is no point with an x-coordinate in $\{x_0, \ldots, x_{i-1}\}$ using the proof π, verifies that (x_i, y) is a valid point (using the Weierstrass equation), and then outputs the point (x_i, y). Since it only

checks the Weierstrass equation, this sampling procedure excludes the point at infinity and allows the prover to choose between the points (x, y) and $(x, -y)$ for a random field element x. To check that no valid point exists with an x-coordinate in $\{x_0, \ldots, x_{i-1}\}$, the verifier ensures that the term $x_j^3 + a \cdot x_j + b$ is not a square, for every $0 \leq j \leq i-1$. This is done by demonstrating the existence of w_j such that $w_j^2 = g \cdot (x_j^3 + a \cdot x_j + b)$, where g is a generator of $\mathbb{F}_q^*$. The proof π is then made of these values $(w_0, \ldots, w_{i-1})$ associated to $(x_0, \ldots, x_{i-1})$.

Observation 26: Optimization in the random point derivation

It is possible to optimize the function `derive_ec_point_from_X`. First, let us stress that (under a standard random oracle assumption for the hash function) it is not necessary to check that the previous x_j are not valid x-coordinates. In other words, one can remove the verification of the proof π while preserving soundness, which simplifies the code. Indeed, letting the adversary the possibility to try further x_j's after a working x_i does not lead to an attack, since other strategies to resample x_0 are possible anyway (by changing the input of the underlying Fiat-Shamir hash computation). As a result, the prover could simply give the final value of the counter to allow the verifier to derive x_i and check that (x_i, y) is a valid point. Moreover, instead of building x_{j+1} by hashing x_j and the counter value j, one could generate x_{j+1} by hashing x_0 and j. By doing so, the verifier just need to perform one hash operation, whatever of the counter value.

Recommendation

We recommend implementing those optimizations since they improve efficiency while simplifying the function.

Multi-scalar multiplications. The file `ec_ops.cairo` further provides three functions for multi-scalar multiplications:

- The function `scalar_mul_g1_fixed_small_scalar` aims to check that two points (P, Q) satisfy $Q = eP$, where a is an input unsigned 128-bit integer.
- The function `msm_g1` aims to check that a tuple of points $(Q, P_1, \ldots, P_n)$ and a tuple of scalars $(e_1, \ldots, e_n)$ satisfy the ECIP relation $Q = e_1 \cdot P_1 + \ldots + e_n \cdot P_n$.
- The function `msm_g1_u128` performs the same check than `msm_g1`, but assuming that the scalars are unsigned 128-bit integers.

Because of the issues raised in Observation 1 (●) and Observation 2 (●), we did not perform an in-depth code review of those functions.

3.3.2 File ec_ops_g2.cairo

The file `ec_ops_g2.cairo` implements several functions to manipulate the $\mathbb{F}_{q^2}$-rational points of an elliptic curve (type `G2Point`):

- The function `is_on_curve` tests whether a point $P \in \mathbb{F}_{q^2}^2$ belongs to the twist of a given curve *and is not the point at infinity*. The function relies on the Cairo function `run_IS_ON_CURVE_G2_circuit`, which is auto-generated using the Python method `BasicECG2._is_on_curve_G2_weirstrass` of the Hydra file `ec.py`. This method simply checks the Weierstrass equation of the twisted curve, namely it checks whether the coordinates satisfy $\mathtt{y}(P)^2 = \mathtt{x}(P)^3 + a' \cdot \mathtt{x}(P) + b'$ where (a', b') are the parameters of the twisted curve. The function returns the corresponding Boolean.
- The function `assert_on_curve` performs the same check as the function `is_on_curve`, except that it raises an error when the given point does not belong to the twisted curve or when it is the point at infinity.
- The function `negate` computes the negation $(x_P, -y_P)$ of a point $P := (x_P, y_P) \in \mathbb{G}_2$.

EC addition. The function `ec_safe_add` computes the sum of two points P and Q, assuming that

- they belong to the given curve,
- none of them is the point at infinity,
- they are not opposite, $P \neq -Q$, and
- the a parameter of the curve is zero (which is the case for BN254, BLS281-381, Secp256k1, Grumpkin).

When $P = Q$, the function relies on the Cairo function `run_DOUBLE_EC_POINT_G2_A_EQ_0_circuit` which implements the doubling operation as defined in Section 2.1.2. This function is auto-generated using the Python method `BasicEC.double_point_a_eq_0` of the Hydra file `ec.py`. Otherwise, the function relies on the Cairo function `run_ADD_EC_POINTS_G2_circuit` which implements the addition operation as defined in Section 2.1.2. This function is auto-generated using the Python method `BasicECG2.add_points` of the Hydra file `ec.py`.

Observation 27: Safe EC addition

While their names are similar, the safe EC addition functions (`ec_safe_add`) for $\mathbb{G}_1$ and $\mathbb{G}_2$ do not make the same assumptions about the input points. The $\mathbb{G}_1$ safe addition supports the point at infinity while the $\mathbb{G}_2$ safe addition does not. Moreover, the G_2 addition returns `None` when $P = -Q$, while the G_1 addition returns the point at infinity.

Recommendation

We recommend harmonizing the assumptions underlying the two safe EC addition functions. The current design suggests that the $\mathbb{G}_1$ safe addition is intended to be used in a standalone context, while the $\mathbb{G}_2$ safe addition appears to assume an internal usage within the current file. If the $\mathbb{G}_2$ function is indeed not intended for external use, we suggest renaming it to clearly indicate its internal scope and adding a brief comment to explicitly state that the function is not meant to be used outside the file.

EC scalar multiplication. The function `ec_mul` computes the scalar multiplication of a point P by a scalar s. First, the function treats the border cases:

1. it returns $\mathcal{O}$ if the input point is $\mathcal{O}$;
2. it returns `None` if the input point is not on the curve;
3. it returns $\mathcal{O}$ if the scalar is 0;
4. it returns the input point P if the scalar is 1.

Then, it gets the bit decomposition (in little-endian representation) of the scalar s using the function `get_bits_little` and calls the sub-routine `ec_mul_inner`. The latter computes the scalar multiplication using the relation $[s]P = \sum_j [b_j]\left([2^j]P\right)$, where $\{b_j\}$ are the bit decomposition of s. In the kth loop iteration, the routine computes:

- $\sum_{j\leq k}[b_j]\left([2^j]P\right) \leftarrow [b_k]\left([2^k]P\right) + \sum_{j<k}[b_j]\left([2^j]P\right)$ using the function `ec_safe_add`,
- $[2^{k+1}]P \leftarrow [2]([2^k]P)$ using the auto-generated Cairo function `run_DOUBLE_EC_POINT_G2_A_EQ_0_circuit`.

Observation 28: Completeness of the $\mathbb{G}_2$ EC multiplication

The function `ec_mul` checks that the input point belongs to the twisted curve but it does not check that it belongs to the r-torsion subgroup. Therefore, the input point P could be of order 2^j implying that $[2^j]P = \mathcal{O}$. As a result, `ec_mul` would call the `run_DOUBLE_EC_POINT_G2_A_EQ_0_circuit` function on the point at infinity, which would fail. In summary, one can find inputs for the `ec_mul` function that fails without returning `Option::None`.

Recommendation

We recommend updating the function and documenting its expected behavior. Several options are possible:

1. Assuming that the input point belongs to the subgroup (in that case, one does not need to check the curve membership);
2. Checking that the input point belongs to the subgroup;

3. Supporting scalar multiplication for points of the twist outside the subgroup.

The last option is probably the safest and only requires to add some checks in the implementation (i.e. to handle the cases when the intermediary points are the point at infinity).

Observation 29: Hardcoded curve

The function `ec_mul` takes the curve index as input but currently only works correctly for BLS12-381. This is because the tests used to distinguish between the cases $P = Q$, $P = -Q$, and $P \notin \{Q, -Q\}$ rely on the subroutines `eq_mod_p` and `eq_neg_mod_p`, which in turn use modular operations with the field order of the BLS12-381 curve as modulus.

Recommendation

We recommend updating the tests to avoid relying on a hardcoded prime modulus. Instead, the correct field order should be selected dynamically based on the `curve_index`.

3.3.3 File `pairing/pairing_check.cairo`

The source file `pairing/pairing_check.cairo` implements two functions, namely `multi_pairing_check_bn254_2P_2F` and `multi_pairing_check_bls12_381_2P_2F`, that aim to check that two pairs of points $(P_0, Q_0), (P_1, Q_1) \in \mathbb{G}_1 \times \mathbb{G}_2$ satisfy the relation

$$e(P_0, Q_0) \cdot e(P_1, Q_1) = 1 \ ,$$

over the BN254 curve and the BLS12-381 curve respectively. Those functions follow the algorithms described in Section 2.3.2, up to some details that we present below.

The structures `MPCheckHintBN254` and `MPCheckHintBLS12_381` correspond to the hints for the multi-pairing checks. They contain the λ-th root $c \in \mathbb{F}_{q^k}$ of the Miller loop output (`lambda_root`), its inverse $c^{(\mathrm{inv})} = c^{-1}$ (`lambda_root_inverse`), the scaling factor $u \in \mathbb{F}_{q^{k'}}$ (denoted `w` in the code), some intermediate values $r_0, \ldots, r_{L+2} \in \mathbb{F}_{q^k}$ of the Miller loop accumulator (`Ris`), and the hint polynomial $Q(X)$ (`big_Q`). In the case of the BLS12-381 curve, the hint does not include the λ-root c (only its inverse), because the multi-pairing check does not need this value. This is due to the special form of λ for BLS12-381 ($\lambda = -\mathtt{z} + q$, see Section 2.3.1).

The two functions closely follow the algorithm MultiPairingCheck (described in Section 2.3.2). After checking that the number of intermediate values of the Miller loop accumulator and the degree of the hint polynomial are correct (with respect to the numbers given in Tables 3 and 4), they precompute the points P_0' and P_1' from the points

P_0 and P_1 using the functions `compute_yInvXnegOverY_BN254` and `compute_yInvXnegOverY_BLS12_381`. Then, they derive the RFA challenges:

- the batching challenge β is computed by hashing the input pairs, the λ-th root c and its inverse, the scaling factor u, and the intermediate values $\{r_i\}_i$;
- the evaluation challenge z is computed by hashing the hint polynomial $Q(X)$.

The hashing operations are performed with a sponge-based approach using functions of the module `hashing`. The initial state of the second hashing operation corresponds to the final state of the first hashing operation.

Observation 30: Non-standard definition of sponge-based XOF

The challenges are derived using an extendable-output hash function (XOF) using the sponge mode in a non-standard way. As explained above, the initial state of the second XOF operation corresponds to the final state of the first XOF operation. This is equivalent to use a single sponge-based XOF, in which

1. one absorbs input data,
2. one squeezes the state to get some output,
3. one absorbs again additional input data, and
4. one squeezes again to get some additional output.

The sponge construction was not designed to be used in this manner. While it is not clear whether this usage would lead to an attack, it deviates from the standard application of the construction and therefore falls outside the scope of existing security analysis results.

Recommendation

We recommend using two distinct instances of sponge-based XOF for the two challenges. In that case, the second hash operation should explicitly take β as input.

Observation 31: Invalid domain separation in XOFs

Some domain separation is implemented in the XOFs. For example, in `multi_pairing_check_bn254_2P_2F`, the width-3 state of the first sponge construction is initialized with

```
('MPCHECK_BN254_2P_2F', 0, 1) ,
```

where `'MPCHECK_BN254_2P_2F'` is a short string literal (that represents a single field element). The field element that corresponds to the capacity is the last coordinate, and so is initialized with `1`. The domain separation is provided by `'MPCHECK_BN254_2P_-`

`2F'`, however this value is put in the part of the internal state where the hash input will be absorbed (i.e. the state part that corresponds to the rate). This constant simply shifts the value of the first hashed field elements and does *not* provide secure domain separation.

Recommendation

We recommend initializing the state part that corresponds to the rate with zeroes and using the domain separator to initialize the state part that corresponds to the capacity (the last coordinate of the state in the current implementation).

The functions then call the auto-generated subroutines `run_BN254_MP_CHECK_PREPARE_LAMBDA_ROOT_circuit` and `run_BLS12_381_MP_CHECK_PREPARE_LAMBDA_ROOT_circuit` that implement the algorithm InitializeMultiPairingCheck (Algorithm 16). Those subroutines are based on the Python class `MPCheckPrepareLambdaRootEvaluations` of the file `cairo1_mpcheck_circuits.py`. In the case of BLS12-381, since the λ-th root is not part of the hints (only its inverse), the subroutine does not check that $c \cdot c^{(\mathrm{inv})} = 1$ and does not need to output `lhs`. Therefore, the variable `rels` in the main function will be initialized upon its next use.

Observation 32: Ambiguous function name

The function name "prepare lambda root" is ambiguous since it does not build the λ-th root but performs some precomputation related to it.

Recommendation

We recommend choosing another function name, for example InitializeMultiPairingCheck as used in our presentation (see Section 2.3).

Observation 33: Confusing variable usage

In the method `_execute_circuit_logic` of the class `MPCheckPrepareLambdaRootEvaluations`, a variable `c_or_c_inv` is used to refer to c for the BN254 curve and to $c^{(\mathrm{inv})}$ for the BLS12-381. This usage of the same variable to refer two different notions depending on the context is confusing.

Recommendation

We recommend duplicating the code and specializing each copy for a specific curve, rather than relying on a factorized version, which is not justified in this context.

This approach improves code readability without adding unnecessary complexity. If factorization is desired, we recommend limiting it to the computation over $c^{(\mathrm{inv})}$, which is shared by both curves.

After computing the c-related values, the functions run the first iterations of the Miller loop by calling the auto-generated subroutines `run_BN254_MP_CHECK_INIT_BIT_2P_2F_circuit` and `run_BLS12_381_MP_CHECK_INIT_BIT_2P_2F_circuit` that implement the algorithm MML-FirstIterations. Those subroutines are based on the Python class `FixedG2MPCheckInitBit` of the file `cairo1_mpcheck_circuits.py`. Then, it runs the (remaining iterations of the) Miller loop. For each iteration, it calls an auto-generated Cairo function that implements MML-BatchedIterations for a specific set of bits.

The multi-pairing check functions finalize their computation by calling the auto-generated subroutines `run_BN254_MP_CHECK_FINALIZE_BN_2P_2F_circuit` and `run_BLS12_381_MP_CHECK_FINALIZE_BLS_2P_circuit` that implement the algorithm CheckFinalize. Those subroutines are based on the Python classes `FixedG2MPCheckFinalizeBN` and `MPCheckFinalizeBLS` of the file `cairo1_mpcheck_circuits.py`. Moreover, the functions check that the last intermediate value r_{L+2} of the pairing computation equals 1. The functions could be optimized by hardcoding this variable. However, keeping it configurable allows for a generic framework that supports cases where the pairing output is not 1, but another public value (e.g., in Groth16 proof verification).

Observation 34: Large number of function arguments

The calls to most of the subroutines have a long list of arguments. For example, the functions

`run_BN254_MP_CHECK_INIT_BIT_2P_2F_circuit`,
`run_BN254_MP_CHECK_BIT00_2P_2F_circuit`,
`run_BN254_MP_CHECK_BIT10_2P_2F_circuit`,
`run_BN254_MP_CHECK_BIT01_2P_2F_circuit`, and
`run_BN254_MP_CHECK_FINALIZE_BN_2P_2F_circuit`

takes respectively 11, 13, 16, 16 and 19 arguments. Having such long lists of arguments degrades the readability of the source code.

Recommendation

We recommend finding strategies to mitigate this. For example, one could use a data structure `mml_data` as in Algorithm 21.

Observation 35: First iterations of the Miller loop

There is a dedicated subroutine that handles the first iterations of the Miller loop.

Having such a dedicated function degrades the readability of the code source and increases the code redundancy. Moreover, it is not clear to us whether this leads to computational improvements.

Recommendation

We recommend refactoring the code to avoid having such a dedicated function, or at least to avoid redundant code.

Observation 36: Code redundancy for batched iterations

There is a lot of redundant code to deal with all the possible cases in the Python class `BaseFixedG2PointsMPCheck` in the file `cairo1_mpcheck_circuits.py`. Moreover, the code is not flexible as it supports only some specific sets of batched iterations (those for which the underlying bits are `00`, `01`, `10`, `0` and `1`).

Recommendation

We recommend factorizing the code to avoid redundancy and to support any set of batched iterations.

Observation 37: Heavy handling of the batching data

Currently, each time a subroutine needs to aggregrate a new RFA relation with the prior ones, it takes the previous batching coefficient β_{i-1}, computes the current one as $\beta_i = \beta_{i-1}^2$ and output this new batching coefficient. This implies keeping track of the current batching coefficient.

Recommendation

We recommend simplifying this aggregation process. Let us denote `lhs` the batched previous relations and `rel` the new relation to aggregate. We propose to compute the new batched value as

$$\texttt{new_lhs} = \beta \cdot \texttt{lhs} + \texttt{rel} \ .$$

This approach has the advantage that the subroutine only needs to take β as input (which remains constant throughout the multi-pairing check) and does not need to return any output. This simplifies the current API of the subroutines. Additionally, this solution addresses the issue raised in Observation 3 (●).

Observation 38: Confusing variable names

In the file `cairo1_mpcheck_circuits.py`, the variable `line` sometimes contains line evaluations instead of actual lines. For instance, this is the case for the method `_multiply_line_evaluations` of `BaseFixedG2PointsMPCheck` and the method `_execute_circuit_logic` of `FixedG2MPCheckFinalizeBN`.

Recommendation

We recommend updating the name of those variables.

3.3.4 File `pairing/groth16.cairo`

Multi-pairing check. This file `pairing/groth16.cairo` implements two functions, namely

`multi_pairing_check_bn254_3P_2F_with_extra_miller_loop_result` and
`multi_pairing_check_bls12_381_3P_2F_with_extra_miller_loop_result`.

These functions aim to check that three pairs of points $(P_0, Q_0), (P_1, Q_1), (P_2, Q_2) \in \mathbb{G}_1 \times \mathbb{G}_2$ satisfy the relation

$$e(P_0, Q_0) \cdot e(P_1, Q_1) \cdot e(P_2, Q_2) \cdot f = 1$$

for some public value f, over the BN254 curve and the BLS12-381 curve, respectively. Those functions follow the algorithms described in Section 2.3.2 and their implementation is very similar to the functions of the file `pairing/pairing_check.cairo`. The differences compared to the latter functions are the following:

- The initial state of the first sponge-based XOF is defined as the output of a permutation call:

  ```
  hades_permutation(int.from_bytes(b'MPCHECK_BN254_3P_2F', "big"), 0, 1)
  ```

  instead of `('MPCHECK_BN254_2P_2F', 0, 1)`;
- The number of pairings is three instead of two, which has no impact on the algorithm procedure besides increasing the computational overhead;
- The pairing product is checked to equal a public value f^{-1} instead of 1;
- The point Q_2 is dynamic, meaning that we cannot rely on precomputation for this point (while Q_0 and Q_1 are predefined).

Observation 39: Non-standard definition of sponge-based XOF

Similar to Observation 30 (●), the initial state of the second and third XOF operations correspond to the final states of previous XOF operations.

Recommendation

We recommend using three distinct XOF instances, for which the inputs of a XOF instance include the output of the previous one.

Observation 40: Non-standard domain separation in XOFs

While the implemented method for initializing the state of the sponge-based XOF does provide domain separation, a more standard approach involves injecting the domain separation constant directly into the capacity part of the state.

Recommendation

We recommend adopting this more standard method for domain separation.

While Algorithm 21 (MultiPairingCheck) verifies that a product of pairings equals one, the implemented multi-pairing checks instead ensure that the pairing product equals a known public value f^{-1}. This is done by first multiplying the pairing product by f, and then checking that the result equals one. To perform this check, the implementation uses a dedicated instantiation of the randomized field arithmetic. Specifically, it employs a hint polynomial and a challenge value to validate the equation. As there is only a single equation to verify, batching is not used, so the challenge consists solely of the evaluation point, which is derived by hashing the public constant and the hint polynomial.

Observation 41: Double instances of the randomized field arithmetic

As described above, to verify the final product in the multi-pairing check, the function uses a second instance of the randomized field arithmetic (the first one is used to verify the rest of the multi-pairing check).

Recommendation

We recommend relying on a single instance of the randomized field arithmetic, by checking the final relation to the previous ones. This would result in a simpler and more efficient implementation: there is no need to compute hash operations to derive the RFA challenge (saving computation), to receive a second hint polynomial (reducing

communication), or to perform the RFA check twice (simplifying the code).

Observation 42: Redundant code

The functions `multi_pairing_check_bn254_3P_2F_with_extra_miller_loop_result` and `multi_pairing_check_bls12_381_3P_2F_with_extra_miller_loop_result` follow the same pattern as the functions `multi_pairing_check_bn254_2P_2F` and `multi_pairing_check_bls12_381_2P_2F`, respectively, up to small differences.

Recommendation

We recommend factorizing the code based on a generic implementation of the multi-pairing checks. This is compatible with using auto-generated subroutines according to the input parameters.

Groth16 proofs. The file `pairing/groth16.cairo` implements the verification algorithm for the Groth16 proof system, as explained in Section 2.4.2. Two instances, `verify_groth16_bn254` and `verify_groth16_bls12_381`, of the verification algorithm are available for the curves BN254 and BLS12-381 respectively.

The structure `Groth16VerifyingKey` corresponds to the verification key of the proof system: it contains the values $\gamma \in \mathbb{G}_2$ (`gamma_g2`), $\delta \in \mathbb{G}_2$ (`delta_g2`) and $f \in \mathbb{F}_{q^k}$ of the form $f := e(\alpha, \beta)$ (`alpha_beta_miller_loop_result`). By design choice, the structure does not include the values $\{\Psi_i\}_{i=1}^{\ell}$ since ℓ is not fixed. Then the structure `Groth16Proof` corresponds to the Groth16 proof to verify; it contains the points $A \in \mathbb{G}_1$, $B \in \mathbb{G}_2$ and $C \in \mathbb{G}_1$, together with the scalars $(a_i)_{i=1}^{\ell}$ (`public_inputs`). A structure `Groth16ProofRaw` is also defined which represents a proof composed of the points (A, B, C), without the scalars $(a_i)_{i=1}^{\ell}$.

Observation 43: Application-dependent structure

The data structure `Groth16ProofRaw` is only used in the application related to `Risc0`.

Recommendation

We recommend moving the definition of this structure in the file `utils/calldata.cairo`.

The functions `verify_groth16_bn254` and `verify_groth16_bls12_381` implement the algorithm Groth16.Verify (Algorithm 24). These functions compute $X = \sum_{i=1}^{\ell} a_i \Psi_i$ using `msm_g1`, check that the points in the proof belongs to the curves (excluding the point at infinity) and run the multi-pairing check.

● Observation 44: ECIP usage in Groth16 proof verification

To verify the computation of the term $\sum_{i=1}^{\ell} a_i \Psi_i$, the Groth16 verification algorithm relies on the function `msm_g1` which uses the ECIP check described in Section 2.2. However, as stated by Observation 1 (●) and Observation 2 (●), the function `msm_g1` is not sound.

Recommendation

Fixing Observation 1 (●) and Observation 2 (●) would fix this observation.

● Observation 45: Subgroup membership

The functions `verify_groth16_bn254` and `verify_groth16_bls12_381` check that the points that compose a proof transcript belong to the curves, but they do not check that the points belong to the sub-groups $\mathbb{G}_1$ (for A and C) and $\mathbb{G}_2$ (for B). Such a check should be performed to ensure that the proof verification is sound (see `https://github.com/zcash/zcash/pull/1938` for exemple).

Recommendation

We recommend to check that the points inside a proof transcript indeed belong to the r-torsion group (i.e. that they satisfy $[r]P = \mathcal{O}$).

3.3.5 File `pairing/single_pairing_tower.cairo`

The file `pairing/single_pairing_tower.cairo` implements functions for the *Miller loop* and the *final exponentiation* of a single pairing computation for the curves BN254 and BLS12-381.

Structure `E12T`. The file provides functions that implements the trait `num::traits::One` for the structure `E12T`, namely a getter that outputs one and two functions that test whether an integer is one or different from one respectively.

■ Observation 46: Function location

While the data structure `E12T` is defined in the file `definitions/structs/fields.cairo`, the trait `num::traits::One` is implemented in this file. This trait is usually implemented in the same file than the data structure. For example, both `E12D` and its implementtation for the trait `num::traits::One` are implemented in the file `defini-`

tions/structs/fields.cairo.

Recommendation

We recommend implementing those functions in the same file as the data structure definition as in the rest of the library.

BN254 curve. The function `miller_loop_bn254_tower` implements the computation of the Miller loop for the BN254 curve, as in Algorithm 5 without the embedded c exponentiation and with signed scalar decomposition with digits in $\{-1, 0, 1\}$.

The function `final_exp_bn254_tower` implements the final exponentiation step of the pairing computation on the BN254 curve. While the standard exponent used is typically $\frac{q^k-1}{r}$, it is sometimes advantageous to use a multiple of this value, namely $s \cdot \frac{q^k-1}{r}$ for some s such that $r \nmid s$. Although this increases the size of the exponent, it provides flexibility in choosing an exponent that can reduce computational complexity. This optimization strategy is employed in `final_exp_bn254_tower`, which computes the map $x \in \mathbb{F}_{q^k} \mapsto x^{s \cdot \frac{q^k-1}{r}}$, using the specific choice $s := 2\mathtt{z} \cdot (6\mathtt{z}^2 + 3\mathtt{z} + 1)$, as described in [DG15, Alg. 6]. The implementation relies on the subroutine `expt_bn254_tower`, which computes $x^{\mathtt{z}}$ for elements $x \in \mathbb{F}_{q^k}$, assuming that x lies in the cyclotomic subgroup. It is important to note that this optimization leads to computing a power of the pairing. As noted in [DG15], "*this is not a problem because a fixed power of a pairing is also a pairing. But it can be a disadvantage if we implement a standard pairing. In case of interoperability and compatibility requirements, this method could be avoided.*"

BLS12-381 curve. The function `miller_loop_bls12_381_tower` implements the computation of the Miller loop for the BLS12-381 curve as in Algorithm 5 without the embedded c exponentiation.

Question 4: Unclear conjugaison operation

A conjugation operation is performed over the result of the Miller loop at the end of the `miller_loop_bls12_381_tower` function (line 478):

```
return fp12_conjugate(Mi, 1);
```

While it does not impact the soundness, this operation impacts the definition of the underlying pairing scheme, which deviates from the pairing definition of Theorem 1. Why is this conjugation operation inserted here?

The function `final_exp_bls12_381_tower` implements the final exponentation of the pairing computation of the BN254 curve. Similarly than for the BN254 curve, given a field element $x \in \mathbb{F}_{q^k}$, it does not compute $x^{\frac{q^k-1}{r}}$, but it computes $x^{3 \cdot \frac{q^k-1}{r}}$ as proposed in

[HHT20]. The implementation uses the subroutine `expt_bls12_381_tower` that computes $x^{\mathbf{z}}$ from a field element $x \in \mathbb{F}_{q^k}$, assuming that x belongs to the cyclotomic subgroup. This subroutine is optimized thanks to the Karabina's squaring method [Kar10], adapted for the field of the BLS12-381 curve as in the `gnark-crypto` library. Karabina's method [Kar10] relies on a compressed representation of elements of the cyclotomic subgroup $G_{\Phi_{12}}(q)$. Specifically, there exist a compression function $\mathcal{C}$ and a decompression function $\mathcal{D}$ such that $\mathcal{C}(g) \in \mathbb{F}_{q^2}^4$ and $\mathcal{D}(\mathcal{C}(g)) = g$ for any $g \in G_{\Phi_{12}}(q) \subset \mathbb{F}_{q^{12}}$. This result is formalized in the following proposition, which is an adaptation of [Kar10, Th. 3.1] for the BLS12-381 curve.

Proposition 3.1 *Let $\mathbb{F}_q$ be the prime field of the BLS12-381 curve. Let $g = (g_0 + g_1 \cdot v + g_2 \cdot v^2) + (g_3 + g_4 \cdot v + g_5 \cdot v^2) \cdot w \in G_{\Phi_{12}(q)} \backslash \{1\}$. Define the compression function $\mathcal{C}$ and the decompression function $\mathcal{D}$ as follow*

$$\begin{aligned} \mathcal{C}(g) &= [g_1, g_2, g_3, g_5], \\ \mathcal{D}([\tilde{g}_1, \tilde{g}_2, \tilde{g}_3, \tilde{g}_5]) &= (\tilde{g}_0 + \tilde{g}_1 \cdot v + \tilde{g}_2 \cdot v^2) + (\tilde{g}_3 + \tilde{g}_4 \cdot v + \tilde{g}_5 \cdot v^2) \cdot w, \end{aligned}$$

where

$$\begin{cases} \tilde{g}_4 = \frac{u \cdot \tilde{g}_5^2 + 3 \cdot \tilde{g}_1^2 - 2 \cdot \tilde{g}_2}{4 \cdot \tilde{g}_3}, & \text{if } \tilde{g}_3 \neq 0 \\ \tilde{g}_4 = \frac{2 \cdot \tilde{g}_1 \cdot \tilde{g}_5}{\tilde{g}_2}, & \text{if } \tilde{g}_3 = 0 \\ \tilde{g}_0 = u \cdot (2 \cdot \tilde{g}_4^2 + \tilde{g}_3 \cdot \tilde{g}_5 - 3 \cdot \tilde{g}_2 \cdot \tilde{g}_1) + 1. \end{cases}$$

Then $\mathcal{D}$ is well-defined for all $\mathcal{C}(g)$ with $g \in G_{\Phi_{12}(q)} \backslash \{1\}$, and $\mathcal{D}(\mathcal{C}) = g$ for all $g \in \mathbb{G}_{\Phi_{12}(q)} \backslash \{1\}$.

The interest of considering compressed field elements $\mathcal{C}(g)$ is that we can compute the compressed square $\mathcal{C}(g^2)$ directly from $\mathcal{C}(g)$, as stated by the following proposition (adaptation of [Kar10, Th. 3.2] for the BLS12-381 curve).

Proposition 3.2 *Let $h = g^2$, where $h = (h_0 + h_1 \cdot v + h_2 \cdot h^2) + (h_3 + h_4 \cdot v + h_5 \cdot v^2) \cdot w$. Then*

$$\mathcal{C}(g^2) = [h_1, h_2, h_3, h_5],$$

where

$$\begin{aligned} h_1 &= 3 \cdot g_3^2 + 3 \cdot E \cdot g_2^2 - 2 \cdot g_1, \\ h_2 &= 3 \cdot u \cdot g_5^2 + 3 \cdot g_1^2 - 2 \cdot g_2, \\ h_3 &= 6 \cdot u \cdot g_1 \cdot g_5 + 2 \cdot g_3, \\ h_5 &= 6 \cdot g_3 \cdot g_2 + 2 \cdot g_5. \end{aligned}$$

The above proposition implies that we can efficiently compute arbitrary powers of g. This property is leveraged to accelerate the function `expt_bls12_381_tower`, where Karabina's decompression function $\mathcal{D}$ is implemented as `decompress_karabina_bls12_381`, and

the squaring operation on a compressed field element is handled by the auto-generated subroutine `run_BLS12_381_E12T_CYCLO_SQUARE_COMPRESSED_circuit`. The latter is derived from the Python class `E12TCyclotomicSquareCompressedCircuit` in the file `cairo1_tower_pairing.py`.

Observation 47: Representation of $G_{\Phi_{12}(q)}$ elements

In the functions `expt_half_bls12_381_tower` and `decompress_karabina_bls12_381`, the compressed elements of $G_{\Phi_{12}(q)} \subset \mathbb{F}_{q^{12}}$ are stored using the structure for standard field elements `E12T`. As a result, some attributes of the structure are not used. When calling the sub-routine `decompress_karabina_bls12_381`, the function `expt_half_bls12_381_tower` assign some values to those attributes, which are never used, which is confusing. On the other hand, the subroutine `run_BLS12_381_E12T_CYCLO_SQUARE_COMPRESSED_circuit` takes a compressed element as input as 4 field elements of $\mathbb{F}_{q^2}$ instead of a `E12T` instance.

Recommendation

We recommend defining a dedicated structure, `CompressedE12T`, to represent compressed field elements from the cyclotomic subgroup, and using this structure consistently wherever such elements are manipulated.

Observation 48: Optimized squaring in the cyclotomic subgroup

In the function `expt_half_bls12_381_tower`, the initial squaring operations are performed using Karabina's compressed form, while the final squarings are carried out on the uncompressed form. This follows the design adopted in the `gnark-crypto` library. Our assumption is that this design choice stems from the high cost of the decompression step, particularly due to the division it involves. Since the field element must eventually be decompressed, the benefits of using the compressed form may be outweighed by the cost of the final decompression when the number of squarings is relatively low. However, the context here is different: when using arithmetic circuits in Cairo, the cost of an inversion (or division) is comparable to that of a multiplication, as we operate within a verification paradigm. As a result, employing the optimized squaring operation should consistently yield performance benefits for Garaga.

Recommendation

Provided that the above intuition on computational cost is confirmed, we recommend using the optimized squaring operation uniformly throughout the library.

3.4 Signatures

The folder `signatures` provides implementations of the verification algorithms of two EC-based signature schemes: EC-Schnorr in the file `schnorr.cairo` and ECDSA in the file `ecdsa.cairo`.

3.4.1 File `schnorr.cairo`

The file `signature/schnorr.cairo` implements the function `is_valid_schnorr_signature` that corresponds to the verification algorithm of the EC-Schnorr signature scheme, described in Algorithm 22 (EC-Schnorr.PartialVerify) in Section 2.4.1. This function takes as input a data structure of type `SchnorrSignatureWithHint` that contains:

- the public key (a point $P = (\mathtt{x}(P), \mathtt{y}(P))$) and the signature (x_R, e, s), both inside a structure `SchnorrSignature`, and
- the hints for the multi-scalar multiplication.

It also takes the identifier of the underlying elliptic curve. As explained in Section 2.4.1, `is_valid_schnorr_signature` does not implement the full verification algorithm, as it assumes that the hash e is correctly derived from x_R and the message.

The file `signature/schnorr.cairo` also implements a serialization method for the data structure `SchnorrSignature`.

Observation 49: Undocumented assumption

The verification function assumes that the hash e is correctly derived from x_R and the message. This is not explicitly mentioned in the function docstring which simply states that the function "implements the verification equation $sG - eP = R$".

Recommendation

We recommend explicitly mentioning the assumption in the function docstring. We also recommend modifying the function name `is_valid_schnorr_signature` (for example, `is_valid_schnorr_signature_assuming_hash`) to avoid any misinterpretation.

Observation 50: Incorrect structure comment

The comment that describes the data structure `SchnorrSignature` indicates that all the structure attributes are `u256` variables, while it is not the case.

Recommendation

We recommend fixing this comment.

Observation 51: Same structure for signature and public key

The public key and the signature are represented using the same data structure (`SchnorrSignature`) in the verification function `is_valid_schnorr_signature`. Since a public key is used to verify many signatures produced with the corresponding private key, it usually corresponds to a specific argument of the verification function.

Recommendation

We recommend separating the public key from the data structures `SchnorrSignature` and `SchnorrSignatureWithHint`.

Observation 52: ECIP usage in Schnorr verification

To compute the term $[s]G - [e]P_{\mathsf{pk}}$, the verification algorithm relies on the function `msm_g1` which uses the ECIP check described in Section 2.2. However, as stated by Observation 1 (●) and Observation 2 (●), the function `msm_g1` is not sound.

Recommendation

Fixing Observation 1 (●) and Observation 2 (●) would fix this observation.

Observation 53: Incorrect check for the point at infinity

As explained in Algorithm 22, the verification function checks that $R \neq \mathcal{O}$. In practice, this check is performed by checking $\mathtt{x}(R) \neq 0$ and $\mathtt{y}(R) \neq 0$. As a result, the check rejects points with zero coordinates that are not the point at infinity $\mathcal{O}$, such as, for instance, $(0, 2)$ on the BLS12-381 curve.

Recommendation

We recommend using the dedicated checks implemented in `ec/ec_ops.cairo`.

Observation 54: Useless instruction

The function has the instruction

```
let rx:  u384 = rx.into();
```

while `rx` was already of type `u384` (see the definition of the data structure `SchnorrSignature`).

Recommendation

We recommend removing this instruction.

3.4.2 File `ecdsa.cairo`

The file `signature/ecdsa.cairo` implements the function `is_valid_ecdsa_signature` that corresponds to the verification algorithm for the ECDSA signature scheme, described in Algorithm 23 (ECDSA.PartialVerify) in Section 2.4.1. This function takes as input a data structure of type `ECDSASignatureWithHint` that contains

- the public key (a point $P = (\mathtt{x}(P), \mathtt{y}(P))$) and the extended signature (r, s, z, v), both inside a structure `ECDSASignature`;
- the hints for the multi-scalar multiplication.

It also takes the identifier of the underlying elliptic curve. As explained in Section 2.4.1, `is_valid_ecdsa_signature` does not implement the full verification algorithm, as it externalizes the hash computation and takes the hashed message z as argument.

Observation 55: Same structure for signature and public key

The public key and the signature are represented using the same data structure (`ECDSASignature`) in the verification function `is_valid_ecdsa_signature`. Since a public key is used to verify many signatures produced with the corresponding private key, it usually corresponds to a specific argument of the verification function.

Recommendation

We recommend separating the public key from the data structures `ECDSASignature` and `ECDSASignatureWithHint`.

Observation 56: ECIP usage in ECDSA verification

To compute the term $[u_1]G + [u_2]P_{\mathsf{pk}}$, the verification algorithm relies on the function `msm_g1` which uses the ECIP check described in Section 2.2. However, as stated by Observation 1 (●) and Observation 2 (●), the function `msm_g1` is not sound.

Recommendation

Fixing Observation 1 (●) and Observation 2 (●) would fix this observation.

Observation 57: Incorrect check for the point at infinity

As explained in Algorithm 22, the verification function checks that $R' \neq \mathcal{O}$. In practice, this check is performed by checking $\mathtt{x}(R) \neq 0$ and $\mathtt{y}(R) \neq 0$. As a result, the check rejects points with zero coordinates that are not the point at infinity $\mathcal{O}$, such as, for instance:

$$(0, \mathtt{0x66485c780e2f83d72433bd5d84a06bb6541c2af31dae871728bf856a174f93f4})$$

on the Secp256r1 curve.

Recommendation

We recommend using the dedicated checks implemented in `ec/ec_ops.cairo`.

Observation 58: Inconsistent comment

According to the comment in the code, the sanity checks consists in checking that "r, s are non-zero and less than n". However, it actually cheks that $s, z \in \{1, \ldots, n-1\}$ and $r \neq 0$.

Recommendation

We recommend updating the checks and the comment. Checking the range of z seems useless for security. Moreover, the check that $r < n$ is implicitly done latter in the verification algorithm as one checks that r is equal to some value smaller than n.

Observation 59: Modular reduction function

At the end of the verification algorithm, $\mathtt{x}(R')$ modulo n is computed. This is done using the function `add_mod_p` by adding zero (modulo n) to $\mathtt{x}(R')$.

Recommendation

We recommend implementing a dedicated function to reduce a value modulo n in the file `basic_field_ops.cairo`.

3.5 Utilities

3.5.1 File `utils.cairo`

The module file `utils.cairo` implements three helper functions related to the variable type `u384`:

- `u384_assert_zero` ensures that the input value is zero;
- `u384_assert_eq` ensures that the two input values are equal;
- `u384_eq_zero` checks if the input value is zero.

3.5.2 File `hashes.cairo`

The source file `hashes.cairo` implements a hash function over the BN254's scalar field (or equivalently over the Grumpkin's field), named `poseidon_hash_2_bn254`, that compresses two field elements into a single one. The code of this function relies on an arithmetic circuit, that uses the constants in `POSEIDON_GRUMPKIN_GRUMPKIN_CONSTANTS`. The arithmetic circuit is auto-generated, thanks to the Python file `poseidon_bn254.py`. It relies on an internal state of width 3, with a capacity of 1 (the last state coordinate), and uses 8 full rounds and 57 partial rounds. The used S-box is the permutation $x \mapsto x^5$, which is a natural choice since $3 \mid (q_{\mathsf{Grumpkin}} - 1)$ and $5 \nmid (q_{\mathsf{Grumpkin}} - 1)$.

3.5.3 File `utils/hashing.cairo`

The source file `utils/hashing.cairo` implements many absorption functions for hashing, depending on the nature of the absorbed data. All those functions follow the same pattern: they take a width-3 state and the data to be hashed, they absorb the input data using the permutation `hades_permutation` and they return the updated width-3 state. The implemented functions are the following:

- `hash_u384_transcript` absorbs a list of `u384` variables;
- `hash_u288_transcript` absorbs a list of `u288` variables;
- `hash_E12DMulQuotient_u384` absorbs a degree-10 polynomial with coefficients of type `u384`;
- `hash_E12DMulQuotient_u288` absorbs a degree-10 polynomial with coefficients of type `u288`;
- `hash_E12D_u384` absorbs an element of a degree-12 field extension, where an element of the prime field is represented using the type `u384`;

- `hash_E12D_u288` absorbs an element of a degree-12 field extension, where an element of the prime field is represented using the type `u288`;
- `hash_MillerLoopResultScalingFactor_u384` absorbs a scaling factor involved in a pairing check (see Section 2.3.2), where an element of the prime field is represented using the type `u384`;
- `hash_MillerLoopResultScalingFactor_u288` absorbs a scaling factor involved in a pairing check (see Section 2.3.2), where an element of the prime field is represented using the type `u288`;
- `hash_E12D_u384_transcript` absorbs a list of elements of a degree-12 field extension, where an element of the prime field is represented using the type `u384`;
- `hash_E12D_u288_transcript` absorbs a list of elements of a degree-12 field extension, where an element of the prime field is represented using the type `u288`;
- `hash_G1G2Pair` absorbs a couple of points living in $\mathbb{G}_1 \times \mathbb{G}_2$, represented by the structure `G1G2Pair`;
- `hash_G1Point` performs a full hash of a point living in $\mathbb{G}_1$, represented by the structure `G1Point`.

Let us stress that the hashing operations are performed using arithmetic operations over a 252-bit field. The values of type `u384` and `u288` are thus transformed as `felt252` variables before being hashed.

Observation 60: Function API

While all the other hashing functions take the current hash state and return an updated one, the function `hash_G1Point` just takes a data to hash and returns the corresponding hash digest.

Recommendation

Since `hash_G1Point` is never used in the library and since such a hash operation is already defined in the file `ec/ec_ops.cairo`, we recommend removing this function.

Observation 61: Data structure for hash states

The file `utils/hashing.cairo` implements a structure `PoseidonState` that represents a width-3 hash state. However, this structure is never used. All the hashing functions takes three `felt252` variables as arguments and return a triple of such variables.

Recommendation

We recommend using the structure `PoseidonState` for the inputs and outputs of the hash (absorbing) functions.

Observation 62: Optimized hashing with `u288`

Hashing `u288` variables is as expensive as hashing `u384` variable in the current implementation, because such a variable is represented as a pair of `felt252`. In the case of a `u384` variable, the 192 least significant bits are stored in the first `felt252` variable while the 192 most significant bits are stored in the second one. In the case of a `u288` variable, the 192 least significant bits are stored in the first `felt252` variable while the 96 most significant bits are stored in the second one. An easy optimization would consists in using three `felt252` variables to represent two `u288` values, which would decrease the hash computational overhead by 1/3.

Recommendation

We recommend optimizing the hash operations with `u288`.

3.5.4 File `utils/neg_3.cairo`

The source file `utils/neg_3.cairo` implements several utility functions about the basis-(-3) decomposition and the positive-negative decomposition.

- The function `sign` takes a `felt252` value –namely a value in $\{0, \ldots, q_{\mathsf{stark}} - 1\}$– and it returns 1 if it is less than or equal to $(q_{\mathsf{stark}} - 1)/2$ and returns -1 otherwise.
- The function `sign_to_u384` formats the output of the previous function as a element of the field $\mathbb{F}_q$ of the given curve in a `u384` variable. While the field representation of 1 is always the same, the field representation of -1 depends on the curve since it is encoded as $q - 1$. In pratice, the field representation of -1 of all the implemented curve is directly available through the getters provided by the file `definitions/curves.cairo`.
- The function `neg_3_base_le` computes the base-(-3) decomposition $\vec{d} = (d_0, d_1, \ldots)$ of a `u128` value x, in the little-endian order: $x := \sum_{i \geq 0} d_i \cdot (-3)^i$. For the value 0, the decomposition is a tuple containing a single zero. For all the other values, the last term of the decomposition is a non-zero coefficient.
- Given a `u128` value x, the function `scalar_to_epns` computes the base-(-3) decomposition $\vec{d} = (d_0, d_1, \ldots)$ of x and outputs the tuple $(|x^+|, |x^-|, \texttt{sign}(x^+), \texttt{sign}(x^-))$ of `felt252` values, where $x^+ := \sum_{i \geq 0, d_i = 1} (-1)^i$ and $x^- := \sum_{i \geq 0, d_i = -1} (-1)^i$ such that $x = x^+ - x^-$.

- Given a `u128` value x and its base-(-3) decomposition $\vec{d} = (d_0, d_1, \ldots)$, the function `scalar_to_epns_with_digits` outputs the $(|x^+|, |x^-|, \texttt{sign}(x^+), \texttt{sign}(x^-))$ of `felt252` values, where $x^+ := \sum_{i \geq 0, d_i = 1} (-1)^i$ and $x^- := \sum_{i \geq 0, d_i = -1} (-1)^i$ such that $x = x^+ - x^-$. It checks that the provided decomposition is indeed the correct decomposition of x.

- Given a list of `u128` values $\{x_i\}_i$ and their base-(-3) decompositions, the function `u128_array_to_epns` outputs the positive-negative decomposition $(|x_i^+|, |x_i^-|, \texttt{sign}(x_i^+), \texttt{sign}(x_i^-))$ of x_i for all i. The input basis-(-3) decompositions can be omitted. If it is the case, the positive-negative decompositions are computed using `scalar_to_epns`. If the basis-(-3) decompositions are provided, the output decompositions are computed using the routine `scalar_to_epns_with_digits`, which also tests that the provided decompositions are correct.

- The function `u256_array_to_low_high_epns` takes as input a list of `u256` values $\vec{x} := (x_1, \ldots, x_\ell)$. Let us denote x_i^L and x_i^H respectively the 128 least significant bits and the 128 most significant bits of x_i such that $x_i := x_i^H \cdot 2^{128} + x_i^L$. The function also takes as input the basis-(-3) decompositions of x_i^L and x_i^H for all i. It returns two arrays: the first one contains the positive-negative decomposition $(|x_i^{L,+}|, |x_i^{L,-}|, \texttt{sign}(x_i^{L,+}), \texttt{sign}(x_i^{L,-}))$ of x_i^L for all i, while the second one contains the positive-negative decomposition $(|x_i^{H,+}|, |x_i^{H,-}|, \texttt{sign}(x_i^{H,+}), \texttt{sign}(x_i^{H,-}))$ of x_i^H for all i. The input basis-(-3) decompositions can be omitted. If it is the case, the positive-negative decompositions are computed using `scalar_to_epns`. If the basis-(-3) decompositions are provided, the output decompositions are computed using the routine `scalar_to_epns_with_digits`, which also tests that the provided decompositions are correct.

Observation 63: Sanity check on the input basis-(-3) decompositions

The functions `u128_array_to_epns` and `u256_array_to_low_high_epns` support omitting the input basis-(-3) decompositions. When decompositions are provided, a default behavior is implemented in cases where the number of decompositions differs from the number of values to decompose. If there are fewer decompositions than values, the functions automatically recompute the missing basis-(-3) decompositions. If there are more decompositions than values, the excess decompositions at the end are ignored. This way how the input basis-(-3) decompositions are handled does not seem clean.

Recommendation

When the array containing the decompositions is provided, we recommend checking that it is of the *same size* than the array that contains the values to decompose. If the aim is to support cases where some basis-(-3) decompositions are not pro-

vided, we recommend defining them explicitly as `None`. More precisely, the type of the decomposition array would be `Option<Span<Option<Span<felt252>>>>`, instead of `Option<Span<Span<felt252>>>`.

3.5.5 File `utils/calldata.cairo`

The file `utils/calldata.cairo` implements several serialization-related functions. First, it provides three data structures `FullProofWithHintsBN254`, `FullProofWithHintsBLS12_381` and `FullProofWithHintsRisc0`. The two first structures represent a Groth16 proof with hints for BN254 and BLS12-381, respectively, while the last structure is a Groth16 proof tailored to a specific application (RISC Zero). It then implements two utility functions `downcast_u288` and `downcast_u384` that transform a tuple of `felt252` values (representing unsigned 96-bit integers) into a `u288` value (for BN254) or a `u384` value (for BLS12-381). The main functions `deserialize_full_proof_with_hints_bn254`, `deserialize_full_proof_with_hints_bls12_381`, and `deserialize_full_proof_with_hints_risc0` consist in transforming a list of `felt252` elements into an instance of the data structure `FullProofWithHintsBN254`, `FullProofWithHintsBLS12_381` or `FullProofWithHintsRisc0`. Let us mention that the function `deserialize_full_proof_with_hints_bls12_381` uses a subroutine `deserialize_mpcheck_hint_bls12_381` to deserialize the hints of the multi-pairing checks, while the other functions do not use such subroutines.

■ Observation 64: Separation between utility and application-dependent functions

There is no clear separation between the utility functions of the Garaga library and the application-dependent functions. For example, in the folder `utils`, the files `utils/hashing.cairo` and `utils/neg_3.cairo` implements helpers for some functions of the library, while the files `utils/noir.cairo` and `utils/risc0.cairo` are tailored to specific applications. Moreover, the file `calldata.cairo` contains serialization-related functions for both the generic Groth16 proof system described in `ec/pairing/groth16.cairo` and the application RISC Zero. This means that a user wishing to use the deserialization function of the Groth16 proof system for their own application must rely on a source file partly belonging to an unrelated application.

Recommendation

We recommend clearly separating the helper functions of the Garaga library and the application-dependent functions. For example, there could be a dedicated folder `app` containing all the application-dependent files.

References

[BKLS02] Paulo S. L. M. Barreto, Hae Yong Kim, Ben Lynn, and Michael Scott. Efficient algorithms for pairing-based cryptosystems. In Moti Yung, editor, *Advances in Cryptology – CRYPTO 2002*, volume 2442 of *Lecture Notes in Computer Science*, pages 354–368, Santa Barbara, CA, USA, August 18–22, 2002. Springer Berlin Heidelberg, Germany.

[BLS03] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. Constructing elliptic curves with prescribed embedding degrees. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *SCN 02: 3rd International Conference on Security in Communication Networks*, volume 2576 of *Lecture Notes in Computer Science*, pages 257–267, Amalfi, Italy, September 12–13, 2003. Springer Berlin Heidelberg, Germany.

[BN06] Paulo S. L. M. Barreto and Michael Naehrig. Pairing-friendly elliptic curves of prime order. In Bart Preneel and Stafford Tavares, editors, *SAC 2005: 12th Annual International Workshop on Selected Areas in Cryptography*, volume 3897 of *Lecture Notes in Computer Science*, pages 319–331, Kingston, Ontario, Canada, August 11–12, 2006. Springer Berlin Heidelberg, Germany.

[Bow19] Sean Bowe. Faster subgroup checks for BLS12-381. Cryptology ePrint Archive, Report 2019/814, 2019.

[DG15] Sylvain Duquesne and Loubna Ghammam. Memory-saving computation of the pairing final exponentiation on BN curves. Cryptology ePrint Archive, Report 2015/192, 2015.

[Eag22a] Liam Eagen. `ecip_calculations.ipynb`, 2022. `https://gist.github.com/Liam-Eagen/666d0771f4968adccd6087465b8c5bd4`.

[Eag22b] Liam Eagen. Zero knowledge proofs of elliptic curve inner products from principal divisors and weil reciprocity. Cryptology ePrint Archive, Report 2022/596, 2022.

[HHT20] Daiki Hayashida, Kenichiro Hayasaka, and Tadanori Teruya. Efficient final exponentiation via cyclotomic structure for pairings over families of elliptic curves. Cryptology ePrint Archive, Report 2020/875, 2020.

[Kar10] Koray Karabina. Squaring in cyclotomic subgroups. Cryptology ePrint Archive, Report 2010/542, 2010.

[LW16] Johnson Lau and Pieter Wuille. Bip146: Dealing with signature encoding malleability. BIPS, 2016. `https://github.com/bitcoin/bips/blob/master/bip-0146.mediawiki`.

[NE24] Andrija Novakovic and Liam Eagen. On proving pairings. Cryptology ePrint Archive, Report 2024/640, 2024.

[Sco19] Michael Scott. Pairing implementation revisited. Cryptology ePrint Archive, Report 2019/077, 2019.

[Sco21] Michael Scott. A note on group membership tests for G_1, G_2 and G_T on BLS pairing-friendly curves. Cryptology ePrint Archive, Report 2021/1130, 2021.

[Sil09] Joseph H. Silverman. *The Arithmetic of Elliptic Curves*. Graduate texts in mathematics. Springer, 2009. 2nd edition.

[Ver10] Frederik Vercauteren. Optimal pairings. *IEEE Trans. Inf. Theory*, 56(1):455–461, 2010.

[WNR20] Pieter Wuille, Jonas Nick, and Tim Ruffing. Bip340: Schnorr signatures for secp256k1. BIPS, 2020. `https://bips.xyz/340`.

A Attack test vector

```
q = 16453374226060185169115816459822154444259013373939385337466919286373701242819

E = Elliptic Curve defined by y^2 = x^3 + 7 over Finite Field of size
↪  16453374226060185169115816459822154444259013373939385337466919286373701242819

P1 = (11666948528491623156829972546395796079737445919577881819828312910977116825362 :
↪  14191341157043118312599010003813758966688752045882168117719216846269565205918 : 1)

P2 = (8526780166717039978375957467105921598277417003817488357122128576454125212114 :
↪  11278793636990495912888352690239063009594477202179798728455113884747115841943 : 1)

e1 = -54794069906441464102274107416612850 80

e2 = 90549395054625533496154092466074530 35

Q = (12158379362665585203770451183825870850495056048477488629366964941483881086 3 :
↪  14401313570541738157628476837681821303755498981853231913755487170209126234 19 : 1)

hat_e1 = 16453374226060185169115816459822154444253533966948741191056691875632039957739

hat_e2 = e2

hat_Q = (10636800013640729479425016074152879453725018268282953168589709313336650844609 :
↪  98822018098632753706114424695480856522329698539511050009517484648684820281 61 : 1)

SumDLog = ((7927607442498349821895643145502187422404719226460810644607805863272355615850*x^3 +
↪  6060379601897553165624771156678886279296374683417332641112496609017848979178*x^2 +
↪  1294181024824175340983353419241496810062827560281747350913527579745468006318 0*x +
↪  1166720045334620879464604559244642031728788029168221947782523650019574521275 3)/(x^6 +
↪  1259143596328505135098799839430433250199821326399862561168972743590132163729 9*x^5 +
↪  5626020538171482125004543645354863098598932453399486334014602566641303506562*x^4 +
↪  1300552529397956276448030572885111615097759028049681289909201650604450014056 9*x^3 +
↪  5873180612694433611336906461019555292692425978293452594493495619440745246998*x^2 +
↪  6475395315080004536800172597839732801674500425917633663168379393741722060296*x +
↪  8771805927556013505783057802847040835548065093780763606309519110442994769839))*y +
↪  (36142283869889460193193408945288374 30*x^2 +
↪  1176390017168183011462307955342710345069358737237913304947235657454435847349 5*x +
↪  1434177165264951274164851607005372335431638432906371257899152060118138553459 0)/(x^3 +
↪  1259143596328505135098799839430433250199821326399862561168972743590132163729 9*x^2 +
↪  5626020538171482125004543645354863098598932453399486334014602566641303506562*x +
↪  1300552529397956276448030572885111615097759028049681289909201650604450014056 2)

hat_SumDLog =
↪  ((8265649923683612718552824588675757216328507660506470912984492013087673006093*x^3 +
↪  9106456639378939457491959793276539511852747767363597687803519566878737690666*x^2 +
↪  3148387089576996562940025011512057453438834809877814255563627202974421161081*x +
↪  6294258301731457698786401883873847528523694652895642485887946767467394890156)/(x^6 +
↪  2076219743270977723600686831989711756778145556200447329393687771979509603553*x^5 +
↪  3823967882221686797158573469527246949284487335320905214326018492326678077290*x^4 +
↪  1434342410688727062938670597409656239380296848347558129876092674204871198351 2*x^3 +
↪  1453353820289684406520480782392798229744701889340313130575581440385656722487 1*x^2 +
↪  1031440094949162241099419782686857420073239797330695116281521015991304529821 1*x +
↪  1683723391849783391012043059743010091066699140692757066524971476098776427621))*y +
↪  (36142283869889460193193408945288374 30*x^2 +
↪  7047773043772313055827932537800841595585561298357545304263586819006857033777*x +
↪  3503924654683785039543110368815626850639562027577529163341077630906462372 19)/(x^3 +
↪  2076219743270977723600686831989711756778145556200447329393687771979509603553*x^2 +
↪  3823967882221686797158573469527246949284487335320905214326018492326678077290*x +
↪  1434342410688727062938670597409656239380296848347558129876092674204871198350 5)
```

```
A0 = (3552171586098075668091394808220862662220431191367658810228900791426743966588 :
↪  7416722947369645156902160874492294367891980041520932349277916945531856704555 : 1)

basisSum = 9331770280471073999155138532970274696763231615747212779792605341433789552462

hat_basisSum = 9331770280471073999155138532970274696763231615747212779792605341433789552462

LHS = 2468706279707616237848361883436348797822310104124009680036213319930488687830

RHS = 2468706279707616237848361883436348797822310104124009680036213319930488687830
```